

Applications, injections, surjections, bijections et composition de fonctions

Charles EDOU NZE

Table des matières

0.0.1	1. Présentation du concept clé	1
0.0.2	2. Formalisation	1
0.0.3	3. Démonstrations	2
0.0.4	4. Exercices d'Application	2
0.0.5	5. Application en Intelligence Artificielle	3
0.0.6	6. Liens Sémantiques	3
1	Exercices d'application	4
1.1	Exercice 1	4
1.1.1	Exercice 1/10	4
1.2	Exercice 2	8
1.2.1	Exercice 2/10 : Injectivité et Surjectivité de Fonctions Réelles Simples	8
1.3	Exercice 3	13
1.4	Exercice 3/10 - Jalon 5 : Composition et propriétés des fonctions	13
1.5	Exercice 4	17
1.6	Exercice 4/10 : Jalon 5 - Applications, injections, surjections, bijections et composition de fonctions	17
1.6.1	Énoncé	17
1.6.2	Analyse de l'énoncé	17
1.6.3	Correction exhaustive "pas-à-pas"	17
1.6.4	Liens avec l'Intelligence Artificielle	18
1.7	Exercice 5	20
1.8	Exercice 5/10 : Jalon 5 - Applications, injections, surjections, bijections et composition de fonctions	20
1.9	Exercice 6	23
1.10	Exercice 6/10 : Construction d'une bijection entre $\mathbb{N}$ et $\mathbb{Z}$	23
1.10.1	Énoncé	23
1.10.2	Analyse de l'énoncé	23
1.10.3	Correction exhaustive pas-à-pas	24
1.10.4	Liens avec l'Intelligence Artificielle	25
1.11	Exercice 7	27
1.12	Exercice 7/10 : Images directes et images réciproques d'ensembles	27
1.13	Exercice 8	32
1.13.1	Exercice 8/10 : Injectivité et existence d'une fonction inverse à gauche	32
1.14	Exercice 9	36
1.15	Exercice 9/10 : Surjectivité et existence d'une fonction inverse à droite .	36
1.15.1	Énoncé Rigoureux	36
1.15.2	Analyse de l'Énoncé	36
1.15.3	Correction Exhaustive Pas-à-Pas	37

1.15.4	Liens avec l'Intelligence Artificielle	39
1.16	Exercice 10	41
1.17	Exercice 10/10 : Le Théorème de Cantor	41
1.17.1	Énoncé de l'Exercice	41
1.17.2	Analyse de l'Énoncé	41
1.17.3	Correction Exhaustive Pas-à-Pas	42
1.17.4	Liens avec l'Intelligence Artificielle	43
2	Travaux Pratiques	45
2.1	TP 1	45
2.2	TP 1/5 - Jalon 5 : Injectivité et Surjectivité sur Ensembles Finis	45
2.2.1	Introduction	45
2.2.2	Objectifs du TP	45
2.2.3	Prérequis	46
2.2.4	Partie 1 : Représentation d'une fonction	46
2.2.5	Partie 2 : Vérification de l'injectivité	47
2.2.6	Partie 3 : Vérification de la surjectivité	49
2.2.7	Partie 4 : Vérification de la bijectivité (Bonus)	51
2.2.8	Conclusion	53
2.3	TP 2	54
2.4	TP 2/5 : Composition de Fonctions - Implémentation et Vérification Em- pirique	54
2.4.1	Introduction	54
2.4.2	Objectifs du TP	54
2.4.3	Partie 1 : Rappels Mathématiques sur la Composition de Fonctions	54
2.4.4	Partie 2 : Implémentation de la Fonction <code>compose(f, g)</code>	55
2.4.5	Partie 3 : Vérification Empirique et Tests Unitaires (avec <code>assert</code>)	56
2.4.6	Consignes de Rendu	61
2.4.7	Critères d'Évaluation	61
2.5	TP 3	62
2.6	Travail Pratique (TP) 3/5 : Calcul de l'Image Réciproque d'un Sous- Ensemble	62
2.6.1	Introduction	62
2.6.2	Rappel Mathématique : L'Image Réciproque	62
2.6.3	Énoncé du Problème	62
2.6.4	Implémentation en Python	63
2.6.5	Justification Mathématique de l'Implémentation	67
2.6.6	Conclusion	68
2.7	TP 4	69
2.8	TP 4/5 - Jalon 5 : Dictionnaire Inversible (Bijection) avec Gestion Stricte des Erreurs	69
2.8.1	Introduction	69
2.8.2	Prérequis	69
2.8.3	Objectifs du TP	69
2.8.4	Rappel Mathématique : La Bijection	69
2.8.5	Exercice 1 : Conception et Implémentation de la classe <code>InvertibleDict</code>	70
2.9	Test 1 : Initialisation et ajout basique	76
2.10	Test 2 : Initialisation avec des données	76

2.11	Test 3 : Tentative d'ajout de clé dupliquée (avec valeur différente)	76
2.12	Test 4 : Tentative d'ajout de valeur dupliquée (avec clé différente)	77
2.13	Test 5 : Modification d'une paire existante (clé -> nouvelle valeur unique)	77
2.14	Test 6 : Tentative de modification d'une clé vers une valeur déjà existante	77
2.15	Test 7 : Suppression d'une paire	77
2.16	Test 8 : Tentative de suppression d'une clé inexistante	78
2.17	Test 9 : Méthodes <code>keys()</code> , <code>values()</code> , <code>items()</code>	78
2.18	Test 10 : Représentation string	78
2.19	Test 11 : Initialisation avec des données dupliquées (doit échouer)	78
2.20	TP 5	79
2.21	TP 5/5 - Mini-projet IA : Implémentation d'une transformation (fonction) et de son inverse pour normaliser/dénormaliser des données en 1D	79
2.21.1	Objectifs pédagogiques	79
2.21.2	Introduction : Pourquoi normaliser les données en IA ?	79
2.21.3	Partie 1 : Rappels Mathématiques et Définition de la Transfor- mation Min-Max	80
2.21.4	Partie 2 : Implémentation en Python Pur	81
2.21.5	Partie 3 : Validation Mathématique et Tests avec <code>assert</code>	84
2.21.6	Conclusion et Réflexions	88
3	Annexes : Représentations Visuelles (TikZ)	89
3.1	Schéma d'une Injection	89
3.2	Schéma d'une Surjection	89
3.3	Schéma d'une Bijection	90

0.0.1 1. Présentation du concept clé

- **La Métaphore** : Imaginez un archer qui tire des flèches sur des cibles.
 - L'**Application**, c'est la règle du jeu : chaque flèche *doit* être tirée et atteindre *une seule* cible (on ne peut pas rater la cible, ni toucher deux cibles avec une seule flèche).
 - L'**Injection**, c'est quand chaque cible ne reçoit *au plus qu'une* flèche (pas de jaloux, personne ne partage sa cible).
 - La **Surjection**, c'est quand *toutes* les cibles reçoivent au moins une flèche (personne n'est oublié).
 - La **Bijection**, c'est le cas parfait : chaque flèche a sa cible unique, et chaque cible a sa flèche unique. C'est comme un dictionnaire parfait entre deux mondes.
- **Le “Pourquoi on a inventé ça”** : Les mathématiques consistent souvent à transformer des objets en d'autres objets. Les fonctions sont les “machines à transformer”. Comprendre si une machine est “réversible” (bijection) ou si elle “perd de l'information” (non injective) est crucial pour savoir si on peut revenir en arrière.
- **Visualisation** : Imaginez deux ensembles de points reliés par des fils. Si les fils ne s'emmêlent jamais sur le même point d'arrivée, c'est injectif. Si chaque point d'arrivée a au moins un fil qui lui arrive dessus, c'est surjectif.

0.0.2 2. Formalisation

A. Définitions Formelles

Soient E (ensemble de départ non vide) et F (ensemble d'arrivée non vide) deux ensembles structurés. 1. **Application** ($f : E \rightarrow F$) : Relation qui à chaque élément $x \in E$ associe un unique élément $y \in F$, noté $f(x)$. 2. **Injection** : f est injective si $\forall x, x' \in E, f(x) = f(x') \Rightarrow x = x'$. 3. **Surjection** : f est surjective si $\forall y \in F, \exists x \in E, y = f(x)$. 4. **Bijection** : f est bijective si elle est à la fois injective et surjective. Cela équivaut à dire que $\forall y \in F, \exists! x \in E, y = f(x)$. 5. **Composition** ($g \circ f$) : Soient $f : E \rightarrow F$ et $g : F \rightarrow G$. L'application $g \circ f : E \rightarrow G$ est définie par $(g \circ f)(x) = g(f(x))$.

B. Théorèmes, Propositions & Lemmes

Proposition (Composition et Bijection) : Soient $f : E \rightarrow F$ et $g : F \rightarrow G$ deux applications bijectives. Alors $g \circ f : E \rightarrow G$ est bijective et sa réciproque est donnée par $(g \circ f)^{-1} = f^{-1} \circ g^{-1}$.

Caractérisation de l'injectivité par la composition : Soit un ensemble de départ non vide $E \neq \emptyset$. Une application $f : E \rightarrow F$ est injective si et seulement s'il existe une application $g : F \rightarrow E$ telle que $g \circ f = \text{Id}_E$ (rétraction à gauche).

0.0.3 3. Démonstrations

Démonstration du Théorème Pivot : Injection de la composée $g \circ f$

Soient $f : E \rightarrow F$ et $g : F \rightarrow G$. Montrons que si f et g sont injectives, alors $g \circ f$ est injective.

1. **Initialisation / Cadre** : Soient $x, x' \in E$. Supposons que $(g \circ f)(x) = (g \circ f)(x')$. Notre but est de montrer que $x = x'$.
2. **Étape 1 : Utilisation de la définition de la composée** $(g \circ f)(x) = (g \circ f)(x') \Rightarrow g(f(x)) = g(f(x'))$.
3. **Étape 2 : Utilisation de l'injectivité de g** Comme g est injective, pour tous $y, y' \in F$, $g(y) = g(y') \Rightarrow y = y'$. Ici, posons $y = f(x)$ et $y' = f(x')$. Puisque $g(f(x)) = g(f(x'))$, alors par injectivité de g , on a :

$$f(x) = f(x')$$

4. **Étape 3 : Utilisation de l'injectivité de f** Comme f est injective, pour tous $z, z' \in E$, $f(z) = f(z') \Rightarrow z = z'$. Comme nous avons établi que $f(x) = f(x')$, alors par injectivité de f , on a :

$$x = x'$$

5. **Conclusion** : Nous avons montré que $(g \circ f)(x) = (g \circ f)(x') \Rightarrow x = x'$. L'application $g \circ f$ est donc injective. La démonstration est achevée.

0.0.4 4. Exercices d'Application

Exercice 1 : Application Directe (Fonction réelle)

Énoncé : Soit $f : \mathbb{R} \rightarrow \mathbb{R}$ définie par $f(x) = 2x + 3$. Démontrer que f est bijective et déterminer sa réciproque f^{-1} . **Correction Détaillée :** * *Analyse de l'énoncé :* On doit montrer que pour tout $y \in \mathbb{R}$, l'équation $f(x) = y$ possède une unique solution $x \in \mathbb{R}$. * *Résolution pas-à-pas :* 1. Soit $y \in \mathbb{R}$. Résolvons $2x + 3 = y$. 2. $2x = y - 3$. 3. $x = \frac{y-3}{2}$. 4. Comme $y \in \mathbb{R}$, alors $\frac{y-3}{2}$ est un nombre réel bien défini. 5. Il existe donc une unique solution $x = \frac{1}{2}y - \frac{3}{2}$ pour chaque y . 6. f est donc bijective. 7. Sa réciproque est $f^{-1} : \mathbb{R} \rightarrow \mathbb{R}$ définie par $f^{-1}(y) = \frac{1}{2}y - \frac{3}{2}$. * *Conclusion :* La fonction est une bijection de la droite réelle sur elle-même.

Exercice 2 : Niveau Avancé (Composition et Surjection)

Énoncé : Soient $f : E \rightarrow F$ et $g : F \rightarrow G$. Démontrer que si $g \circ f$ est surjective, alors g est surjective. **Correction Détaillée :** * *Analyse de l'énoncé :* On utilise la définition de la surjectivité : atteindre tous les points de l'ensemble d'arrivée. * *Résolution pas-à-pas :* 1. Soit $z \in G$. Notre but est de trouver un antécédent $y \in F$ par g tel que $g(y) = z$. 2. Comme $g \circ f : E \rightarrow G$ est surjective, par définition, il existe $x \in E$ tel que $(g \circ f)(x) = z$. 3. En utilisant la définition de la composition : $g(f(x)) = z$. 4. Posons $y = f(x)$. Comme $x \in E$ et $f : E \rightarrow F$, alors y est un élément de F . 5. Nous avons trouvé un élément $y \in F$ tel que $g(y) = z$. 6. Cela est vrai pour tout $z \in G$. * *Conclusion :* L'application g est donc surjective. (Note : On ne peut rien conclure sur la surjectivité de f sans information supplémentaire).

0.0.5 5. Application en Intelligence Artificielle

- **Le Pont Théorique :** Les réseaux de neurones sont des **compositions massives** de fonctions élémentaires (couches).
- **Exemple Concret :** Dans les **Normalizing Flows** (modèles génératifs d'IA), on cherche à apprendre une suite de transformations $f_1, f_2, \dots, f_n$ qui sont toutes des **bijections**. Pourquoi ? Parce qu'une bijection permet de transformer une distribution de probabilité simple (comme une Gaussienne) en une distribution complexe (une image), tout en étant capable de faire le calcul inverse (calculer la probabilité d'une image) de manière exacte. Si une couche n'est pas bijective, on perd de l'information et le modèle devient "aveugle" à certaines données.

0.0.6 6. Liens Sémantiques

- **Concepts Précédents requis :** Jalon-4
- **Concepts Futurs dépendants :** Jalon-6.md | Jalon 6, Jalon 52 (Applications continues entre espaces topologiques et définition fine des homéomorphismes.), Jalon 111 (Applications différentiables entre variétés)

Chapitre 1

Exercices d'application

1.1 Exercice 1

1.1.1 Exercice 1/10

Niveau de difficulté : ★ sur 5

Sujet : Reconnaître une application, injection, surjection sur des ensembles finis (diagrammes sagittaux traduits en ensembles).

Énoncé

Soient les ensembles finis $E = \{1, 2, 3\}$ et $F = \{a, b, c, d\}$. On définit la relation f de E vers F par l'ensemble de couples :

$$f = \{(1, a), (2, b), (3, a)\}$$

Déterminez, en justifiant rigoureusement chaque réponse : 1. Si f est une application de E dans F . 2. Si f est une injection de E dans F . 3. Si f est une surjection de E dans F .

Analyse de l'énoncé

Cet exercice introductif vise à solidifier la compréhension des définitions fondamentales d'une application, d'une injection et d'une surjection sur des ensembles finis. La relation f est donnée explicitement sous forme d'un ensemble de couples, ce qui est une représentation directe d'un "diagramme sagittal" où les flèches sont les couples $(x, f(x))$.

Nous allons rappeler les définitions clés :

— **Application (ou fonction) :** Une relation f de E vers F est une application si et seulement si tout élément $x \in E$ possède une unique image $y \in F$. Formellement :

1. Pour tout $x \in E$, il existe $y \in F$ tel que $(x, y) \in f$. (Existence d'une image)
2. Pour tout $x \in E$, si $(x, y_1) \in f$ et $(x, y_2) \in f$, alors $y_1 = y_2$. (Unicité de l'image) En d'autres termes, chaque élément de l'ensemble de départ E doit être le premier élément d'exactly un couple dans f .

- **Injection** : Une application $f : E \rightarrow F$ est injective si et seulement si deux éléments distincts de E ont toujours des images distinctes dans F . Formellement : Pour tout $x_1, x_2 \in E$, si $x_1 \neq x_2$, alors $f(x_1) \neq f(x_2)$. Une formulation équivalente, souvent plus pratique pour la démonstration, est la contraposée : Pour tout $x_1, x_2 \in E$, la condition formelle exige que si $f(x_1) = f(x_2)$, alors nécessairement $x_1 = x_2$, ce qui démontre l'absence d'information perdue. En d'autres termes, chaque élément de l'ensemble d'arrivée F possède au plus une pré-image dans E .
- **Surjection** : Une application $f : E \rightarrow F$ est surjective si et seulement si tout élément $y \in F$ possède au moins une pré-image $x \in E$. Formellement : Pour tout $y \in F$, il existe $x \in E$ tel que $f(x) = y$. Ceci est équivalent à dire que l'image de l'application, notée $f(E) = \{f(x) \mid x \in E\}$, est égale à l'ensemble d'arrivée F . En d'autres termes, chaque élément de l'ensemble d'arrivée F doit être le second élément d'au moins un couple dans f .

Pour des ensembles finis, ces propriétés peuvent être vérifiées par inspection directe de tous les éléments et de leurs images/pré-images.

Correction exhaustive pas-à-pas

Soient les ensembles $E = \{1, 2, 3\}$ et $F = \{a, b, c, d\}$. La relation f est donnée par $f = \{(1, a), (2, b), (3, a)\}$.

1. f est-elle une application de E dans F ? Pour que f soit une application, chaque élément de E doit avoir une et une seule image dans F . Nous allons vérifier cette condition pour chaque élément de E .

- **Pour l'élément $1 \in E$** : Nous cherchons les couples dans f dont le premier élément est 1. Nous trouvons le couple $(1, a)$. Il n'y a pas d'autre couple dans f dont le premier élément est 1. Donc, l'élément 1 a une unique image, qui est a .
- **Pour l'élément $2 \in E$** : Nous cherchons les couples dans f dont le premier élément est 2. Nous trouvons le couple $(2, b)$. Il n'y a pas d'autre couple dans f dont le premier élément est 2. Donc, l'élément 2 a une unique image, qui est b .
- **Pour l'élément $3 \in E$** : Nous cherchons les couples dans f dont le premier élément est 3. Nous trouvons le couple $(3, a)$. Il n'y a pas d'autre couple dans f dont le premier élément est 3. Donc, l'élément 3 a une unique image, qui est a .

Puisque chaque élément de E (à savoir 1, 2, 3) possède une et une seule image dans F , la relation f satisfait la définition d'une application.

Conclusion pour la question 1 : Oui, f est une application de E dans F .

2. f est-elle une injection de E dans F ? Pour que f soit une injection, des éléments distincts de E doivent avoir des images distinctes dans F . Autrement dit, si $f(x_1) = f(x_2)$, alors x_1 doit être égal à x_2 .

Nous allons examiner les images des éléments de E : $f(1) = a$, $f(2) = b$, $f(3) = a$.

Nous observons que $f(1) = a$ et $f(3) = a$. Nous avons donc $f(1) = f(3)$. Cependant, les éléments de départ sont 1 et 3, et $1 \neq 3$. Puisque nous avons trouvé deux éléments distincts de E (à savoir 1 et 3) qui ont la même image dans F (à savoir a), la condition d'injectivité n'est pas remplie.

Conclusion pour la question 2 : Non, f n'est pas une injection de E dans F .

3. f est-elle une surjection de E dans F ? Pour que f soit une surjection, chaque élément de F doit avoir au moins une pré-image dans E . Autrement dit, l'image de f , notée $f(E)$, doit être égale à l'ensemble d'arrivée F .

Calculons l'image de f , $f(E) : f(E) = \{f(x) \mid x \in E\}$ $f(E) = \{f(1), f(2), f(3)\}$ En utilisant les images que nous avons identifiées : $f(E) = \{a, b, a\}$ En éliminant les doublons pour obtenir un ensemble : $f(E) = \{a, b\}$

Maintenant, comparons $f(E)$ avec l'ensemble d'arrivée $F = \{a, b, c, d\}$. Nous avons $f(E) = \{a, b\}$ et $F = \{a, b, c, d\}$. Pour que f soit surjective, il faut que $f(E) = F$. Cependant, nous constatons que $c \in F$ mais $c \notin f(E)$. De même, $d \in F$ mais $d \notin f(E)$. Puisque les éléments c et d de F n'ont aucune pré-image dans E , la condition de surjectivité n'est pas remplie.

Conclusion pour la question 3 : Non, f n'est pas une surjection de E dans F .

Liens avec l'Intelligence Artificielle

Les concepts d'applications, d'injections et de surjections sont fondamentaux en mathématiques discrètes et ont des répercussions directes dans de nombreux domaines de l'Intelligence Artificielle, même si leur application n'est pas toujours explicite au premier abord.

1. Représentation des Données et des Relations :

- En IA, les données sont souvent représentées sous forme de structures discrètes (graphes, bases de données relationnelles, ensembles de tuples). Une relation comme celle de l'exercice peut être vue comme une table dans une base de données où E est un ensemble de clés et F un ensemble de valeurs.
- Vérifier si une relation est une application, c'est s'assurer de l'intégrité des données : chaque "clé" (élément de E) doit correspondre à une "valeur" unique (élément de F). C'est une contrainte de base pour de nombreux systèmes de gestion de données utilisés en IA.

2. Fonctions de Hachage et Tables de Hachage :

- Les fonctions de hachage, essentielles pour l'efficacité des structures de données comme les tables de hachage (utilisées pour l'accès rapide aux données en IA), sont des applications. Idéalement, une fonction de hachage serait injective (pas de collisions), mais en pratique, ce n'est pas le cas pour des ensembles de départ plus grands que l'ensemble d'arrivée. L'étude des collisions (non-injectivité) est cruciale pour la performance des algorithmes.

3. Apprentissage Automatique (Machine Learning) :

- **Classification :** Un classifieur en apprentissage automatique est une application $h : X \rightarrow Y$, où X est l'espace des caractéristiques d'entrée et Y est l'ensemble des étiquettes de classe. Pour un classifieur, il est essentiel que chaque entrée ait une unique prédiction (c'est une application). La surjectivité est souvent souhaitable (le modèle doit pouvoir prédire toutes les classes possibles), tandis que l'injectivité est rarement une propriété recherchée ou réalisable (des entrées différentes peuvent et doivent souvent être classées de la même manière).
- **Intégration de Caractéristiques (Feature Engineering) :** Les transformations de caractéristiques (par exemple, la normalisation, la vectorisation de

texte) sont des applications. Comprendre si ces transformations sont injectives (préservent l'information unique de chaque donnée) ou non (introduisent des collisions ou une perte d'information) est crucial pour la conception de modèles performants.

- **Réseaux de Neurones** : Chaque couche d'un réseau de neurones applique une fonction (combinaison linéaire suivie d'une fonction d'activation) à son entrée. L'ensemble du réseau est une composition de telles applications. L'analyse de la surjectivité des couches de sortie est liée à la capacité du réseau à générer une diversité de sorties.

4. Logique et Raisonnement Automatisé :

- Dans la logique formelle et les systèmes de raisonnement automatisé, les relations et les fonctions sont utilisées pour modéliser des connaissances et des inférences. La vérification des propriétés de ces relations est une étape fondamentale pour garantir la cohérence et la validité des systèmes.

En somme, bien que l'exercice porte sur des ensembles finis très simples, les principes sous-jacents de l'existence et de l'unicité des images (application), de la distinction des pré-images (injection) et de la couverture de l'espace d'arrivée (surjection) sont des concepts mathématiques omniprésents qui structurent la conception, l'analyse et l'optimisation de nombreux algorithmes et systèmes d'Intelligence Artificielle.

1.2 Exercice 2

1.2.1 Exercice 2/10 : Injectivité et Surjectivité de Fonctions Réelles Simples

Niveau de difficulté : ★ (1 étoile sur 5)

Énoncé

Soient les fonctions suivantes :

1. $f_1 : \mathbb{R} \rightarrow \mathbb{R}$ définie par $f_1(x) = ax + b$, où $a, b \in \mathbb{R}$ et $a \neq 0$.
2. $f_2 : \mathbb{R} \rightarrow \mathbb{R}$ définie par $f_2(x) = x^2$.
3. $f_3 : \mathbb{R} \rightarrow \mathbb{R}$ définie par $f_3(x) = e^x$.

Pour chacune de ces fonctions, déterminez si elle est injective, surjective, les deux (bijective), ou aucune des deux. Justifiez rigoureusement vos réponses.

Analyse de l'énoncé

Cet exercice vise à consolider la compréhension des définitions d'injectivité et de surjectivité pour des fonctions réelles élémentaires. Il s'agit d'appliquer directement les définitions formelles :

- **Injectivité** : Une fonction $f : E \rightarrow F$ est injective si et seulement si pour tout $x_1, x_2 \in E$, si $f(x_1) = f(x_2)$, alors $x_1 = x_2$. Autrement dit, chaque élément de l'ensemble d'arrivée est l'image d'au plus un élément de l'ensemble de départ.
- **Surjectivité** : Une fonction $f : E \rightarrow F$ est surjective si et seulement si pour tout $y \in F$, il existe au moins un $x \in E$ tel que $f(x) = y$. Autrement dit, chaque élément de l'ensemble d'arrivée est l'image d'au moins un élément de l'ensemble de départ. L'ensemble image $f(E)$ est égal à l'ensemble d'arrivée F .

Pour prouver l'injectivité, on part de l'hypothèse $f(x_1) = f(x_2)$ pour des éléments arbitraires x_1, x_2 du domaine E , et on cherche à déduire $x_1 = x_2$. Pour prouver la non-injectivité, il suffit de trouver un contre-exemple : deux éléments distincts $x_1 \neq x_2$ dans E qui ont la même image $f(x_1) = f(x_2)$.

Pour prouver la surjectivité, on prend un y arbitraire dans l'ensemble d'arrivée F et on cherche à résoudre l'équation $f(x) = y$ pour $x \in E$. Si une solution x existe et appartient à E pour tout $y \in F$, la fonction est surjective. Pour prouver la non-surjectivité, on cherche un contre-exemple : un élément $y \in F$ pour lequel l'équation $f(x) = y$ n'a aucune solution dans E .

La difficulté est faible car les fonctions sont bien connues et les manipulations algébriques sont simples. L'accent est mis sur la rigueur de la démonstration et la bonne application des définitions.

Correction exhaustive pas-à-pas

Appliquons les définitions à chaque fonction.

1. Fonction $f_1 : \mathbb{R} \rightarrow \mathbb{R}$ définie par $f_1(x) = ax + b$, avec $a, b \in \mathbb{R}$ et $a \neq 0$.

Injectivité de f_1 Pour prouver l'injectivité, nous devons montrer que pour tout $x_1, x_2 \in \mathbb{R}$, si $f_1(x_1) = f_1(x_2)$, alors $x_1 = x_2$.

Soient $x_1, x_2 \in \mathbb{R}$ tels que $f_1(x_1) = f_1(x_2)$. Par définition de f_1 , l'égalité des images s'écrit :

$$ax_1 + b = ax_2 + b$$

Pour isoler les termes en x , nous soustrayons b des deux côtés de l'équation :

$$ax_1 + b - b = ax_2 + b - b$$

$$ax_1 = ax_2$$

Puisque l'énoncé spécifie que $a \neq 0$, nous pouvons diviser les deux côtés de l'équation par a :

$$\frac{ax_1}{a} = \frac{ax_2}{a}$$

$$x_1 = x_2$$

Nous avons démontré que si $f_1(x_1) = f_1(x_2)$, alors $x_1 = x_2$. Par conséquent, la fonction f_1 est injective.

Surjectivité de f_1 Pour prouver la surjectivité, nous devons montrer que pour tout $y \in \mathbb{R}$ (l'ensemble d'arrivée), il existe au moins un $x \in \mathbb{R}$ (l'ensemble de départ) tel que $f_1(x) = y$.

Soit $y \in \mathbb{R}$ un élément arbitraire de l'ensemble d'arrivée. Nous cherchons à trouver un $x \in \mathbb{R}$ tel que $f_1(x) = y$. Par définition de f_1 , cela revient à résoudre l'équation suivante pour x :

$$ax + b = y$$

Pour isoler x , nous soustrayons b des deux côtés de l'équation :

$$ax + b - b = y - b$$

$$ax = y - b$$

Puisque $a \neq 0$, nous pouvons diviser les deux côtés par a :

$$x = \frac{y - b}{a}$$

Étant donné que $y \in \mathbb{R}$, $b \in \mathbb{R}$ et $a \in \mathbb{R}$ avec $a \neq 0$, la valeur $x = \frac{y-b}{a}$ est un nombre réel bien défini. Ainsi, pour tout $y \in \mathbb{R}$, nous avons trouvé un $x \in \mathbb{R}$ tel que $f_1(x) = y$. Par conséquent, la fonction f_1 est surjective.

Conclusion pour f_1 Puisque f_1 est à la fois injective et surjective, f_1 est une fonction bijective.

2. Fonction $f_2 : \mathbb{R} \rightarrow \mathbb{R}$ définie par $f_2(x) = x^2$.

Injectivité de f_2 Pour prouver la non-injectivité, nous devons trouver deux éléments distincts $x_1, x_2 \in \mathbb{R}$ tels que $f_2(x_1) = f_2(x_2)$.

Considérons les valeurs $x_1 = 2$ et $x_2 = -2$. Ces deux éléments appartiennent à l'ensemble de départ $\mathbb{R}$ et sont distincts : $x_1 \neq x_2$ car $2 \neq -2$. Calculons leurs images sous la fonction f_2 : $f_2(x_1) = f_2(2) = (2)^2 = 4$. $f_2(x_2) = f_2(-2) = (-2)^2 = 4$. Nous observons que $f_2(2) = f_2(-2)$ alors que $2 \neq -2$. Par conséquent, la fonction f_2 n'est pas injective.

Surjectivité de f_2 Pour prouver la non-surjectivité, nous devons trouver un élément $y \in \mathbb{R}$ (l'ensemble d'arrivée) pour lequel il n'existe aucun $x \in \mathbb{R}$ (l'ensemble de départ) tel que $f_2(x) = y$.

Soit $y \in \mathbb{R}$ un élément arbitraire. Nous cherchons $x \in \mathbb{R}$ tel que $f_2(x) = y$. Par définition de f_2 , cela revient à résoudre l'équation :

$$x^2 = y$$

Nous savons que le carré de tout nombre réel est toujours un nombre réel positif ou nul. C'est-à-dire, pour tout $x \in \mathbb{R}$, $x^2 \geq 0$. Considérons un y strictement négatif, par exemple $y = -1$. L'équation devient $x^2 = -1$. Il n'existe aucun nombre réel x dont le carré est égal à -1 . Par conséquent, pour $y = -1 \in \mathbb{R}$, il n'existe aucun $x \in \mathbb{R}$ tel que $f_2(x) = -1$. Par conséquent, la fonction f_2 n'est pas surjective.

Conclusion pour f_2 Puisque f_2 n'est ni injective ni surjective, f_2 n'est pas une bijection.

3. Fonction $f_3 : \mathbb{R} \rightarrow \mathbb{R}$ définie par $f_3(x) = e^x$.

Injectivité de f_3 Pour prouver l'injectivité, nous devons montrer que pour tout $x_1, x_2 \in \mathbb{R}$, si $f_3(x_1) = f_3(x_2)$, alors $x_1 = x_2$.

Soient $x_1, x_2 \in \mathbb{R}$ tels que $f_3(x_1) = f_3(x_2)$. Par définition de f_3 , l'égalité des images s'écrit :

$$e^{x_1} = e^{x_2}$$

Pour résoudre cette équation, nous pouvons appliquer la fonction logarithme népérien ($\ln$) aux deux côtés. La fonction $\ln$ est bien définie pour les nombres strictement positifs, et la fonction exponentielle e^x est toujours strictement positive pour tout $x \in \mathbb{R}$.

$$\ln(e^{x_1}) = \ln(e^{x_2})$$

En utilisant la propriété fondamentale des logarithmes $\ln(e^u) = u$ pour tout $u \in \mathbb{R}$:

$$x_1 = x_2$$

Nous avons démontré que si $f_3(x_1) = f_3(x_2)$, alors $x_1 = x_2$. Par conséquent, la fonction f_3 est injective.

Surjectivité de f_3 Pour prouver la non-surjectivité, nous devons trouver un élément $y \in \mathbb{R}$ (l'ensemble d'arrivée) pour lequel il n'existe aucun $x \in \mathbb{R}$ (l'ensemble de départ) tel que $f_3(x) = y$.

Soit $y \in \mathbb{R}$ un élément arbitraire. Nous cherchons $x \in \mathbb{R}$ tel que $f_3(x) = y$. Par définition de f_3 , cela revient à résoudre l'équation :

$$e^x = y$$

Nous savons que la fonction exponentielle e^x est toujours strictement positive pour tout $x \in \mathbb{R}$. C'est-à-dire, pour tout $x \in \mathbb{R}$, $e^x > 0$. Considérons un y négatif ou nul, par exemple $y = 0$. L'équation devient $e^x = 0$. Il n'existe aucun nombre réel x tel que $e^x = 0$. De même, si nous prenons $y = -5$, l'équation devient $e^x = -5$. Il n'existe aucun nombre réel x tel que $e^x = -5$, car e^x doit toujours être positif. Par conséquent, pour tout $y \in \mathbb{R}$ tel que $y \leq 0$, il n'existe aucun $x \in \mathbb{R}$ tel que $f_3(x) = y$. Par exemple, pour $y = -10 \in \mathbb{R}$, il n'existe aucun $x \in \mathbb{R}$ tel que $f_3(x) = -10$. Par conséquent, la fonction f_3 n'est pas surjective.

Conclusion pour f_3 Puisque f_3 est injective mais n'est pas surjective, f_3 n'est pas une bijection.

Liens avec l'Intelligence Artificielle

Les concepts d'injectivité et de surjectivité, bien que fondamentaux en mathématiques pures, trouvent des échos significatifs dans plusieurs domaines de l'Intelligence Artificielle, notamment en traitement de données, en apprentissage automatique et en cryptographie.

1. **Représentations et Encodages** : En IA, les données brutes sont souvent transformées en représentations numériques (encodages) pour être traitées par des algorithmes. Une fonction d'encodage idéale est injective. Si un encodage n'est pas injectif (comme $f_2(x) = x^2$ où 2 et -2 sont mappés à 4), cela signifie que différentes entrées peuvent produire la même représentation. Cela peut entraîner une perte d'information ou une ambiguïté, rendant impossible de distinguer les entrées originales. Par exemple, dans l'encodage de mots (word embeddings), on souhaite que des mots sémantiquement distincts aient des représentations distinctes.
2. **Fonctions de Hachage** : Les fonctions de hachage sont utilisées pour mapper des données de taille arbitraire à des valeurs de taille fixe. Elles sont cruciales pour les tables de hachage, la vérification d'intégrité et la cryptographie. Idéalement, une fonction de hachage devrait être "quasi-injective" pour minimiser les collisions (où différentes entrées produisent la même sortie). Bien qu'il soit impossible d'avoir une fonction de hachage parfaitement injective pour des domaines potentiellement infinis vers des codomaines finis, l'objectif est de s'en approcher le plus possible pour garantir l'unicité des identifiants ou la robustesse des signatures numériques.
3. **Compression de Données** : Les algorithmes de compression de données visent à réduire la taille des données. Une compression "sans perte" est une fonction qui est bijective de l'ensemble des données originales vers l'ensemble des données compressées. Cela garantit que les données originales peuvent être entièrement reconstruites à partir des données compressées, ce qui implique que la fonction de compression doit être injective (pas de perte d'information) et surjective (toutes les données compressées peuvent être décompressées).
4. **Modèles Génératifs (GANs, VAEs)** : Dans les modèles génératifs, on cherche à générer de nouvelles données (images, textes, etc.) à partir d'un espace latent de variables aléatoires. La "qualité" de la génération peut être liée à la capacité de la fonction de génération (le générateur) à couvrir l'espace des données réelles (surjectivité) et à produire des échantillons variés et non redondants (injectivité dans l'espace latent). Un générateur qui n'est pas suffisamment surjectif pourrait souffrir de "mode collapse", où il ne génère qu'une petite variété d'échantillons.
5. **Fonctions d'Activation dans les Réseaux de Neurones** : Bien que l'injectivité et la surjectivité ne soient pas des critères directs pour les fonctions d'activation (comme ReLU, Sigmoid, Tanh), la compréhension de leur comportement sur leur domaine et codomaine est essentielle. Par exemple, la fonction sigmoïde mappe $\mathbb{R}$ à $(0, 1)$, elle est injective mais pas surjective sur $\mathbb{R}$. Cela signifie qu'elle compresse l'information dans un intervalle borné, ce qui peut être utile pour la normalisation mais peut aussi entraîner une perte de gradient pour des entrées très grandes ou très petites.

En somme, la compréhension des propriétés d'injectivité et de surjectivité permet d'évaluer la fidélité, la réversibilité et la couverture des transformations de données, des concepts cruciaux pour la conception et l'analyse des algorithmes d'Intelligence Artificielle.

1.3 Exercice 3

1.4 Exercice 3/10 - Jalon 5 : Composition et propriétés des fonctions

Niveau de difficulté : **

Énoncé

Soient les fonctions f et g définies comme suit : $f : \mathbb{R} \rightarrow \mathbb{R}$, donnée par $f(x) = x - 1$.
* $g : \mathbb{R} \rightarrow \mathbb{R}$, donnée par $g(x) = x^2$.

On considère la fonction composée $h = g \circ f$.

1. Déterminer l'expression explicite de la fonction $h(x)$. Préciser rigoureusement son ensemble de départ et son ensemble d'arrivée.
 2. Étudier l'injectivité de la fonction h . Justifier votre réponse de manière exhaustive.
 3. Étudier la surjectivité de la fonction h . Justifier votre réponse de manière exhaustive.
-

Analyse de l'énoncé

Cet exercice vise à évaluer la compréhension des concepts fondamentaux de composition de fonctions, d'injectivité et de surjectivité.

1. **Composition de fonctions** ($h = g \circ f$) : Il s'agit d'appliquer la fonction f en premier, puis la fonction g au résultat de f . La définition formelle $(g \circ f)(x) = g(f(x))$ sera utilisée. Il est crucial de vérifier que les ensembles de définition et d'arrivée sont compatibles pour que la composition soit bien définie. Ici, l'ensemble d'arrivée de f est $\mathbb{R}$, qui est aussi l'ensemble de départ de g , donc la composition est bien définie sur $\mathbb{R}$. L'ensemble de départ de h sera celui de f , et l'ensemble d'arrivée de h sera celui de g .
2. **Injectivité** : Une fonction $\phi : E \rightarrow F$ est injective si et seulement si pour tout $x_1, x_2 \in E$, l'égalité $\phi(x_1) = \phi(x_2)$ implique $x_1 = x_2$. Pour prouver qu'une fonction n'est *pas* injective, il suffit de trouver un contre-exemple, c'est-à-dire deux éléments distincts $x_1 \neq x_2$ dans l'ensemble de départ qui ont la même image par ϕ .
3. **Surjectivité** : Une fonction $\phi : E \rightarrow F$ est surjective si et seulement si pour tout $y \in F$ (l'ensemble d'arrivée), il existe au moins un $x \in E$ (l'ensemble de départ) tel que $\phi(x) = y$. Pour prouver qu'une fonction n'est *pas* surjective, il suffit de trouver un élément y_0 dans l'ensemble d'arrivée F pour lequel il n'existe aucun x dans l'ensemble de départ E tel que $\phi(x) = y_0$.

Les fonctions f et g sont des fonctions polynomiales simples. La fonction $f(x) = x - 1$ est une translation, qui est bijective. La fonction $g(x) = x^2$ est une fonction quadratique, qui n'est ni injective ni surjective sur $\mathbb{R}$. La composition de ces deux fonctions devrait hériter des propriétés de la fonction quadratique en termes d'injectivité et de surjectivité.

Correction exhaustive pas-à-pas

Question 1 : Déterminer l'expression explicite de $h(x)$ et préciser ses ensembles. Soient les fonctions : * $f : \mathbb{R} \rightarrow \mathbb{R}$, définie par $f(x) = x - 1$. * $g : \mathbb{R} \rightarrow \mathbb{R}$, définie par $g(x) = x^2$.

Nous cherchons la fonction composée $h = g \circ f$.

Étape 1 : Vérification de la compatibilité des ensembles. L'ensemble d'arrivée de f est $\mathbb{R}$. L'ensemble de départ de g est $\mathbb{R}$. Puisque l'ensemble d'arrivée de f est égal à l'ensemble de départ de g ($\mathbb{R} = \mathbb{R}$), la composition $g \circ f$ est bien définie.

Étape 2 : Détermination des ensembles de départ et d'arrivée de h . L'ensemble de départ de $h = g \circ f$ est l'ensemble de départ de f , qui est $\mathbb{R}$. L'ensemble d'arrivée de $h = g \circ f$ est l'ensemble d'arrivée de g , qui est $\mathbb{R}$. Ainsi, $h : \mathbb{R} \rightarrow \mathbb{R}$.

Étape 3 : Calcul de l'expression explicite de $h(x)$. Par définition de la composition de fonctions, pour tout $x \in \mathbb{R}$:

$$h(x) = (g \circ f)(x) = g(f(x))$$

Nous substituons l'expression de $f(x)$ dans g :

$$h(x) = g(x - 1)$$

Maintenant, nous appliquons la définition de g , qui est $g(u) = u^2$ pour tout $u \in \mathbb{R}$. Ici, $u = x - 1$.

$$h(x) = (x - 1)^2$$

Pour une expression développée, nous pouvons utiliser l'identité remarquable $(a - b)^2 = a^2 - 2ab + b^2$:

$$h(x) = x^2 - 2 \cdot x \cdot 1 + 1^2$$

$$h(x) = x^2 - 2x + 1$$

Conclusion de la Question 1 : La fonction h est définie par $h : \mathbb{R} \rightarrow \mathbb{R}$, et son expression explicite est $h(x) = (x - 1)^2$ (ou $h(x) = x^2 - 2x + 1$).

Question 2 : Étudier l'injectivité de la fonction h . Pour étudier l'injectivité de $h : \mathbb{R} \rightarrow \mathbb{R}$ définie par $h(x) = (x - 1)^2$, nous devons vérifier si pour tout $x_1, x_2 \in \mathbb{R}$, l'égalité $h(x_1) = h(x_2)$ implique $x_1 = x_2$.

Étape 1 : Poser l'hypothèse d'égalité des images. Soient $x_1, x_2 \in \mathbb{R}$ tels que $h(x_1) = h(x_2)$. Ceci signifie :

$$(x_1 - 1)^2 = (x_2 - 1)^2$$

Étape 2 : Résoudre l'équation pour x_1 et x_2 . L'équation $A^2 = B^2$ est équivalente à $A = B$ ou $A = -B$. Donc, nous avons deux cas possibles : * **Cas 1 :** $x_1 - 1 = x_2 - 1$ En ajoutant 1 aux deux membres de l'équation, nous obtenons : $x_1 = x_2$ Ce cas correspond à la condition d'injectivité.

— **Cas 2 :** $x_1 - 1 = -(x_2 - 1)$ $x_1 - 1 = -x_2 + 1$ En ajoutant 1 aux deux membres de l'équation : $x_1 = -x_2 + 2$

Étape 3 : Chercher un contre-exemple (si la fonction n'est pas injective). Le Cas 2 montre qu'il est possible d'avoir $h(x_1) = h(x_2)$ sans que $x_1 = x_2$. Pour prouver que h n'est pas injective, il suffit de trouver un exemple concret de $x_1 \neq x_2$ tels que $h(x_1) = h(x_2)$. Choisissons une valeur pour x_1 , par exemple $x_1 = 0$. Alors $h(0) = (0 - 1)^2 = (-1)^2 = 1$. Nous cherchons un $x_2 \neq 0$ tel que $h(x_2) = 1$. $(x_2 - 1)^2 = 1$ Ceci implique

$x_2 - 1 = 1$ ou $x_2 - 1 = -1$. Si $x_2 - 1 = 1$, alors $x_2 = 1 + 1 = 2$. Si $x_2 - 1 = -1$, alors $x_2 = -1 + 1 = 0$. Nous avons trouvé $x_2 = 2$. Vérifions : $h(2) = (2 - 1)^2 = 1^2 = 1$. Nous avons donc $h(0) = 1$ et $h(2) = 1$. Puisque $0 \neq 2$ mais $h(0) = h(2)$, la fonction h n'est pas injective.

Conclusion de la Question 2 : La fonction h n'est pas injective. Par exemple, $h(0) = 1$ et $h(2) = 1$, alors que $0 \neq 2$.

Question 3 : Étudier la surjectivité de la fonction h . Pour étudier la surjectivité de $h : \mathbb{R} \rightarrow \mathbb{R}$ définie par $h(x) = (x - 1)^2$, nous devons vérifier si pour tout $y \in \mathbb{R}$ (l'ensemble d'arrivée), il existe au moins un $x \in \mathbb{R}$ (l'ensemble de départ) tel que $h(x) = y$.

Étape 1 : Poser l'équation $h(x) = y$. Nous cherchons à savoir si, pour tout $y \in \mathbb{R}$, l'équation $(x - 1)^2 = y$ admet au moins une solution $x \in \mathbb{R}$.

Étape 2 : Analyser l'équation $(x - 1)^2 = y$. Le terme $(x - 1)^2$ est le carré d'un nombre réel $(x - 1)$. Par les propriétés des nombres réels, le carré de tout nombre réel est toujours supérieur ou égal à zéro. Donc, pour tout $x \in \mathbb{R}$, nous avons $(x - 1)^2 \geq 0$. Ceci signifie que l'image de la fonction h (l'ensemble des valeurs que $h(x)$ peut prendre) est l'intervalle $[0, +\infty[$.

Étape 3 : Comparer l'image avec l'ensemble d'arrivée. L'ensemble d'arrivée de h est $\mathbb{R}$. L'image de h est $[0, +\infty[$. Puisque $[0, +\infty[\neq \mathbb{R}$, la fonction h n'est pas surjective.

Étape 4 : Fournir un contre-exemple (si la fonction n'est pas surjective). Pour prouver que h n'est pas surjective, il suffit de trouver un $y_0 \in \mathbb{R}$ tel qu'il n'existe aucun $x \in \mathbb{R}$ vérifiant $h(x) = y_0$. Choisissons un y_0 qui n'est pas dans l'image de h , c'est-à-dire un nombre réel strictement négatif. Par exemple, soit $y_0 = -1$. Nous cherchons un $x \in \mathbb{R}$ tel que $h(x) = -1$.

$$(x - 1)^2 = -1$$

Comme nous l'avons établi, le carré de tout nombre réel est non-négatif. Il est donc impossible qu'un carré soit égal à -1 dans l'ensemble des nombres réels. L'équation $(x - 1)^2 = -1$ n'admet aucune solution réelle x . Par conséquent, il n'existe aucun $x \in \mathbb{R}$ tel que $h(x) = -1$.

Conclusion de la Question 3 : La fonction h n'est pas surjective. Par exemple, la valeur -1 appartient à l'ensemble d'arrivée $\mathbb{R}$, mais il n'existe aucun $x \in \mathbb{R}$ tel que $h(x) = -1$.

Liens avec l'Intelligence Artificielle

Les concepts d'applications, d'injectivité, de surjectivité et de composition sont absolument fondamentaux en Intelligence Artificielle, en particulier dans le domaine de l'apprentissage profond (Deep Learning).

1. **Composition de fonctions et Réseaux de Neurones :** Un réseau de neurones profond est, par essence, une composition de fonctions. Chaque "couche" du réseau applique une transformation (souvent une combinaison linéaire suivie d'une fonction d'activation non linéaire) aux données d'entrée. Si $L_1, L_2, \dots, L_k$ sont les fonctions représentant les couches successives d'un réseau, alors la fonction globale apprise par le réseau est $L_k \circ L_{k-1} \circ \dots \circ L_1$. Comprendre la composition est donc essentiel pour saisir comment les informations sont transformées et traitées à travers les différentes étapes d'un modèle d'IA.

2. Injectivité et Surjectivité dans les Modèles Génératifs :

- **Modèles Génératifs (GANs, VAEs) :** Ces modèles apprennent à générer de nouvelles données (images, textes, etc.) à partir d'un "espace latent" (un espace de vecteurs de faible dimension). La fonction qui mappe l'espace latent à l'espace des données est une application complexe.
- **Injectivité :** Si cette application n'est pas injective, cela signifie que différents points dans l'espace latent peuvent produire la même sortie dans l'espace des données. C'est un problème connu sous le nom de "mode collapse" dans les GANs, où le générateur ne produit qu'une variété limitée d'échantillons, ignorant d'autres "modes" (types de données) de la distribution cible. Une perte d'injectivité implique une perte d'information lors de la transformation.
- **Surjectivité :** Si l'application de l'espace latent à l'espace des données n'est pas surjective (par rapport à la distribution de données réelles), cela signifie que le modèle ne peut pas générer certains types de données qui existent dans la distribution réelle. Le modèle ne couvre pas l'intégralité de l'espace des données, ce qui limite sa capacité à produire des échantillons diversifiés et réalistes.

3. Réseaux de Neurones Inversibles (Invertible Neural Networks - INN) :

Certaines architectures de réseaux de neurones sont spécifiquement conçues pour être bijectives (à la fois injectives et surjectives). Ces INN sont utilisés dans des domaines comme l'estimation de densité, la compression de données sans perte, ou la génération d'images. La bijectivité garantit qu'il n'y a pas de perte d'information lors du passage à travers le réseau et que chaque sortie correspond à une entrée unique, et vice-versa. Cela permet, par exemple, de calculer la vraisemblance exacte des données, ce qui est crucial pour certaines tâches.

4. Représentation et Réduction de Dimension :

Lorsque l'on utilise des fonctions pour transformer des données brutes en "features" (caractéristiques) plus significatives, les propriétés d'injectivité et de surjectivité sont importantes. Une transformation non injective peut entraîner une perte d'information, où des données d'entrée distinctes sont représentées de manière identique, rendant leur discrimination impossible par la suite. La surjectivité est liée à la capacité de la représentation à capturer toutes les variations pertinentes des données originales.

En somme, une compréhension approfondie de ces concepts mathématiques permet aux chercheurs et ingénieurs en IA de concevoir, d'analyser et de diagnostiquer les comportements de modèles complexes, en particulier lorsqu'il s'agit de transformations de données et de génération de contenu.

1.5 Exercice 4

1.6 Exercice 4/10 : Jalon 5 - Applications, injections, surjections, bijections et composition de fonctions

Niveau de difficulté : **

1.6.1 Énoncé

Soient E , F et G trois ensembles non vides. Soient $f : F \rightarrow G$ et $g : E \rightarrow F$ deux applications.

Démontrer l'implication suivante :

$$(f \circ g \text{ est injective}) \implies (g \text{ est injective})$$

1.6.2 Analyse de l'énoncé

Cet exercice nous demande de prouver une propriété fondamentale de la composition d'applications concernant l'injectivité. Nous sommes dans un cadre purement abstrait, où les ensembles E, F, G et les applications f, g ne sont pas spécifiés au-delà de leurs types.

Rappel des définitions clés :

1. **Application** $h : A \rightarrow B$: Pour tout élément $a \in A$, il existe un unique élément $b \in B$ tel que $h(a) = b$.
2. **Application** $h : A \rightarrow B$ **injective** : Pour tous $a_1, a_2 \in A$, si $h(a_1) = h(a_2)$, alors $a_1 = a_2$. Autrement dit, des éléments distincts de l'ensemble de départ sont toujours envoyés sur des éléments distincts de l'ensemble d'arrivée.
3. **Application composée** $f \circ g : E \rightarrow G$: Pour tout $x \in E$, $(f \circ g)(x) = f(g(x))$. L'application g est appliquée en premier, puis f est appliquée au résultat de g .

Ce que l'on nous donne (hypothèse) : L'application $f \circ g : E \rightarrow G$ est injective. Cela signifie que pour tous $x_1, x_2 \in E$, si $(f \circ g)(x_1) = (f \circ g)(x_2)$, alors $x_1 = x_2$.

Ce que l'on doit démontrer (conclusion) : L'application $g : E \rightarrow F$ est injective. Cela signifie que pour tous $x_1, x_2 \in E$, si $g(x_1) = g(x_2)$, alors $x_1 = x_2$.

Stratégie de démonstration : Pour prouver que g est injective, nous devons partir de l'hypothèse $g(x_1) = g(x_2)$ pour des éléments arbitraires $x_1, x_2 \in E$, et en déduire que $x_1 = x_2$. L'information cruciale à utiliser sera l'injectivité de $f \circ g$.

1.6.3 Correction exhaustive “pas-à-pas”

Soient E , F et G trois ensembles non vides. Soient $f : F \rightarrow G$ et $g : E \rightarrow F$ deux applications.

Nous voulons démontrer que si $f \circ g$ est injective, alors g est injective.

Étape 1 : Rappel des définitions formelles.

- L'application composée $f \circ g$ est définie pour tout $x \in E$ par $(f \circ g)(x) = f(g(x))$. Son ensemble de départ est E et son ensemble d'arrivée est G .
- L'hypothèse que $f \circ g$ est injective signifie que : Pour tous $x_1, x_2 \in E$, si $(f \circ g)(x_1) = (f \circ g)(x_2)$, alors $x_1 = x_2$.
- La conclusion que nous voulons atteindre est que g est injective, ce qui signifie que : Pour tous $x_1, x_2 \in E$, si $g(x_1) = g(x_2)$, alors $x_1 = x_2$.

Étape 2 : Début de la démonstration de l'injectivité de g .

Pour démontrer que g est injective, nous devons prendre deux éléments arbitraires x_1 et x_2 dans l'ensemble de départ de g , qui est E , et supposer que leurs images par g sont égales. Ensuite, nous devons montrer que cela implique que x_1 et x_2 sont en fait le même élément.

Soient $x_1 \in E$ et $x_2 \in E$ deux éléments quelconques. Supposons que $g(x_1) = g(x_2)$.

Étape 3 : Application de f et utilisation de l'hypothèse d'injectivité de $f \circ g$.

Puisque $g(x_1)$ et $g(x_2)$ sont des éléments de l'ensemble F (l'ensemble d'arrivée de g et l'ensemble de départ de f), et que nous avons supposé qu'ils sont égaux, nous pouvons appliquer l'application f à ces deux images égales.

Appliquons f aux deux membres de l'égalité $g(x_1) = g(x_2)$:

$$f(g(x_1)) = f(g(x_2))$$

Par définition de la composition d'applications, nous savons que $f(g(x_1)) = (f \circ g)(x_1)$ et $f(g(x_2)) = (f \circ g)(x_2)$. Donc, l'égalité précédente peut être réécrite comme :

$$(f \circ g)(x_1) = (f \circ g)(x_2)$$

À ce stade, nous avons établi que si $g(x_1) = g(x_2)$, alors $(f \circ g)(x_1) = (f \circ g)(x_2)$. Or, l'hypothèse de l'énoncé est que l'application $f \circ g$ est injective. Par définition de l'injectivité de $f \circ g$, si $(f \circ g)(x_1) = (f \circ g)(x_2)$, alors cela implique que les éléments de départ x_1 et x_2 doivent être égaux. Donc, en utilisant l'injectivité de $f \circ g$, nous déduisons :

$$x_1 = x_2$$

Étape 4 : Conclusion.

Nous avons commencé par supposer que $g(x_1) = g(x_2)$ pour des $x_1, x_2 \in E$ arbitraires, et nous avons logiquement déduit que $x_1 = x_2$. Ceci correspond précisément à la définition de l'injectivité de l'application g .

Par conséquent, nous avons démontré que si $f \circ g$ est injective, alors g est injective.

1.6.4 Liens avec l'Intelligence Artificielle

Le concept d'injectivité et de composition de fonctions est fondamental en Intelligence Artificielle, en particulier dans le domaine de l'apprentissage profond (Deep Learning).

1. **Réseaux de Neurones comme Compositions de Fonctions** : Un réseau de neurones profond est intrinsèquement une composition de fonctions. Chaque couche du réseau peut être vue comme une application $f_i : \mathbb{R}^{d_i} \rightarrow \mathbb{R}^{d_{i+1}}$, où d_i est la dimension de l'espace d'entrée de la couche i . Un réseau de L couches est alors la composition $F = f_L \circ f_{L-1} \circ \dots \circ f_1$. L'injectivité de la composition $f \circ g$ implique

l'injectivité de g est une propriété cruciale ici. Si la première couche (g) d'un réseau de neurones n'est pas injective, elle "fusionne" des informations distinctes dès le début. Aucune couche subséquente (f) ne pourra "dé-fusionner" ces informations, car l'information perdue est irrécupérable.

2. **Préservation de l'Information et Représentations Latentes :** Dans des architectures comme les auto-encodeurs, l'objectif est d'apprendre une représentation latente (un encodage) des données d'entrée. L'encodeur $g : \text{Données} \rightarrow \text{Latent}$ cherche à compresser l'information. Le décodeur $f : \text{Latent} \rightarrow \text{Données}$ tente de reconstruire l'entrée à partir de la représentation latente. Si l'encodeur g n'est pas injectif, cela signifie que différentes données d'entrée peuvent être mappées à la même représentation latente. Dans ce cas, même un décodeur parfait f ne pourrait pas reconstruire l'entrée originale de manière unique, car il y aurait une ambiguïté sur quelle entrée originale correspond à cette représentation latente. La propriété démontrée ici signifie que si l'auto-encodeur complet ($f \circ g$) est capable de reconstruire l'entrée de manière unique (ce qui est une forme d'injectivité si l'on considère l'identité comme la fonction cible), alors l'encodeur g lui-même doit avoir été injectif pour ne pas perdre d'information essentielle.
3. **Fonctions de Hachage et Collisions :** Bien que les fonctions de hachage ne soient généralement pas injectives (elles sont conçues pour mapper un grand espace d'entrée à un espace de sortie plus petit, entraînant des "collisions"), le concept est pertinent. Si une fonction de hachage g est utilisée comme première étape d'un processus, et qu'elle n'est pas injective, alors toute fonction f appliquée ensuite à la sortie de g ne pourra pas distinguer les entrées originales qui ont produit la même valeur de hachage. L'injectivité de g est une condition nécessaire pour la préservation de la distinction des entrées à travers une composition.

En résumé, cette propriété mathématique souligne l'importance de l'injectivité des premières étapes d'un pipeline de traitement de l'information (comme un réseau de neurones) pour garantir que l'information pertinente n'est pas perdue prématurément, ce qui limiterait la capacité des étapes ultérieures à accomplir leur tâche.

1.7 Exercice 5

1.8 Exercice 5/10 : Jalon 5 - Applications, injections, surjections, bijections et composition de fonctions

Professeur Émérite : Chers étudiantes et étudiants, abordons aujourd'hui un résultat fondamental concernant la surjectivité et la composition de fonctions. Ce type de démonstration abstraite est essentiel pour développer une intuition solide en algèbre et en analyse, et pour comprendre la structure des transformations.

Énoncé de l'Exercice (Difficulté : ★★★)

Soient E , F et G trois ensembles non vides. Considérons deux fonctions : $* g : E \rightarrow F$
 $* f : F \rightarrow G$

On suppose que la fonction composée $f \circ g : E \rightarrow G$ est surjective.

Démontrez rigoureusement que la fonction $f : F \rightarrow G$ est surjective.

Analyse de l'Énoncé

Cet exercice nous demande de prouver une implication : si la composition de deux fonctions est surjective, alors la seconde fonction (celle qui est appliquée en dernier) doit nécessairement être surjective.

1. Comprendre la surjectivité :

- Une fonction $h : A \rightarrow B$ est dite surjective si et seulement si pour tout élément b de l'ensemble d'arrivée B , il existe au moins un élément a de l'ensemble de départ A tel que $h(a) = b$. En d'autres termes, l'image de h est égale à son ensemble d'arrivée : $\text{Im}(h) = B$.

2. Hypothèse : $f \circ g : E \rightarrow G$ est surjective.

- Cela signifie que pour tout $z \in G$, il existe au moins un $x \in E$ tel que $(f \circ g)(x) = z$.
- Par définition de la composition, cela équivaut à $f(g(x)) = z$.

3. Conclusion à démontrer : $f : F \rightarrow G$ est surjective.

- Cela signifie que pour tout $z \in G$, il existe au moins un $y \in F$ tel que $f(y) = z$.

4. Stratégie de démonstration :

- Pour prouver que f est surjective, nous devons partir d'un élément arbitraire z dans l'ensemble d'arrivée de f (qui est G).
 - Notre objectif est de trouver un élément y dans l'ensemble de départ de f (qui est F) tel que $f(y) = z$.
 - L'hypothèse sur la surjectivité de $f \circ g$ est notre seule source d'information. Elle nous donne un $x \in E$ tel que $f(g(x)) = z$.
 - L'expression $f(g(x)) = z$ contient déjà la forme $f(\text{quelque chose}) = z$. Le "quelque chose" est $g(x)$.
 - Si nous posons $y = g(x)$, alors y est un élément de F (car $g : E \rightarrow F$) et nous aurons $f(y) = z$. C'est précisément ce que nous cherchons !
-

Correction Exhaustive Pas-à-Pas

Pour démontrer que $f : F \rightarrow G$ est surjective, nous devons montrer que pour tout élément z de l'ensemble d'arrivée G , il existe au moins un élément y de l'ensemble de départ F tel que $f(y) = z$.

1. **Initialisation** : Soit z un élément quelconque et arbitrairement choisi de l'ensemble G . Notre but est de trouver un $y \in F$ tel que $f(y) = z$.
2. **Utilisation de l'hypothèse** : Nous savons par hypothèse que la fonction composée $f \circ g : E \rightarrow G$ est surjective.
 - Par définition de la surjectivité, cela signifie que pour tout élément de l'ensemble d'arrivée G , il existe au moins un élément dans l'ensemble de départ E qui lui est appliqué.
 - Puisque $z \in G$ (choisi à l'étape 1), il existe donc au moins un élément $x_0 \in E$ tel que $(f \circ g)(x_0) = z$.
3. **Décomposition de la composition** : Par définition de la composition de fonctions, l'expression $(f \circ g)(x_0)$ est équivalente à $f(g(x_0))$.
 - En substituant cette équivalence dans l'équation de l'étape 2, nous obtenons :

$$f(g(x_0)) = z$$

4. **Identification du candidat pour y** : Nous cherchons un élément $y \in F$ tel que $f(y) = z$. L'équation $f(g(x_0)) = z$ nous fournit directement un tel candidat.
 - Posons $y_0 = g(x_0)$.
5. **Vérification de l'appartenance de y_0 à F** : La fonction g est définie comme $g : E \rightarrow F$.
 - Puisque $x_0 \in E$, l'image de x_0 par g , qui est $y_0 = g(x_0)$, est nécessairement un élément de l'ensemble F .
 - Donc, $y_0 \in F$.
6. **Vérification de la condition $f(y_0) = z$** : En substituant y_0 dans l'équation de l'étape 3, nous obtenons :
 - $f(y_0) = z$.
7. **Conclusion** : Nous avons démontré que pour tout $z \in G$ (choisi arbitrairement au début), il existe un élément $y_0 \in F$ (spécifiquement $y_0 = g(x_0)$ pour un certain $x_0 \in E$) tel que $f(y_0) = z$.
 - Ceci est précisément la définition de la surjectivité de la fonction $f : F \rightarrow G$.

Par conséquent, si la fonction composée $f \circ g$ est surjective, alors la fonction f est surjective.

Liens avec l'Intelligence Artificielle

Le concept de surjectivité et de composition de fonctions est omniprésent en Intelligence Artificielle, particulièrement dans le domaine de l'apprentissage profond (Deep Learning).

1. **Réseaux de Neurones comme Compositions de Fonctions** :

- Un réseau de neurones est fondamentalement une composition de fonctions. Chaque couche du réseau peut être vue comme une fonction $f_i : H_{i-1} \rightarrow H_i$, où H_{i-1} et H_i sont les espaces des activations de la couche précédente et de la couche actuelle, respectivement.
- Le réseau entier est alors une fonction composée $F = f_L \circ f_{L-1} \circ \dots \circ f_1 : X \rightarrow Y$, où X est l'espace d'entrée et Y est l'espace de sortie.
- Dans notre exercice, g pourrait représenter les premières couches du réseau ($f_{L-1} \circ \dots \circ f_1$), et f la dernière couche (ou les dernières couches) du réseau (f_L).

2. Surjectivité et Capacité Représentative :

- Si le réseau global $F = f \circ g$ est capable de produire n'importe quelle sortie désirée dans l'espace G (c'est-à-dire qu'il est "surjectif" sur l'espace des cibles), alors notre démonstration implique que la dernière partie du réseau, f , doit elle-même être surjective.
- Cela signifie que la couche de sortie (ou le "head" du modèle) doit avoir une capacité suffisante pour mapper les représentations internes (les "features" produites par g) à l'ensemble complet des sorties possibles. Si f n'était pas surjective, il existerait des sorties cibles que le réseau ne pourrait jamais atteindre, peu importe la qualité des représentations générées par g .

3. Modèles Génératifs (GANs, VAEs) :

- Dans les modèles génératifs, comme les Generative Adversarial Networks (GANs) ou les Variational Autoencoders (VAEs), le générateur G est une fonction qui mappe un espace latent Z (souvent un bruit aléatoire) vers l'espace des données réelles X . L'objectif est que G soit "surjectif" sur la variété des données réelles, c'est-à-dire qu'il puisse générer n'importe quelle donnée réaliste.
- Si le générateur G est lui-même un réseau profond, $G = G_k \circ \dots \circ G_1$, alors la surjectivité de l'ensemble G implique que la dernière couche G_k doit être capable de couvrir l'espace des données. Si G_k est trop restrictive, le modèle ne pourra pas générer toute la diversité des données, même si les couches précédentes ont appris des représentations latentes riches.

4. Implications pour la Conception de Modèles :

- Ce théorème souligne l'importance de la conception de la couche de sortie. Si l'on sait que le problème requiert une couverture complète de l'espace de sortie (par exemple, classification multi-classes où toutes les classes sont possibles, ou génération d'images avec une grande diversité), alors la fonction de la dernière couche doit être choisie de manière à être potentiellement surjective.
- Par exemple, pour une classification, une couche `softmax` est souvent utilisée, qui peut en principe produire n'importe quelle distribution de probabilité sur les classes, permettant ainsi à la fonction f d'être "surjective" sur l'espace des distributions de probabilité.

En somme, ce résultat mathématique simple a des répercussions profondes sur la compréhension de la capacité et des limitations des architectures d'apprentissage profond, en soulignant que la "puissance" de la transformation finale est cruciale pour la capacité globale du système à atteindre toutes les cibles possibles.

1.9 Exercice 6

1.10 Exercice 6/10 : Construction d'une bijection entre $\mathbb{N}$ et $\mathbb{Z}$

Jalon 5 : Applications, injections, surjections, bijections et composition de fonctions

Niveau de difficulté : ★★★

1.10.1 Énoncé

Soient les ensembles $\mathbb{N} = \{0, 1, 2, 3, \dots\}$ l'ensemble des nombres entiers naturels et $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ l'ensemble des nombres entiers relatifs.

1. Construire explicitement une fonction $f : \mathbb{N} \rightarrow \mathbb{Z}$.
 2. Démontrer rigoureusement que la fonction f ainsi construite est une bijection.
-

1.10.2 Analyse de l'énoncé

L'objectif de cet exercice est de démontrer l'équipotence des ensembles $\mathbb{N}$ et $\mathbb{Z}$ en construisant une bijection explicite entre eux. Bien que l'on sache intuitivement que ces deux ensembles sont "infinis", la notion d'équipotence pour les ensembles infinis est plus subtile et nécessite la construction d'une fonction qui soit à la fois injective et surjective.

1. Construction explicite de $f : \mathbb{N} \rightarrow \mathbb{Z}$: Nous devons trouver une règle qui associe à chaque entier naturel n un unique entier relatif $f(n)$, de telle sorte que tous les entiers relatifs soient atteints et qu'aucun ne soit atteint plus d'une fois. Une stratégie courante pour construire une bijection entre $\mathbb{N}$ et $\mathbb{Z}$ est de "zigzaguer" entre les entiers positifs (y compris zéro) et les entiers négatifs. Par exemple : * $0 \mapsto 0$ * $1 \mapsto -1$ * $2 \mapsto 1$ * $3 \mapsto -2$ * $4 \mapsto 2$ * $5 \mapsto -3$ etc.

Cette observation suggère une distinction basée sur la parité de n : * Si n est pair, $n = 2k$ pour un certain $k \in \mathbb{N}$, alors $f(n)$ devrait être un entier non-négatif. En observant la séquence, $f(0) = 0$, $f(2) = 1$, $f(4) = 2$, on voit que $f(2k) = k$. * Si n est impair, $n = 2k + 1$ pour un certain $k \in \mathbb{N}$, alors $f(n)$ devrait être un entier négatif. En observant la séquence, $f(1) = -1$, $f(3) = -2$, $f(5) = -3$, on voit que $f(2k + 1) = -(k + 1)$.

Nous allons formaliser cette fonction par morceaux.

2. Démonstration de la bijectivité : Pour prouver que f est une bijection, nous devons démontrer deux propriétés : * **Injectivité (ou injection) :** Pour tous $n_1, n_2 \in \mathbb{N}$, si $f(n_1) = f(n_2)$, alors $n_1 = n_2$. Cela signifie que deux éléments distincts de $\mathbb{N}$ ne peuvent pas avoir la même image dans $\mathbb{Z}$. * **Surjectivité (ou surjection) :** Pour tout $z \in \mathbb{Z}$, il existe au moins un $n \in \mathbb{N}$ tel que $f(n) = z$. Cela signifie que chaque élément de $\mathbb{Z}$ est l'image d'au moins un élément de $\mathbb{N}$.

La preuve de l'injectivité et de la surjectivité nécessitera une analyse par cas, en fonction de la parité de n pour l'injectivité, et du signe de z pour la surjectivité.

1.10.3 Correction exhaustive pas-à-pas

1. Construction explicite de la fonction $f : \mathbb{N} \rightarrow \mathbb{Z}$

Nous proposons la fonction f définie par :

$$f(n) = \begin{cases} \frac{n}{2} & \text{si } n \text{ est pair} \\ -\frac{n+1}{2} & \text{si } n \text{ est impair} \end{cases}$$

Vérifions quelques valeurs pour s'assurer de la cohérence avec la stratégie de "zigzag" :
* Pour $n = 0$ (pair) : $f(0) = \frac{0}{2} = 0$. * Pour $n = 1$ (impair) : $f(1) = -\frac{1+1}{2} = -\frac{2}{2} = -1$. *
Pour $n = 2$ (pair) : $f(2) = \frac{2}{2} = 1$. * Pour $n = 3$ (impair) : $f(3) = -\frac{3+1}{2} = -\frac{4}{2} = -2$. *
Pour $n = 4$ (pair) : $f(4) = \frac{4}{2} = 2$.

La fonction semble correcte et correspond à notre intuition.

2. Démonstration que f est une bijection

Pour prouver que f est une bijection, nous devons montrer qu'elle est injective et surjective.

a) Preuve de l'injectivité Soient $n_1, n_2 \in \mathbb{N}$ tels que $f(n_1) = f(n_2)$. Nous devons montrer que $n_1 = n_2$. Soit $z = f(n_1) = f(n_2)$.

Observons les images de f en fonction de la parité de n : * Si n est pair, $n = 2k$ pour un $k \in \mathbb{N}$, alors $f(n) = k$. Les images sont $\{0, 1, 2, \dots\} = \mathbb{N}$. * Si n est impair, $n = 2k + 1$ pour un $k \in \mathbb{N}$, alors $f(n) = -(k + 1)$. Les images sont $\{-1, -2, -3, \dots\} = \mathbb{Z}^-$.

Les ensembles d'images pour les n pairs ($\mathbb{N}$) et les n impairs ($\mathbb{Z}^-$) sont disjoints. En effet, $\mathbb{N} \cap \mathbb{Z}^- = \emptyset$. Par conséquent, si $f(n_1) = f(n_2)$, alors $f(n_1)$ et $f(n_2)$ doivent nécessairement appartenir au même sous-ensemble d'images. Cela implique que n_1 et n_2 doivent avoir la même parité.

Nous considérons deux cas :

Cas 1 : n_1 et n_2 sont tous les deux pairs. Soient $n_1 = 2k_1$ et $n_2 = 2k_2$ pour certains $k_1, k_2 \in \mathbb{N}$. Alors $f(n_1) = \frac{n_1}{2} = \frac{2k_1}{2} = k_1$. Et $f(n_2) = \frac{n_2}{2} = \frac{2k_2}{2} = k_2$. Puisque $f(n_1) = f(n_2)$, nous avons $k_1 = k_2$. En multipliant par 2, nous obtenons $2k_1 = 2k_2$, ce qui signifie $n_1 = n_2$.

Cas 2 : n_1 et n_2 sont tous les deux impairs. Soient $n_1 = 2k_1 + 1$ et $n_2 = 2k_2 + 1$ pour certains $k_1, k_2 \in \mathbb{N}$. Alors $f(n_1) = -\frac{n_1+1}{2} = -\frac{(2k_1+1)+1}{2} = -\frac{2k_1+2}{2} = -(k_1 + 1)$. Et $f(n_2) = -\frac{n_2+1}{2} = -\frac{(2k_2+1)+1}{2} = -\frac{2k_2+2}{2} = -(k_2 + 1)$. Puisque $f(n_1) = f(n_2)$, nous avons $-(k_1 + 1) = -(k_2 + 1)$. En multipliant par -1 , nous obtenons $k_1 + 1 = k_2 + 1$. En soustrayant 1, nous obtenons $k_1 = k_2$. En multipliant par 2 et en ajoutant 1, nous obtenons $2k_1 + 1 = 2k_2 + 1$, ce qui signifie $n_1 = n_2$.

Dans tous les cas possibles où $f(n_1) = f(n_2)$, nous avons montré que $n_1 = n_2$. Par conséquent, la fonction f est injective.

b) Preuve de la surjectivité Soit $z \in \mathbb{Z}$. Nous devons trouver un $n \in \mathbb{N}$ tel que $f(n) = z$. Nous considérons deux cas pour z :

Cas 1 : $z \geq 0$. (C'est-à-dire $z \in \mathbb{N}$) Nous cherchons un $n \in \mathbb{N}$ tel que $f(n) = z$. Puisque $z \geq 0$, l'image z doit provenir d'un n pair, selon la définition de f . Nous posons donc $f(n) = \frac{n}{2}$. Pour que $\frac{n}{2} = z$, il faut que $n = 2z$. Vérifions que ce n est valide : * Puisque $z \in \mathbb{N}$, $z \geq 0$. Donc $2z \geq 0$. Ainsi $n = 2z \in \mathbb{N}$. * Puisque $n = 2z$, n est un

nombre pair. * En appliquant f à ce n : $f(n) = f(2z) = \frac{2z}{2} = z$. Nous avons bien trouvé un $n \in \mathbb{N}$ (en l'occurrence $n = 2z$) tel que $f(n) = z$ pour tout $z \in \mathbb{N}$.

Cas 2 : $z < 0$. (C'est-à-dire $z \in \mathbb{Z}^-$) Nous cherchons un $n \in \mathbb{N}$ tel que $f(n) = z$. Puisque $z < 0$, l'image z doit provenir d'un n impair, selon la définition de f . Nous posons donc $f(n) = -\frac{n+1}{2}$. Pour que $-\frac{n+1}{2} = z$, nous résolvons pour n : $-\frac{n+1}{2} = z \Rightarrow \frac{n+1}{2} = -z \Rightarrow n+1 = -2z \Rightarrow n = -2z - 1$. Vérifions que ce n est valide : * Puisque $z \in \mathbb{Z}^-$, $z \leq -1$. * Alors $-z \geq 1$. * Donc $-2z \geq 2$. * Par conséquent, $n = -2z - 1 \geq 2 - 1 = 1$. * Ainsi $n = -2z - 1 \in \mathbb{N}$ (c'est un entier naturel strictement positif). * Puisque $n = -2z - 1$, n est de la forme $2(\text{entier}) + 1$ (car $-2z$ est pair, donc $-2z - 1$ est impair). Donc n est un nombre impair. * En appliquant f à ce n : $f(n) = f(-2z - 1) = -\frac{(-2z-1)+1}{2} = -\frac{-2z}{2} = -(-z) = z$. Nous avons bien trouvé un $n \in \mathbb{N}$ (en l'occurrence $n = -2z - 1$) tel que $f(n) = z$ pour tout $z \in \mathbb{Z}^-$.

Dans tous les cas, pour tout $z \in \mathbb{Z}$, nous avons trouvé un $n \in \mathbb{N}$ tel que $f(n) = z$. Par conséquent, la fonction f est surjective.

c) Conclusion Puisque la fonction $f : \mathbb{N} \rightarrow \mathbb{Z}$ est à la fois injective et surjective, elle est une bijection. Ceci démontre que les ensembles $\mathbb{N}$ et $\mathbb{Z}$ ont la même cardinalité, c'est-à-dire qu'ils sont équipotents.

1.10.4 Liens avec l'Intelligence Artificielle

La construction d'une bijection entre $\mathbb{N}$ et $\mathbb{Z}$ est un concept fondamental en mathématiques discrètes et en théorie de la calculabilité, qui sont des piliers théoriques de l'Intelligence Artificielle.

1. **Numérisation et Encodage des Données :** En IA, de nombreuses données sont de nature discrète (catégories, mots, symboles). Pour qu'un algorithme puisse les traiter, elles doivent souvent être converties en représentations numériques. Une bijection comme celle-ci illustre le principe d'encodage : chaque élément d'un ensemble (ici $\mathbb{Z}$) peut être mappé de manière unique et réversible à un élément d'un autre ensemble (ici $\mathbb{N}$). En PNL (Traitement du Langage Naturel), par exemple, les mots d'un vocabulaire sont souvent encodés en entiers (indices), et des techniques plus avancées comme le "word embedding" construisent des vecteurs numériques pour chaque mot. La capacité de passer d'une représentation à l'autre sans perte d'information est cruciale.
2. **Théorie de la Calculabilité et Limites de l'IA :** La démonstration que $\mathbb{N}$ et $\mathbb{Z}$ sont équipotents est un exemple simple de la notion de "dénumérabilité". La théorie de la calculabilité, développée par des figures comme Alan Turing, repose sur l'idée que tout ce qui est "calculable" par un algorithme peut être représenté par des opérations sur des nombres naturels. L'ensemble de tous les programmes informatiques possibles est dénombrable (on peut les énumérer avec des entiers naturels). Comprendre les propriétés des ensembles dénombrables et non dénombrables (comme $\mathbb{R}$) est essentiel pour définir les limites théoriques de ce que les ordinateurs et, par extension, les systèmes d'IA peuvent accomplir. Par exemple, le problème de l'arrêt de Turing démontre qu'il n'existe pas d'algorithme général pour déterminer si un programme donné se terminera ou non, une limitation fondamentale qui s'applique à toute IA.

3. **Structures de Données et Hachage** : Bien que la bijection entre $\mathbb{N}$ et $\mathbb{Z}$ soit un cas spécifique, le principe de mappage unique est lié aux fonctions de hachage utilisées dans les structures de données (tables de hachage) qui sont omniprésentes en IA pour l'efficacité. Une fonction de hachage tente de mapper des données d'entrée de taille arbitraire à une valeur de hachage de taille fixe (souvent un entier). Bien que les fonctions de hachage ne soient pas des bijections (des collisions peuvent se produire), l'objectif est de s'en rapprocher le plus possible pour garantir une distribution uniforme et des recherches rapides, ce qui est vital pour la performance des algorithmes d'apprentissage automatique et des bases de données utilisées par l'IA.
4. **Numérotation de Gödel** : Un exemple plus avancé de bijection entre des objets complexes et les nombres naturels est la numérotation de Gödel. Cette technique attribue un nombre naturel unique à chaque formule et preuve dans un système formel. Cela permet de traduire des énoncés sur les propriétés des systèmes formels en énoncés sur les propriétés des nombres naturels, ce qui a conduit aux célèbres théorèmes d'incomplétude de Gödel. Ces théorèmes ont des implications profondes pour la logique, les fondements des mathématiques et, indirectement, pour la compréhension des limites de la raisonnement formel et de l'intelligence artificielle symbolique.

1.11 Exercice 7

1.12 Exercice 7/10 : Images directes et images réciproques d'ensembles

Jalon 5 : Applications, injections, surjections, bijections et composition de fonctions

Niveau de difficulté : ★★★★★

Énoncé

Soient E et F deux ensembles non vides. Soit $f : E \rightarrow F$ une application.

1. Soient A et B deux sous-ensembles de E (i.e., $A \subseteq E$ et $B \subseteq E$). Démontrer que l'image directe de l'union de A et B par f est égale à l'union des images directes de A et B par f . Autrement dit, démontrer que $f(A \cup B) = f(A) \cup f(B)$.
 2. Soient A' et B' deux sous-ensembles de F (i.e., $A' \subseteq F$ et $B' \subseteq F$). Démontrer que l'image réciproque de l'intersection de A' et B' par f est égale à l'intersection des images réciproques de A' et B' par f . Autrement dit, démontrer que $f^{-1}(A' \cap B') = f^{-1}(A') \cap f^{-1}(B')$.
-

Analyse de l'énoncé

Cet exercice vise à consolider la compréhension des définitions d'image directe et d'image réciproque d'ensembles par une application, ainsi que leur interaction avec les opérations ensemblistes fondamentales que sont l'union et l'intersection.

Pour démontrer l'égalité de deux ensembles X et Y (par exemple, $X = f(A \cup B)$ et $Y = f(A) \cup f(B)$), la méthode standard consiste à prouver la double inclusion : 1. $X \subseteq Y$ (tout élément de X est un élément de Y). 2. $Y \subseteq X$ (tout élément de Y est un élément de X).

Nous devons nous appuyer rigoureusement sur les définitions : * **Image directe d'un ensemble** $S \subseteq E$: $f(S) = \{y \in F \mid \exists x \in S, y = f(x)\}$. * **Image réciproque d'un ensemble** $S' \subseteq F$: $f^{-1}(S') = \{x \in E \mid f(x) \in S'\}$. * **Union d'ensembles** : $x \in X \cup Y \iff x \in X \text{ ou } x \in Y$. * **Intersection d'ensembles** : $x \in X \cap Y \iff x \in X \text{ et } x \in Y$.

La difficulté réside dans la manipulation précise des quantificateurs ($\exists$, $\forall$) et des connecteurs logiques (et, ou) à chaque étape de la démonstration, sans aucune ellipse mathématique.

Correction exhaustive pas-à-pas

Question 1 : Démontrer que $f(A \cup B) = f(A) \cup f(B)$ Pour démontrer cette égalité, nous allons prouver la double inclusion : $f(A \cup B) \subseteq f(A) \cup f(B)$ et $f(A) \cup f(B) \subseteq f(A \cup B)$.

Étape 1.1 : Démonstration de $f(A \cup B) \subseteq f(A) \cup f(B)$ Soit y un élément quelconque de l'ensemble $f(A \cup B)$. Par définition de l'image directe, cela signifie qu'il existe au moins un élément x dans l'ensemble $A \cup B$ tel que $y = f(x)$.

$$y \in f(A \cup B) \implies \exists x \in A \cup B \text{ tel que } y = f(x)$$

Puisque $x \in A \cup B$, par définition de l'union, cela signifie que x appartient à A ou x appartient à B .

$$x \in A \cup B \iff (x \in A \text{ ou } x \in B)$$

Nous avons donc deux cas possibles pour cet élément x :

Cas 1 : $x \in A$. Si $x \in A$ et $y = f(x)$, alors par définition de l'image directe, y est un élément de $f(A)$.

$$x \in A \text{ et } y = f(x) \implies y \in f(A)$$

Cas 2 : $x \in B$. Si $x \in B$ et $y = f(x)$, alors par définition de l'image directe, y est un élément de $f(B)$.

$$x \in B \text{ et } y = f(x) \implies y \in f(B)$$

Dans les deux cas (soit $y \in f(A)$, soit $y \in f(B)$), nous pouvons conclure que y appartient à l'union de $f(A)$ et $f(B)$.

$$(y \in f(A) \text{ ou } y \in f(B)) \implies y \in f(A) \cup f(B)$$

Puisque nous avons montré que si $y \in f(A \cup B)$, alors $y \in f(A) \cup f(B)$, nous avons prouvé l'inclusion :

$$f(A \cup B) \subseteq f(A) \cup f(B)$$

Étape 1.2 : Démonstration de $f(A) \cup f(B) \subseteq f(A \cup B)$ Soit y un élément quelconque de l'ensemble $f(A) \cup f(B)$. Par définition de l'union, cela signifie que y appartient à $f(A)$ ou y appartient à $f(B)$.

$$y \in f(A) \cup f(B) \iff (y \in f(A) \text{ ou } y \in f(B))$$

Nous avons donc deux cas possibles pour cet élément y :

Cas 1 : $y \in f(A)$. Si $y \in f(A)$, alors par définition de l'image directe, il existe au moins un élément x_A dans A tel que $y = f(x_A)$.

$$y \in f(A) \implies \exists x_A \in A \text{ tel que } y = f(x_A)$$

Puisque $x_A \in A$, par définition de l'union, il est également vrai que $x_A \in A \cup B$.

$$x_A \in A \implies x_A \in A \cup B$$

Ainsi, nous avons un élément $x_A \in A \cup B$ tel que $y = f(x_A)$. Par définition de l'image directe, cela signifie que $y \in f(A \cup B)$.

$$(\exists x_A \in A \cup B \text{ tel que } y = f(x_A)) \implies y \in f(A \cup B)$$

Cas 2 : $y \in f(B)$. Si $y \in f(B)$, alors par définition de l'image directe, il existe au moins un élément x_B dans B tel que $y = f(x_B)$.

$$y \in f(B) \implies \exists x_B \in B \text{ tel que } y = f(x_B)$$

Puisque $x_B \in B$, par définition de l'union, il est également vrai que $x_B \in A \cup B$.

$$x_B \in B \implies x_B \in A \cup B$$

Ainsi, nous avons un élément $x_B \in A \cup B$ tel que $y = f(x_B)$. Par définition de l'image directe, cela signifie que $y \in f(A \cup B)$.

$$(\exists x_B \in A \cup B \text{ tel que } y = f(x_B)) \implies y \in f(A \cup B)$$

Dans les deux cas (soit $y \in f(A)$, soit $y \in f(B)$), nous avons montré que $y \in f(A \cup B)$. Puisque nous avons montré que si $y \in f(A) \cup f(B)$, alors $y \in f(A \cup B)$, nous avons prouvé l'inclusion :

$$f(A) \cup f(B) \subseteq f(A \cup B)$$

Étape 1.3 : Conclusion pour la Question 1 Puisque nous avons démontré les deux inclusions $f(A \cup B) \subseteq f(A) \cup f(B)$ et $f(A) \cup f(B) \subseteq f(A \cup B)$, nous pouvons conclure que les deux ensembles sont égaux :

$$f(A \cup B) = f(A) \cup f(B)$$

Question 2 : Démontrer que $f^{-1}(A' \cap B') = f^{-1}(A') \cap f^{-1}(B')$ Pour démontrer cette égalité, nous allons prouver la double inclusion : $f^{-1}(A' \cap B') \subseteq f^{-1}(A') \cap f^{-1}(B')$ et $f^{-1}(A') \cap f^{-1}(B') \subseteq f^{-1}(A' \cap B')$.

Étape 2.1 : Démonstration de $f^{-1}(A' \cap B') \subseteq f^{-1}(A') \cap f^{-1}(B')$ Soit x un élément quelconque de l'ensemble $f^{-1}(A' \cap B')$. Par définition de l'image réciproque, cela signifie que l'image de x par f , c'est-à-dire $f(x)$, appartient à l'ensemble $A' \cap B'$.

$$x \in f^{-1}(A' \cap B') \implies f(x) \in A' \cap B'$$

Puisque $f(x) \in A' \cap B'$, par définition de l'intersection, cela signifie que $f(x)$ appartient à A' et $f(x)$ appartient à B' .

$$f(x) \in A' \cap B' \iff (f(x) \in A' \text{ et } f(x) \in B')$$

Nous pouvons séparer cette conjonction en deux affirmations :

1. $f(x) \in A'$. Par définition de l'image réciproque, si $f(x) \in A'$, alors x est un élément de $f^{-1}(A')$.

$$f(x) \in A' \implies x \in f^{-1}(A')$$

2. $f(x) \in B'$. Par définition de l'image réciproque, si $f(x) \in B'$, alors x est un élément de $f^{-1}(B')$.

$$f(x) \in B' \implies x \in f^{-1}(B')$$

Puisque $x \in f^{-1}(A')$ et $x \in f^{-1}(B')$, par définition de l'intersection, nous pouvons conclure que x appartient à l'intersection de $f^{-1}(A')$ et $f^{-1}(B')$.

$$(x \in f^{-1}(A') \text{ et } x \in f^{-1}(B')) \implies x \in f^{-1}(A') \cap f^{-1}(B')$$

Puisque nous avons montré que si $x \in f^{-1}(A' \cap B')$, alors $x \in f^{-1}(A') \cap f^{-1}(B')$, nous avons prouvé l'inclusion :

$$f^{-1}(A' \cap B') \subseteq f^{-1}(A') \cap f^{-1}(B')$$

Étape 2.2 : Démonstration de $f^{-1}(A') \cap f^{-1}(B') \subseteq f^{-1}(A' \cap B')$ Soit x un élément quelconque de l'ensemble $f^{-1}(A') \cap f^{-1}(B')$. Par définition de l'intersection, cela signifie que x appartient à $f^{-1}(A')$ et x appartient à $f^{-1}(B')$.

$$x \in f^{-1}(A') \cap f^{-1}(B') \iff (x \in f^{-1}(A') \text{ et } x \in f^{-1}(B'))$$

Nous pouvons séparer cette conjonction en deux affirmations :

1. $x \in f^{-1}(A')$. Par définition de l'image réciproque, si $x \in f^{-1}(A')$, alors l'image de x par f , c'est-à-dire $f(x)$, appartient à A' .

$$x \in f^{-1}(A') \implies f(x) \in A'$$

2. $x \in f^{-1}(B')$. Par définition de l'image réciproque, si $x \in f^{-1}(B')$, alors l'image de x par f , c'est-à-dire $f(x)$, appartient à B' .

$$x \in f^{-1}(B') \implies f(x) \in B'$$

Puisque $f(x) \in A'$ et $f(x) \in B'$, par définition de l'intersection, nous pouvons conclure que $f(x)$ appartient à l'intersection de A' et B' .

$$(f(x) \in A' \text{ et } f(x) \in B') \implies f(x) \in A' \cap B'$$

Enfin, si $f(x) \in A' \cap B'$, par définition de l'image réciproque, cela signifie que x est un élément de $f^{-1}(A' \cap B')$.

$$f(x) \in A' \cap B' \implies x \in f^{-1}(A' \cap B')$$

Puisque nous avons montré que si $x \in f^{-1}(A') \cap f^{-1}(B')$, alors $x \in f^{-1}(A' \cap B')$, nous avons prouvé l'inclusion :

$$f^{-1}(A') \cap f^{-1}(B') \subseteq f^{-1}(A' \cap B')$$

Étape 2.3 : Conclusion pour la Question 2 Puisque nous avons démontré les deux inclusions $f^{-1}(A' \cap B') \subseteq f^{-1}(A') \cap f^{-1}(B')$ et $f^{-1}(A') \cap f^{-1}(B') \subseteq f^{-1}(A' \cap B')$, nous pouvons conclure que les deux ensembles sont égaux :

$$f^{-1}(A' \cap B') = f^{-1}(A') \cap f^{-1}(B')$$

Liens avec l'Intelligence Artificielle

Les concepts d'images directes et réciproques d'ensembles, ainsi que leurs propriétés en relation avec les opérations ensemblistes, sont des fondations mathématiques omniprésentes en Intelligence Artificielle, bien que souvent implicites dans les applications de haut niveau.

1. **Traitement et Transformation des Données (Feature Engineering)** : En IA, les données sont fréquemment transformées d'un espace à un autre (par exemple, réduction de dimensionnalité, encodage, normalisation). Une telle transformation peut être modélisée comme une application $f : E \rightarrow F$, où E est l'espace des données brutes et F l'espace des caractéristiques transformées.

- La propriété $f(A \cup B) = f(A) \cup f(B)$ signifie que si nous avons deux groupes de données A et B (par exemple, des images de chats et des images de chiens), l'ensemble des caractéristiques transformées pour l'ensemble combiné (chats ou chiens) est simplement l'union des caractéristiques transformées pour les chats et des caractéristiques transformées pour les chiens. Cette propriété intuitive est cruciale pour la cohérence des pipelines de traitement de données.
2. **Classification et Reconnaissance de Formes :** Dans les tâches de classification, une fonction f (le modèle de classification) mappe un ensemble d'entrées E (par exemple, des images) à un ensemble de sorties F (par exemple, des étiquettes de classe).
 - La propriété $f^{-1}(A' \cap B') = f^{-1}(A') \cap f^{-1}(B')$ est particulièrement pertinente. Supposons que A' représente l'ensemble des sorties “chat” et B' l'ensemble des sorties “animal à quatre pattes”. L'intersection $A' \cap B'$ serait l'ensemble des sorties “chat ET animal à quatre pattes”. La propriété démontrée signifie que l'ensemble des images qui sont classées comme “chat ET animal à quatre pattes” est exactement l'intersection des images classées comme “chat” et des images classées comme “animal à quatre pattes”. Cela garantit une interprétation logique et cohérente des prédictions du modèle, essentielle pour la robustesse et l'explicabilité des systèmes d'IA.
 3. **Vérification Formelle et Robustesse des Modèles :** Pour les systèmes d'IA critiques (véhicules autonomes, médecine), il est vital de prouver formellement certaines propriétés de leur comportement. Les identités ensemblistes comme celles-ci sont des briques de base pour des preuves plus complexes. Par exemple, pour analyser la robustesse d'un réseau de neurones face à des perturbations adverses, on pourrait étudier comment des ensembles d'entrées “sûres” (ou “dangereuses”) se transforment à travers les couches du réseau, ou comment des ensembles de sorties désirées (ou indésirables) se “rétro-projettent” dans l'espace d'entrée.
 4. **Représentation des Connaissances et Raisonnement Symbolique :** Dans l'IA symbolique, les connaissances sont souvent représentées par des ensembles et des relations. Les opérations ensemblistes et leurs propriétés sous des transformations fonctionnelles sont fondamentales pour l'inférence logique et la manipulation de concepts. Par exemple, si “mammifère” est un ensemble A' et “carnivore” est un ensemble B' , alors “mammifère carnivore” est $A' \cap B'$. Comprendre comment les fonctions (relations) interagissent avec ces ensembles est au cœur du raisonnement automatique.

En somme, ces propriétés fondamentales de la théorie des ensembles et des fonctions, bien que simples en apparence, sont les piliers logiques qui sous-tendent la capacité des systèmes d'IA à traiter, transformer et interpréter des informations de manière cohérente et prévisible.

1.13 Exercice 8

1.13.1 Exercice 8/10 : Injectivité et existence d'une fonction inverse à gauche

Niveau de difficulté : ★★☆☆

Énoncé

Soient E et F deux ensembles non vides. Soit $f : E \rightarrow F$ une application.

Démontrer que f est injective si et seulement si il existe une application $g : F \rightarrow E$ telle que $g \circ f = \text{Id}_E$.

Où $\text{Id}_E : E \rightarrow E$ est l'application identité sur E , définie par $\text{Id}_E(x) = x$ pour tout $x \in E$.

Analyse de l'énoncé

Cet exercice nous demande de prouver une équivalence fondamentale en théorie des ensembles et des fonctions, reliant la propriété d'injectivité d'une fonction à l'existence d'une "inverse à gauche".

1. **Comprendre l'injectivité** : Une application $f : E \rightarrow F$ est dite injective si et seulement si pour tous $x_1, x_2 \in E$, l'égalité $f(x_1) = f(x_2)$ implique $x_1 = x_2$. En d'autres termes, des éléments distincts de l'ensemble de départ E sont toujours envoyés sur des éléments distincts de l'ensemble d'arrivée F . Il n'y a pas de "collisions" d'images.
 2. **Comprendre l'inverse à gauche** : L'existence d'une application $g : F \rightarrow E$ telle que $g \circ f = \text{Id}_E$ signifie que la composition de g avec f redonne l'identité sur E . Cela implique que pour tout $x \in E$, $(g \circ f)(x) = g(f(x)) = x$. L'application g "annule" l'effet de f pour tout élément de E .
 3. **Structure de la preuve** : L'énoncé est un "si et seulement si" (équivalence logique), ce qui signifie que nous devons prouver deux implications distinctes :
 - **Implication 1** ($\implies$) : Si f est injective, alors il existe une application $g : F \rightarrow E$ telle que $g \circ f = \text{Id}_E$. Pour cette partie, nous devons *construire* explicitement une telle fonction g .
 - **Implication 2** ($\impliedby$) : S'il existe une application $g : F \rightarrow E$ telle que $g \circ f = \text{Id}_E$, alors f est injective. Pour cette partie, nous devons utiliser la propriété de g pour démontrer l'injectivité de f .
 4. **Considérations sur les ensembles** : L'énoncé précise que E et F sont des ensembles non vides. Cette condition est importante, notamment pour la construction de g dans la première implication, où nous pourrions avoir besoin de choisir un élément arbitraire de E .
-

Correction exhaustive pas-à-pas

Nous allons prouver les deux implications séparément.

Partie 1 : f est injective $\implies$ il existe $g : F \rightarrow E$ telle que $g \circ f = \text{Id}_E$.

Hypothèse : $f : E \rightarrow F$ est une application injective. **Objectif :** Construire une application $g : F \rightarrow E$ telle que $g \circ f = \text{Id}_E$.

Puisque E est un ensemble non vide, nous pouvons choisir et fixer un élément arbitraire $x_0 \in E$. Cet élément nous servira pour définir $g(y)$ lorsque y n'est pas une image par f .

Nous allons définir l'application $g : F \rightarrow E$ de la manière suivante :

Pour tout $y \in F$: * **Cas 1 :** Si $y \in \text{Im}(f)$ (l'image de f), cela signifie qu'il existe au moins un $x \in E$ tel que $f(x) = y$. Puisque f est injective, si $f(x_1) = y$ et $f(x_2) = y$, alors $f(x_1) = f(x_2)$, ce qui implique $x_1 = x_2$ par définition de l'injectivité. Par conséquent, pour chaque $y \in \text{Im}(f)$, il existe un *unique* $x \in E$ tel que $f(x) = y$. Dans ce cas, nous définissons $g(y) = x$, où x est l'unique antécédent de y par f .

— **Cas 2 :** Si $y \notin \text{Im}(f)$, cela signifie qu'il n'existe aucun $x \in E$ tel que $f(x) = y$.

Dans ce cas, nous définissons $g(y) = x_0$, où x_0 est l'élément arbitraire de E que nous avons fixé au début.

Cette définition de g est bien une application de F vers E , car pour chaque élément $y \in F$, $g(y)$ est défini de manière unique et appartient à E .

Maintenant, nous devons vérifier que $g \circ f = \text{Id}_E$. Pour cela, nous devons montrer que pour tout $x \in E$, $(g \circ f)(x) = x$.

Soit $x \in E$ un élément quelconque. Calculons $(g \circ f)(x)$: 1. D'abord, nous évaluons $f(x)$. Soit $y_x = f(x)$. 2. Par définition de l'image, $y_x = f(x)$ est un élément de $\text{Im}(f)$. 3. Puisque $y_x \in \text{Im}(f)$, la définition de g pour ce y_x relève du Cas 1. 4. Selon le Cas 1, $g(y_x)$ est défini comme l'unique antécédent de y_x par f . 5. Nous savons que x est un antécédent de y_x par f , car $f(x) = y_x$. 6. Puisque x est l'unique antécédent de y_x par f , il s'ensuit que $g(y_x) = x$. 7. En substituant $y_x = f(x)$, nous obtenons $g(f(x)) = x$.

Cette égalité $g(f(x)) = x$ est vraie pour tout $x \in E$. Par conséquent, $g \circ f = \text{Id}_E$.

Nous avons donc construit une application $g : F \rightarrow E$ telle que $g \circ f = \text{Id}_E$. L'implication f est injective $\implies$ il existe $g : F \rightarrow E$ telle que $g \circ f = \text{Id}_E$ est démontrée.

Partie 2 : Il existe $g : F \rightarrow E$ telle que $g \circ f = \text{Id}_E \implies f$ est injective.

Hypothèse : Il existe une application $g : F \rightarrow E$ telle que $g \circ f = \text{Id}_E$. **Objectif :** Démontrer que f est injective.

Pour prouver que f est injective, nous devons montrer que pour tous $x_1, x_2 \in E$, si $f(x_1) = f(x_2)$, alors $x_1 = x_2$.

Soient $x_1, x_2 \in E$ deux éléments quelconques de E . Supposons que $f(x_1) = f(x_2)$.

Puisque $f(x_1)$ et $f(x_2)$ sont des éléments de F , nous pouvons appliquer l'application g à ces deux éléments. En appliquant g aux deux membres de l'égalité $f(x_1) = f(x_2)$, nous obtenons : $g(f(x_1)) = g(f(x_2))$.

Par définition de la composition d'applications, nous avons : $(g \circ f)(x_1) = (g \circ f)(x_2)$.

Par notre hypothèse, nous savons que $g \circ f = \text{Id}_E$. Donc, nous pouvons remplacer $(g \circ f)(x_1)$ par $\text{Id}_E(x_1)$ et $(g \circ f)(x_2)$ par $\text{Id}_E(x_2)$. L'égalité devient : $\text{Id}_E(x_1) = \text{Id}_E(x_2)$.

Par définition de l'application identité Id_E , nous avons $\text{Id}_E(x_1) = x_1$ et $\text{Id}_E(x_2) = x_2$. Par conséquent, l'égalité se simplifie en : $x_1 = x_2$.

Nous avons donc montré que si $f(x_1) = f(x_2)$, alors $x_1 = x_2$. Ceci est précisément la définition de l'injectivité de f .

L'implication il existe $g : F \rightarrow E$ telle que $g \circ f = \text{Id}_E \implies f$ est injective est démontrée.

Conclusion Puisque nous avons prouvé les deux implications, nous pouvons conclure que f est injective si et seulement si il existe une application $g : F \rightarrow E$ telle que $g \circ f = \text{Id}_E$.

Liens avec l'Intelligence Artificielle

Le concept d'injectivité et d'inverse à gauche trouve des résonances profondes dans plusieurs domaines de l'Intelligence Artificielle, notamment en ce qui concerne la représentation des données, la compression et la robustesse des modèles.

1. Représentation et Encodage des Données (Autoencodeurs) :

- Dans les réseaux de neurones, en particulier les autoencodeurs, l'objectif est d'apprendre une représentation compacte (un encodage) des données d'entrée. Un autoencodeur est composé d'un encodeur $f : X \rightarrow Z$ (où X est l'espace d'entrée et Z est l'espace latent) et d'un décodeur $g : Z \rightarrow X$.
- Idéalement, nous souhaiterions que l'encodeur f soit une fonction injective. Si f est injective, cela signifie que chaque donnée d'entrée unique $x \in X$ est mappée à une représentation latente unique $z \in Z$. Aucune information n'est perdue lors de l'encodage.
- Dans ce scénario idéal, le décodeur g agirait comme une inverse à gauche (et même une inverse tout court si f est surjective sur Z), permettant de reconstruire parfaitement l'entrée originale x à partir de sa représentation latente $z = f(x)$, c'est-à-dire $g(f(x)) = x$.
- En pratique, les autoencodeurs ne garantissent pas l'injectivité (surtout si la dimension de Z est inférieure à celle de X), mais la recherche d'une reconstruction fidèle ($g \circ f \approx \text{Id}_X$) est une tentative d'approcher cette propriété.

2. Compression de Données sans Perte (Lossless Compression) :

- Les algorithmes de compression sans perte visent à transformer des données (par exemple, un fichier texte ou une image) en une représentation plus compacte sans aucune perte d'information.
- Le processus de compression peut être vu comme une fonction $f : \text{Données Originales} \rightarrow \text{Données Compressées}$. Pour que la compression soit sans perte, f doit être injective. Si f n'était pas injective, deux ensembles de données originaux distincts pourraient être compressés en la même représentation, rendant impossible de déterminer l'original lors de la décompression.
- La décompression est alors l'application $g : \text{Données Compressées} \rightarrow \text{Données Originales}$, qui doit agir comme une inverse à gauche (et même une inverse tout court) pour restaurer les données originales : $g \circ f = \text{Id}_{\text{Données Originales}}$.

3. Fonctions de Hachage et Indexation :

- Les fonctions de hachage sont utilisées pour mapper des données de taille arbitraire à des valeurs de taille fixe (les "hachages"). Elles sont fondamentales pour l'indexation de données (tables de hachage) et la vérification d'intégrité.
- Idéalement, une fonction de hachage "parfaite" serait injective sur l'ensemble des clés utilisées, c'est-à-dire qu'elle ne produirait jamais de collisions (deux clés différentes donnant le même hachage).

- Si une fonction de hachage $h : K \rightarrow V$ (où K est l'ensemble des clés et V l'ensemble des valeurs de hachage) était injective, alors il existerait une fonction $g : V \rightarrow K$ telle que $g \circ h = \text{Id}_K$, permettant de retrouver la clé originale à partir de son hachage. En pratique, les fonctions de hachage ne sont pas injectives sur l'ensemble de toutes les entrées possibles, mais elles sont conçues pour minimiser les collisions sur l'ensemble des entrées attendues.

4. Cryptographie :

- Les fonctions injectives sont cruciales en cryptographie. Par exemple, une fonction de chiffrement $E : \text{Message Clair} \rightarrow \text{Message Chiffré}$ doit être injective. Si deux messages clairs différents pouvaient être chiffrés en le même message chiffré, il serait impossible de déchiffrer de manière unique le message original.
- La fonction de déchiffrement $D : \text{Message Chiffré} \rightarrow \text{Message Clair}$ agit alors comme une inverse à gauche (et une inverse à droite) de la fonction de chiffrement, assurant que $D \circ E = \text{Id}_{\text{Message Clair}}$.

En somme, la propriété d'injectivité garantit qu'une transformation ne perd pas d'information, et l'existence d'une inverse à gauche est la manifestation concrète de cette capacité à “revenir en arrière” ou à “annuler” la transformation pour les éléments qui ont été effectivement transformés. C'est un principe fondamental pour la réversibilité et la fidélité des processus en IA.

1.14 Exercice 9

1.15 Exercice 9/10 : Surjectivité et existence d’une fonction inverse à droite

Jalon 5 : Applications, injections, surjections, bijections et composition de fonctions

Niveau de difficulté : ★★★★★

1.15.1 Énoncé Rigoureux

Soient E et F deux ensembles non vides. Soit $f : E \rightarrow F$ une application (ou fonction). Démontrer l’équivalence suivante :

$$f \text{ est surjective} \iff \exists g : F \rightarrow E \text{ telle que } f \circ g = \text{Id}_F$$

où $\text{Id}_F : F \rightarrow F$ est l’application identité sur l’ensemble F , définie par $\text{Id}_F(y) = y$ pour tout $y \in F$.

1.15.2 Analyse de l’Énoncé

Cet exercice fondamental établit un lien direct et profond entre la propriété de surjectivité d’une application et l’existence d’une application “inverse à droite”. C’est une propriété cruciale en théorie des ensembles et en algèbre, dont la compréhension est essentielle pour aborder des concepts plus avancés en mathématiques.

Définitions clés à rappeler :

1. **Surjectivité de f :** Une application $f : E \rightarrow F$ est dite surjective si et seulement si pour tout élément y de l’ensemble d’arrivée F , il existe au moins un élément x de l’ensemble de départ E tel que $f(x) = y$. Formellement :

$$\forall y \in F, \exists x \in E, f(x) = y$$

Cela signifie que l’image de f , notée $\text{Im}(f) = \{f(x) \mid x \in E\}$, est égale à l’ensemble d’arrivée F .

2. **Application identité Id_F :** L’application identité sur F est l’application qui renvoie chaque élément de F sur lui-même.

$$\text{Id}_F : F \rightarrow F, \quad y \mapsto y$$

3. **Composition d’applications $f \circ g$:** Si $g : F \rightarrow E$ et $f : E \rightarrow F$, alors $f \circ g : F \rightarrow F$ est l’application définie par $(f \circ g)(y) = f(g(y))$ pour tout $y \in F$. L’application g est appelée un *inverse à droite* de f .

L’énoncé nous demande de prouver une équivalence logique, ce qui implique la démonstration de deux implications distinctes :

- **Implication directe ($\Rightarrow$)** : Si f est surjective, alors il existe une application $g : F \rightarrow E$ telle que $f \circ g = \text{Id}_F$.
 - Cette partie de la preuve nécessite de *construire* explicitement une telle application g . La surjectivité de f garantit que pour chaque $y \in F$, l'ensemble des antécédents $f^{-1}(\{y\}) = \{x \in E \mid f(x) = y\}$ est non vide. Pour définir $g(y)$, nous devons *choisir* un élément dans cet ensemble $f^{-1}(\{y\})$. Si l'ensemble F est infini, cette “collection de choix” simultanés pour chaque $y \in F$ requiert l'invocation de l'**Axiome du Choix**, un axiome fondamental de la théorie des ensembles. C'est ce point qui justifie en grande partie la difficulté 5 étoiles de l'exercice.
- **Implication réciproque ($\Leftarrow$)** : S'il existe une application $g : F \rightarrow E$ telle que $f \circ g = \text{Id}_F$, alors f est surjective.
 - Cette partie est généralement plus directe et ne nécessite pas l'Axiome du Choix. Nous devons utiliser la propriété $f \circ g = \text{Id}_F$ pour montrer que tout élément de F possède un antécédent par f .

1.15.3 Correction Exhaustive Pas-à-Pas

Nous allons démontrer l'équivalence en prouvant les deux implications séparément, en respectant la règle de la “Zéro Ellipse Mathématique”.

Partie 1 : Implication directe ($\Rightarrow$)

Hypothèse : L'application $f : E \rightarrow F$ est surjective. **Conclusion à démontrer** : Il existe une application $g : F \rightarrow E$ telle que $f \circ g = \text{Id}_F$.

1. **Analyse de l'hypothèse de surjectivité** : Par définition de la surjectivité, pour tout élément y de l'ensemble d'arrivée F , il existe au moins un élément x de l'ensemble de départ E tel que $f(x) = y$. Cela signifie que pour chaque $y \in F$, l'ensemble de ses antécédents par f , noté $f^{-1}(\{y\}) = \{x \in E \mid f(x) = y\}$, est non vide.

$$\forall y \in F, f^{-1}(\{y\}) \neq \emptyset$$

2. **Construction de l'application $g : F \rightarrow E$** : Nous devons définir l'image $g(y)$ pour chaque élément $y \in F$. Pour chaque $y \in F$, nous savons que l'ensemble $f^{-1}(\{y\})$ est non vide. Nous devons choisir un élément spécifique dans cet ensemble pour qu'il soit la valeur de $g(y)$.
 - **Si F est un ensemble fini** : Supposons que $F = \{y_1, y_2, \dots, y_n\}$. Pour chaque $y_i \in F$, l'ensemble $f^{-1}(\{y_i\})$ est non vide. Nous pouvons alors choisir un élément $x_i \in E$ tel que $f(x_i) = y_i$. Nous définissons ensuite l'application $g : F \rightarrow E$ en posant $g(y_i) = x_i$ pour chaque $i \in \{1, \dots, n\}$.
 - **Si F est un ensemble infini** : Dans ce cas, nous avons une famille potentiellement infinie d'ensembles non vides $(f^{-1}(\{y\}))_{y \in F}$. Pour pouvoir effectuer un choix simultané d'un élément dans chacun de ces ensembles, nous faisons appel à l'**Axiome du Choix**. L'Axiome du Choix stipule que pour toute famille non vide d'ensembles non vides $(A_i)_{i \in I}$, il existe une fonction de choix $c : I \rightarrow \bigcup_{i \in I} A_i$ telle que $c(i) \in A_i$ pour tout $i \in I$. Dans notre contexte,

l'ensemble d'indices est $I = F$, et pour chaque $y \in F$, l'ensemble correspondant est $A_y = f^{-1}(\{y\})$. L'Axiome du Choix garantit donc l'existence d'une fonction $g : F \rightarrow E$ telle que pour tout $y \in F$, $g(y) \in f^{-1}(\{y\})$. Par définition de l'ensemble $f^{-1}(\{y\})$, la condition $g(y) \in f^{-1}(\{y\})$ signifie précisément que $f(g(y)) = y$ pour tout $y \in F$.

3. **Vérification de la propriété $f \circ g = \text{Id}_F$** : Nous avons construit une application $g : F \rightarrow E$ telle que pour tout $y \in F$, la relation $f(g(y)) = y$ est satisfaite. Par définition de la composition d'applications, pour tout $y \in F$, l'image de y par $f \circ g$ est donnée par $(f \circ g)(y) = f(g(y))$. Par définition de l'application identité sur F , pour tout $y \in F$, l'image de y par Id_F est donnée par $\text{Id}_F(y) = y$. En combinant ces deux points, nous obtenons que pour tout $y \in F$:

$$(f \circ g)(y) = f(g(y)) = y = \text{Id}_F(y)$$

Puisque cette égalité est vraie pour tout $y \in F$, nous pouvons conclure que les applications $f \circ g$ et Id_F sont égales.

$$f \circ g = \text{Id}_F$$

Ainsi, si f est surjective, il existe bien une application $g : F \rightarrow E$ telle que $f \circ g = \text{Id}_F$.

Partie 2 : Implication réciproque ($\Leftarrow$)

Hypothèse : Il existe une application $g : F \rightarrow E$ telle que $f \circ g = \text{Id}_F$. **Conclusion à démontrer** : L'application $f : E \rightarrow F$ est surjective.

1. **Analyse de l'hypothèse** : Nous avons une application $g : F \rightarrow E$ telle que la composition $f \circ g$ est égale à l'application identité sur F . Par définition de l'égalité d'applications, cela signifie que pour tout élément $y \in F$, l'image de y par $f \circ g$ est égale à l'image de y par Id_F .

$$\forall y \in F, (f \circ g)(y) = \text{Id}_F(y)$$

En utilisant les définitions de la composition et de l'identité, ceci se traduit par :

$$\forall y \in F, f(g(y)) = y$$

2. **Démonstration de la surjectivité de f** : Pour montrer que f est surjective, nous devons prouver que pour tout élément y de l'ensemble d'arrivée F , il existe au moins un élément x de l'ensemble de départ E tel que $f(x) = y$.
- Soit $y \in F$ un élément arbitraire, mais fixé, de l'ensemble d'arrivée F .
 - Notre objectif est de trouver un antécédent $x \in E$ pour ce y sous l'application f .
 - Considérons l'élément $g(y)$. Puisque g est une application de F vers E , l'image $g(y)$ est nécessairement un élément de l'ensemble E .
 - Posons $x_0 = g(y)$. Nous avons donc $x_0 \in E$.
 - Calculons l'image de cet x_0 par l'application f :

$$f(x_0) = f(g(y))$$

- D'après notre hypothèse (analysée au point 1), nous savons que $f(g(y)) = y$.

- Par conséquent, nous avons $f(x_0) = y$.
- Nous avons ainsi démontré que pour tout $y \in F$ (puisque y était arbitraire), il existe un élément $x_0 \in E$ (à savoir $g(y)$) tel que $f(x_0) = y$.

3. **Conclusion** : Par définition, l'application f est surjective.

Ainsi, s'il existe une application $g : F \rightarrow E$ telle que $f \circ g = \text{Id}_F$, alors f est surjective.

Synthèse

Les deux implications ayant été démontrées avec rigueur, nous pouvons conclure que l'équivalence est vraie :

$$f \text{ est surjective} \iff \exists g : F \rightarrow E \text{ telle que } f \circ g = \text{Id}_F$$

1.15.4 Liens avec l'Intelligence Artificielle

Bien que cet exercice soit de nature purement mathématique et fondamentale, les concepts de surjectivité et d'existence d'un inverse à droite trouvent des échos et des applications indirectes, mais significatives, dans divers domaines de l'Intelligence Artificielle, notamment dans la conception et l'analyse de modèles.

1. Modèles Génératifs (GANs, VAEs) et la “Manifold Hypothesis” :

- Dans les modèles génératifs (comme les Generative Adversarial Networks ou les Variational Autoencoders), l'objectif est d'apprendre une application $G : Z \rightarrow X$, où Z est un espace latent (souvent un espace de faible dimension, comme $\mathbb{R}^d$) et X est l'espace des données réelles (par exemple, des images de haute dimension).
- Idéalement, on voudrait que le générateur G soit “surjectif” sur la *variété des données réelles* (data manifold). Si G n'est pas surjective sur cette variété, cela signifie qu'il existe des données réelles valides et réalistes que le modèle ne pourra jamais générer. Un générateur parfaitement surjectif sur la variété des données serait capable de produire n'importe quelle donnée réaliste.
- L'existence d'un “inverse à droite” $g : X \rightarrow Z$ (souvent appelé *encodeur* ou *inférence inverse*) tel que $G \circ g = \text{Id}_X$ (sur la variété des données) signifierait qu'on peut prendre une donnée réelle x , la mapper à un point latent $z = g(x)$, et que le générateur G peut parfaitement reconstruire x à partir de z . C'est précisément l'objectif des **auto-encodeurs**, où l'encodeur g et le décodeur f (ici G) sont entraînés pour minimiser l'erreur de reconstruction $\|x - f(g(x))\|$. Un auto-encodeur parfait serait un exemple concret de cette relation $f \circ g = \text{Id}_X$.

2. Systèmes de Contrôle et Robotique (Cinématique Inverse) :

- En robotique, la cinématique directe est une fonction $f : \Theta \rightarrow \mathcal{P}$ qui mappe un ensemble d'angles articulaires Θ (l'espace de départ E) à une pose d'effecteur final $\mathcal{P}$ (l'espace d'arrivée F).
- Si un robot peut atteindre n'importe quelle pose dans son espace de travail (c'est-à-dire que f est surjective sur l'espace de travail), alors il existe une fonction de cinématique inverse $g : \mathcal{P} \rightarrow \Theta$ (un “inverse à droite”) qui, pour une pose désirée $p \in \mathcal{P}$, fournit un ensemble d'angles articulaires $\theta = g(p)$.

tel que $f(\theta) = p$. C'est-à-dire $f \circ g = \text{Id}_{\mathcal{P}}$. L'existence d'une telle fonction g est cruciale pour la planification de mouvement et le contrôle des robots, permettant de traduire une tâche dans l'espace opérationnel en commandes articulaires.

3. Apprentissage par Renforcement (Reinforcement Learning) :

- Dans certains contextes, on peut modéliser la politique d'un agent comme une fonction $f : S \rightarrow A$ (état vers action) ou $f : S \rightarrow \mathcal{D}(A)$ (état vers distribution de probabilité sur les actions).
- La surjectivité de f pourrait signifier que l'agent peut potentiellement prendre n'importe quelle action dans n'importe quel état (ou générer n'importe quelle distribution d'actions), ce qui est souvent une hypothèse de base pour l'exploration. L'existence d'un inverse à droite pourrait être liée à la capacité de “reconstruire” l'état à partir de l'action prise, bien que ce soit une analogie plus lâche et contextuelle.

4. Compression de Données et Hachage :

- Les fonctions de hachage $h : D \rightarrow H$ (données vers hachage) ne sont généralement pas surjectives sur l'espace de hachage complet, ni injectives. Cependant, l'idée d'un “inverse à droite” peut être conceptualisée dans des systèmes de compression réversibles où l'on peut décompresser parfaitement. Si une fonction de compression $C : X \rightarrow Y$ est surjective sur l'ensemble des données compressées valides Y , alors il existe une fonction de décompression $D : Y \rightarrow X$ telle que $C \circ D = \text{Id}_Y$. Cela signifie que toute donnée compressée valide peut être décompressée pour retrouver une donnée originale, et que la compression de cette donnée originale donne la donnée compressée de départ.

En résumé, la compréhension de la surjectivité et de l'existence d'inverses à droite est fondamentale pour analyser les capacités et les limites des modèles d'IA, en particulier ceux qui impliquent des transformations d'espaces (comme les auto-encodeurs, les générateurs) ou des mappings entre différentes représentations. Elle permet de raisonner sur la complétude d'un modèle à couvrir son espace cible ou sa capacité à être “inverse” pour certaines opérations.

1.16 Exercice 10

1.17 Exercice 10/10 : Le Théorème de Cantor

Jalon 5 : Applications, injections, surjections, bijections et composition de fonctions

Niveau de difficulté : ★★★★★

1.17.1 Énoncé de l'Exercice

Soit E un ensemble non vide. On note $\mathcal{P}(E)$ l'ensemble de tous les sous-ensembles de E , appelé l'ensemble des parties de E .

Démontrer, en utilisant l'argument diagonal de Cantor, qu'il n'existe aucune fonction surjective de E vers $\mathcal{P}(E)$.

Autrement dit, pour toute fonction $f : E \rightarrow \mathcal{P}(E)$, il existe au moins un sous-ensemble $Y \in \mathcal{P}(E)$ tel que pour tout $x \in E$, $f(x) \neq Y$.

1.17.2 Analyse de l'Énoncé

Cet exercice aborde un résultat fondamental de la théorie des ensembles, le théorème de Cantor, qui a des implications profondes sur la notion de taille (cardinalité) des ensembles, en particulier pour les ensembles infinis.

1. **Ensemble E** : Il s'agit d'un ensemble arbitraire, potentiellement fini ou infini. La démonstration est valable dans tous les cas.
2. **Ensemble $\mathcal{P}(E)$** : C'est l'ensemble de tous les sous-ensembles de E .
 - Si $E = \{1, 2\}$, alors $\mathcal{P}(E) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$.
 - Si E a n éléments, $\mathcal{P}(E)$ a 2^n éléments.
 - Pour $n \geq 1$, on a $2^n > n$. Cela suggère déjà que $\mathcal{P}(E)$ est “plus grand” que E pour les ensembles finis. Le théorème de Cantor généralise cette intuition aux ensembles infinis.
3. **Fonction surjective $f : E \rightarrow \mathcal{P}(E)$** : Une fonction f est surjective si chaque élément de l'ensemble d'arrivée ($\mathcal{P}(E)$ ici) est l'image d'au moins un élément de l'ensemble de départ (E ici).
 - L'énoncé nous demande de prouver qu'une telle surjection *n'existe pas*.
 - Cela signifie que, quelle que soit la fonction f que l'on choisit de E vers $\mathcal{P}(E)$, on doit pouvoir trouver un sous-ensemble $Y \in \mathcal{P}(E)$ qui n'est l'image d'aucun élément de E par f . C'est-à-dire, $Y \notin \text{Im}(f)$.
4. **L'argument diagonal de Cantor** : C'est la technique de preuve clé. Elle consiste à construire un élément “spécial” $Y \in \mathcal{P}(E)$ qui est défini de manière à différer de *toutes* les images $f(x)$ pour *tous* les $x \in E$. L'idée est de créer un “point de désaccord” avec chaque $f(x)$.

Ce théorème est d'une importance capitale car il montre qu'il existe différents “niveaux” d'infini. Par exemple, l'ensemble des nombres réels $\mathbb{R}$ a la même cardinalité que $\mathcal{P}(\mathbb{N})$ (l'ensemble des parties des nombres naturels), et le théorème de Cantor implique donc que $|\mathbb{N}| < |\mathbb{R}|$.

1.17.3 Correction Exhaustive Pas-à-Pas

Nous allons procéder par l'absurde.

Étape 1 : Supposition par l'absurde

Supposons qu'il existe une fonction $f : E \rightarrow \mathcal{P}(E)$ qui est surjective. Par définition de la surjectivité, cela signifie que pour tout $Y' \in \mathcal{P}(E)$, il existe au moins un $x' \in E$ tel que $f(x') = Y'$.

Étape 2 : Construction de l'ensemble diagonal de Cantor

Nous allons maintenant construire un sous-ensemble $Y \in \mathcal{P}(E)$ qui sera la contradiction recherchée. Définissons l'ensemble Y comme suit :

$$Y = \{x \in E \mid x \notin f(x)\}$$

Analysons cette définition : * Pour chaque élément $x \in E$, $f(x)$ est un sous-ensemble de E , c'est-à-dire $f(x) \in \mathcal{P}(E)$. * La condition " $x \notin f(x)$ " est une proposition logique qui est soit vraie, soit fausse pour chaque $x \in E$. * L'ensemble Y est donc l'ensemble de tous les éléments x de E qui n'appartiennent *pas* à l'image d'eux-mêmes par f . * Par l'axiome de compréhension, l'ensemble Y est formé d'éléments x qui appartiennent préalablement à E . Ainsi, pour tout $x \in Y$, nous avons logiquement $x \in E$, ce qui caractérise l'inclusion $Y \subseteq E$. Par conséquent, par définition de l'ensemble des parties, $Y \in \mathcal{P}(E)$.

Étape 3 : Utilisation de l'hypothèse de surjectivité

Puisque nous avons supposé que f est surjective et que $Y \in \mathcal{P}(E)$, il doit exister un élément $a \in E$ tel que $f(a) = Y$. Ceci découle directement de la définition de la surjectivité : chaque élément de l'ensemble d'arrivée ($\mathcal{P}(E)$) doit être l'image d'au moins un élément de l'ensemble de départ (E). Puisque Y est un élément de $\mathcal{P}(E)$, il doit avoir un antécédent a dans E .

Étape 4 : Dérivation de la contradiction

Considérons l'élément $a \in E$ que nous avons trouvé à l'Étape 3, tel que $f(a) = Y$. Nous allons maintenant nous poser la question cruciale : l'élément a appartient-il à l'ensemble Y ?

Nous avons deux cas possibles, qui sont mutuellement exclusifs et exhaustifs :

Cas 1 : Supposons que $a \in Y$. 1. Si $a \in Y$, alors, par la définition de l'ensemble Y (Étape 2), tout élément $z \in Y$ satisfait la condition $z \notin f(z)$. 2. En appliquant cette définition à a , nous obtenons $a \notin f(a)$. 3. Cependant, nous savons de l'Étape 3 que $f(a) = Y$. 4. En substituant $f(a)$ par Y dans l'expression $a \notin f(a)$, nous obtenons $a \notin Y$. 5. Nous sommes arrivés à une contradiction : nous avons supposé $a \in Y$ et nous avons déduit $a \notin Y$. Ceci est logiquement impossible.

Cas 2 : Supposons que $a \notin Y$. 1. Si $a \notin Y$, alors, par la définition de l'ensemble Y (Étape 2), tout élément $z \in E$ qui n'est pas dans Y doit satisfaire la négation de la condition de Y , c'est-à-dire $z \in f(z)$. 2. En appliquant cette logique à a , nous obtenons $a \in f(a)$. 3. Cependant, nous savons de l'Étape 3 que $f(a) = Y$. 4. En substituant $f(a)$ par Y dans l'expression $a \in f(a)$, nous obtenons $a \in Y$. 5. Nous sommes arrivés à une contradiction : nous avons supposé $a \notin Y$ et nous avons déduit $a \in Y$. Ceci est également logiquement impossible.

Étape 5 : Conclusion

Dans les deux cas possibles (que $a \in Y$ ou $a \notin Y$), nous aboutissons à une contradiction logique. Puisque notre raisonnement est correct et que les deux cas couvrent toutes les

possibilités pour a , la seule conclusion possible est que notre supposition initiale (Étape 1) doit être fausse.

Par conséquent, il n'existe aucune fonction surjective $f : E \rightarrow \mathcal{P}(E)$. Ceci démontre le théorème de Cantor.

1.17.4 Liens avec l'Intelligence Artificielle

Le théorème de Cantor, bien que fondamentalement mathématique, a des résonances profondes et des analogies frappantes dans le domaine de l'Intelligence Artificielle et de l'informatique théorique :

1. **Le Problème de l'Arrêt (Halting Problem) :** C'est l'analogie la plus directe et la plus célèbre. Le problème de l'arrêt, qui demande s'il est possible de déterminer pour un programme arbitraire et une entrée arbitraire si le programme finira par s'arrêter ou s'exécutera indéfiniment, a été prouvé indécidable par Alan Turing en utilisant un argument diagonal très similaire à celui de Cantor.
 - Imaginez une fonction f qui, pour chaque programme P (élément de E), renvoie un ensemble de programmes $f(P)$ avec lesquels P interagit d'une certaine manière.
 - L'argument diagonal construit un programme "paradoxal" qui, si on lui applique la logique de f , mène à une contradiction, prouvant ainsi qu'une telle f (qui résoudrait le problème de l'arrêt) ne peut exister.
 - Cela établit une limite fondamentale à ce que les ordinateurs peuvent calculer, même en théorie.
2. **Limites de la Représentation et de l'Énumération :**
 - Le théorème de Cantor montre que l'ensemble des sous-ensembles d'un ensemble est "plus grand" que l'ensemble lui-même. En IA, cela peut être interprété comme une limite à la capacité de représenter toutes les "idées" ou "concepts" (qui pourraient être vus comme des sous-ensembles d'un espace de base) en utilisant un système de symboles ou un langage formel qui a la même "taille" que l'espace de base.
 - Par exemple, l'ensemble des fonctions de $\mathbb{N}$ vers $\mathbb{N}$ est non-dénombrable (a la même cardinalité que $\mathcal{P}(\mathbb{N})$), tandis que l'ensemble des programmes informatiques est dénombrable. Cela signifie qu'il existe infiniment plus de fonctions que de programmes pour les calculer. La plupart des fonctions ne sont donc pas calculables par un algorithme. C'est une limite fondamentale pour l'IA symbolique et algorithmique.
3. **Théorèmes d'Incomplétude de Gödel :** Bien que plus complexes, les théorèmes d'incomplétude de Gödel, qui démontrent les limites des systèmes axiomatiques formels, utilisent également des techniques de "référence à soi-même" et des constructions de type diagonal. Ils montrent qu'il y aura toujours des énoncés vrais qui ne peuvent pas être prouvés au sein d'un système formel suffisamment puissant pour contenir l'arithmétique. Ces limites sont cruciales pour comprendre les fondements de la logique et de la connaissance, des domaines centraux pour l'IA.
4. **Complexité et Expressivité des Modèles :** Dans l'apprentissage automatique, nous construisons des modèles (par exemple, des réseaux de neurones) pour apprendre des fonctions. Le théorème de Cantor, à un niveau très abstrait, nous

rappelle que l'espace des fonctions possibles est immensément vaste. Même si nos modèles sont très expressifs, ils ne peuvent “représenter” qu’une infime fraction de toutes les fonctions possibles. Cela souligne l’importance de la régularisation, des biais inductifs et de la recherche d’architectures qui capturent les propriétés pertinentes des fonctions que nous voulons apprendre, plutôt que de tenter de couvrir l’espace entier.

En somme, le théorème de Cantor est un rappel puissant des limites fondamentales de la logique, de la calculabilité et de la représentation, des concepts qui sont au cœur des défis théoriques et pratiques de l’Intelligence Artificielle.

Chapitre 2

Travaux Pratiques

2.1 TP 1

2.2 TP 1/5 - Jalon 5 : Injectivité et Surjectivité sur Ensembles Finis

2.2.1 Introduction

Ce premier Travail Pratique du Jalon 5 se concentre sur la compréhension et l'implémentation des concepts fondamentaux d'applications (fonctions), d'injectivité et de surjectivité dans le contexte des ensembles finis. En mathématiques, une application f d'un ensemble A (domaine) vers un ensemble B (codomaine) associe à chaque élément de A un unique élément de B .

Nous allons représenter ces applications à l'aide de dictionnaires Python, où les clés représenteront les éléments du domaine A et les valeurs les éléments correspondants du codomaine B . Cette approche nous permettra de manipuler des ensembles finis et de vérifier les propriétés d'injectivité et de surjectivité de manière algorithmique.

Rappels Mathématiques

Soit une application $f : A \rightarrow B$.

- **Injectivité** : L'application f est dite **injective** si et seulement si deux éléments distincts du domaine A ont toujours des images distinctes dans le codomaine B . Formellement : $\forall x_1, x_2 \in A, (x_1 \neq x_2 \implies f(x_1) \neq f(x_2))$. Une formulation équivalente est : $\forall x_1, x_2 \in A, (f(x_1) = f(x_2) \implies x_1 = x_2)$. En d'autres termes, chaque élément du codomaine est l'image d'au plus un élément du domaine.
- **Surjectivité** : L'application f est dite **surjective** si et seulement si chaque élément du codomaine B est l'image d'au moins un élément du domaine A . Formellement : $\forall y \in B, \exists x \in A$ tel que $f(x) = y$. Cela signifie que l'image de f , notée $Im(f) = \{f(x) \mid x \in A\}$, est égale au codomaine B .
- **Bijectivité** : L'application f est dite **bijjective** si et seulement si elle est à la fois injective et surjective.

2.2.2 Objectifs du TP

1. Représenter une application $f : A \rightarrow B$ à l'aide d'un dictionnaire Python.

2. Implémenter une fonction Python `is_injective(f_dict)` qui vérifie si une application donnée est injective.
3. Implémenter une fonction Python `is_surjective(f_dict, codomain_set)` qui vérifie si une application donnée est surjective.
4. (Bonus) Implémenter une fonction Python `is_bijective(f_dict, codomain_set)` qui vérifie si une application donnée est bijective.
5. Utiliser des assertions pour valider la justesse mathématique des implémentations.

2.2.3 Prérequis

- Connaissances de base en Python (dictionnaires, ensembles, boucles, conditions).
- Compréhension des concepts d'ensembles, de fonctions, d'injectivité et de surjectivité.

2.2.4 Partie 1 : Représentation d'une fonction

Une fonction $f : A \rightarrow B$ peut être représentée par un dictionnaire où les clés sont les éléments du domaine A et les valeurs sont leurs images dans B . Il est crucial de noter que le dictionnaire ne contient que les paires $(x, f(x))$ pour $x \in A$. Le codomaine B doit être fourni séparément pour vérifier la surjectivité, car le dictionnaire seul ne permet de déduire que l'image de la fonction, pas l'ensemble B complet.

Exemple de représentation : Si $A = \{1, 2, 3\}$ et $B = \{a, b, c, d\}$, et $f(1) = a, f(2) = b, f(3) = c$, alors la fonction peut être représentée par `f_dict = {1: 'a', 2: 'b', 3: 'c'}`. Le codomaine serait `codomain_set = {'a', 'b', 'c', 'd'}`.

```

1 # --- Partie 1 : Représentation d'une fonction ---
2
3 def get_domain(f_dict: dict) -> set:
4     """
5     Extrait le domaine (ensemble de départ) d'une fonction
6     représentée par un dictionnaire.
7     Le domaine est l'ensemble de toutes les clés du dictionnaire.
8
9     Args:
10         f_dict (dict): Dictionnaire représentant la fonction {x: f(x)}.
11
12     Returns:
13         set: L'ensemble des éléments du domaine.
14     """
15     return set(f_dict.keys())
16
17 def get_image(f_dict: dict) -> set:
18     """
19     Extrait l'image (ensemble d'arrivée des valeurs réellement
20     atteintes) d'une fonction.
21     L'image est l'ensemble de toutes les valeurs du dictionnaire.
22
23     Args:

```

```

22         f_dict (dict): Dictionnaire repr sentant la fonction {x:
                f(x)}.
23
24     Returns:
25         set: L'ensemble des lments de l'image.
26     """
27     return set(f_dict.values())
28
29 # Exemple de fonction
30 f_example = {
31     'pomme': 'fruit',
32     'carotte': 'l gume',
33     'banane': 'fruit',
34     'brocoli': 'l gume'
35 }
36
37 # D finition explicite du codomaine pour les tests de
    surjectivit
38 codomain_example = {'fruit', 'l gume', 'c r ale'}
39
40 print(f"Fonction f_example: {f_example}")
41 print(f"Domaine de f_example: {get_domain(f_example)}")
42 print(f"Image de f_example: {get_image(f_example)}")
43 print(f"Codomaine explicite pour f_example: {codomain_example}")
44
45 # Assertions pour valider les fonctions utilitaires
46 assert get_domain(f_example) == {'pomme', 'carotte', 'banane', '
    brocoli'}
47 assert get_image(f_example) == {'fruit', 'l gume'}
48 print("\nAssertions pour les fonctions utilitaires pass es avec
    succ s.")

```

2.2.5 Partie 2 : Vérification de l'injectivité

Pour vérifier l'injectivité d'une fonction $f : A \rightarrow B$, nous devons nous assurer qu'aucun élément du codomaine n'est l'image de plus d'un élément du domaine. En d'autres termes, si nous collectons toutes les images $f(x)$ pour $x \in A$, il ne doit pas y avoir de doublons parmi ces images.

Algorithme : 1. Extraire toutes les valeurs (images) de la fonction représentée par le dictionnaire. 2. Convertir cette collection de valeurs en un ensemble. Un ensemble ne contient pas de doublons. 3. Si la taille de la collection originale de valeurs est égale à la taille de l'ensemble des valeurs (c'est-à-dire qu'il n'y a pas eu de suppression de doublons), alors la fonction est injective. Sinon, elle ne l'est pas.

```

1 # --- Partie 2 : V rification de l'injectivit ---
2
3 def is_injective(f_dict: dict) -> bool:
4     """
5     V rifie si une fonction repr sent e par un dictionnaire est
        injective.

```

```

6     Une fonction est injective si chaque élément du codomaine
       est l'image
7     d'au plus un élément du domaine.
8     Algorithmiquement, cela signifie que toutes les valeurs (
       images) dans le dictionnaire
9     doivent être uniques.
10
11     Args:
12         f_dict (dict): Dictionnaire représentant la fonction {x:
                          f(x)}.
13
14     Returns:
15         bool: True si la fonction est injective, False sinon.
16     """
17     # tape 1: Obtenir toutes les valeurs (images) de la fonction
18     # Les valeurs peuvent contenir des doublons si la fonction n'
       est pas injective.
19     values = list(f_dict.values())
20
21     # tape 2: Créer un ensemble à partir de ces valeurs.
22     # Un ensemble élimine automatiquement les doublons.
23     unique_values = set(values)
24
25     # tape 3: Comparer la taille de la liste originale des
       valeurs avec la taille de l'ensemble.
26     # Si les tailles sont égales, cela signifie qu'il n'y avait
       pas de doublons,
27     # donc chaque élément du domaine a une image unique.
28     return len(values) == len(unique_values)
29
30 # --- Tests et Assertions pour l'injectivité ---
31
32 print("\n--- Tests d'injectivité ---")
33
34 # Fonction injective (chaque élément du domaine a une image
       unique)
35 f_injective = {'a': 1, 'b': 2, 'c': 3}
36 # Mathématiquement: f(a)=1, f(b)=2, f(c)=3. Toutes les images
       sont distinctes.
37 print(f"f_injective: {f_injective} est injective? {is_injective(
       f_injective)}")
38 assert is_injective(f_injective) is True, "f_injective devrait
       être injective"
39
40 # Fonction non injective (deux éléments du domaine ont la même
       image)
41 f_not_injective = {'a': 1, 'b': 2, 'c': 1}
42 # Mathématiquement: f(a)=1 et f(c)=1, mais a != c. Donc non
       injective.
43 print(f"f_not_injective: {f_not_injective} est injective? {

```

```

    is_injective(f_not_injective)}")
44 assert is_injective(f_not_injective) is False, "f_not_injective ne
    devrait PAS tre injective"
45
46 # Fonction avec un seul lment (toujours injective)
47 f_single_element = {'x': 10}
48 print(f"f_single_element: {f_single_element} est injective? {
    is_injective(f_single_element)}")
49 assert is_injective(f_single_element) is True, "f_single_element
    devrait tre injective"
50
51 # Fonction vide (toujours injective par d finition, car la
    condition n'est jamais viol e)
52 f_empty = {}
53 print(f"f_empty: {f_empty} est injective? {is_injective(f_empty)}"
    )
54 assert is_injective(f_empty) is True, "f_empty devrait tre
    injective"
55
56 print("Assertions d'injectivit pass es avec succ s.")

```

2.2.6 Partie 3 : Vérification de la surjectivité

Pour vérifier la surjectivité d'une fonction $f : A \rightarrow B$, nous devons nous assurer que chaque élément du codomaine B est atteint par au moins un élément du domaine A . Cela signifie que l'ensemble des images de la fonction (l'image de f) doit être exactement égal au codomaine B .

Algorithme : 1. Extraire l'ensemble des images de la fonction (les valeurs du dictionnaire). 2. Comparer cet ensemble d'images avec l'ensemble du codomaine B fourni explicitement. 3. Si les deux ensembles sont égaux, la fonction est surjective. Sinon, elle ne l'est pas.

```

1 # --- Partie 3 : V rification de la surjectivit ---
2
3 def is_surjective(f_dict: dict, codomain_set: set) -> bool:
4     """
5     V rifie si une fonction repr sent e par un dictionnaire est
        surjective par rapport
6     un codomaine donn .
7     Une fonction est surjective si chaque lment du codomaine
        est l'image
8     d'au moins un lment du domaine.
9     Algorithmiquement, cela signifie que l'ensemble des images de
        la fonction
10    doit tre gal l'ensemble du codomaine.
11
12    Args:
13        f_dict (dict): Dictionnaire repr sentant la fonction {x:
            f(x)}.
14        codomain_set (set): L'ensemble explicite du codomaine B.
15

```

```

16     Returns:
17         bool: True si la fonction est surjective, False sinon.
18     """
19     # tape 1: Obtenir l'ensemble des images de la fonction.
20     # C'est l'ensemble de toutes les valeurs presentes dans le
21     # dictionnaire.
22     image_set = set(f_dict.values())
23
24     # tape 2: Comparer l'ensemble des images avec l'ensemble du
25     # codomaine.
26     # Si l'image est gale au codomaine, alors tous les
27     # elements du codomaine
28     # sont atteints par la fonction.
29     return image_set == codomain_set
30
31 # --- Tests et Assertions pour la surjectivit ---
32
33 print("\n--- Tests de surjectivit ---")
34
35 # Fonction surjective (l'image couvre tout le codomaine)
36 f_surjective = {'a': 1, 'b': 2, 'c': 3}
37 codomain_123 = {1, 2, 3}
38 # Math matiquement: Im(f) = {1,2,3}. Codomaine = {1,2,3}. Im(f)
39 # == Codomaine. Surjective.
40 print(f"f_surjective: {f_surjective}, Codomaine: {codomain_123}
41 est surjective? {is_surjective(f_surjective, codomain_123)}")
42 assert is_surjective(f_surjective, codomain_123) is True, "
43 f_surjective devrait tre surjective"
44
45 # Fonction non surjective (le codomaine contient des elements
46 # non atteints)
47 f_not_surjective = {'a': 1, 'b': 2, 'c': 2}
48 codomain_123 = {1, 2, 3}
49 # Math matiquement: Im(f) = {1,2}. Codomaine = {1,2,3}. Im(f) !=
50 # Codomaine (3 n'est pas atteint). Non surjective.
51 print(f"f_not_surjective: {f_not_surjective}, Codomaine: {
52 codomain_123} est surjective? {is_surjective(f_not_surjective,
53 codomain_123)}")
54 assert is_surjective(f_not_surjective, codomain_123) is False, "
55 f_not_surjective ne devrait PAS tre surjective"
56
57 # Fonction surjective (m me si non injective)
58 f_surjective_not_injective = {'a': 1, 'b': 2, 'c': 2, 'd': 1}
59 codomain_12 = {1, 2}
60 # Math matiquement: Im(f) = {1,2}. Codomaine = {1,2}. Im(f) ==
61 # Codomaine. Surjective.
62 print(f"f_surjective_not_injective: {f_surjective_not_injective},
63 Codomaine: {codomain_12} est surjective? {is_surjective(
64 f_surjective_not_injective, codomain_12)}")
65 assert is_surjective(f_surjective_not_injective, codomain_12) is
66 True, "f_surjective_not_injective devrait tre surjective"

```

```

52
53 # Fonction vide (non surjective si le codomaine n'est pas vide)
54 f_empty = {}
55 codomain_non_empty = {1}
56 # Math matiquement: Im(f) = {}. Codomaine = {1}. Im(f) !=
    Codomaine. Non surjective.
57 print(f"f_empty: {f_empty}, Codomaine: {codomain_non_empty} est
    surjective? {is_surjective(f_empty, codomain_non_empty)}")
58 assert is_surjective(f_empty, codomain_non_empty) is False, "
    f_empty ne devrait PAS tre surjective si codomaine non vide"
59
60 # Fonction vide (surjective si le codomaine est vide)
61 f_empty = {}
62 codomain_empty = set()
63 # Math matiquement: Im(f) = {}. Codomaine = {}. Im(f) ==
    Codomaine. Surjective.
64 print(f"f_empty: {f_empty}, Codomaine: {codomain_empty} est
    surjective? {is_surjective(f_empty, codomain_empty)}")
65 assert is_surjective(f_empty, codomain_empty) is True, "f_empty
    devrait tre surjective si codomaine vide"
66
67 print("Assertions de surjectivit pass es avec succ s.")

```

2.2.7 Partie 4 : Vérification de la bijectivité (Bonus)

Une fonction est bijective si et seulement si elle est à la fois injective et surjective. L'implémentation est donc directe en utilisant les fonctions précédemment définies.

```

1 # --- Partie 4 : V rification de la bijectivit (Bonus) ---
2
3 def is_bijective(f_dict: dict, codomain_set: set) -> bool:
4     """
5     V rifie si une fonction est bijective.
6     Une fonction est bijective si elle est la fois injective et
        surjective.
7
8     Args:
9         f_dict (dict): Dictionnaire repr sentant la fonction {x:
        f(x)}.
10        codomain_set (set): L'ensemble explicite du codomaine B.
11
12    Returns:
13        bool: True si la fonction est bijective, False sinon.
14    """
15    # Une fonction est bijective si elle est injective ET
        surjective.
16    return is_injective(f_dict) and is_surjective(f_dict,
        codomain_set)
17
18 # --- Tests et Assertions pour la bijectivit ---
19

```

```

20 print("\n--- Tests de bijectivité ---")
21
22 # Fonction bijective (injective et surjective)
23 f_bijective = {'a': 1, 'b': 2, 'c': 3}
24 codomain_123 = {1, 2, 3}
25 # Mathématiquement: Injective (images distinctes), Surjective (Im
    (f) == Codomaine). Bijective.
26 print(f"f_bijective: {f_bijective}, Codomaine: {codomain_123} est
    bijective? {is_bijective(f_bijective, codomain_123)}")
27 assert is_bijective(f_bijective, codomain_123) is True, "
    f_bijective devrait être bijective"
28
29 # Fonction non bijective (non injective mais surjective)
30 f_not_bijective_not_injective = {'a': 1, 'b': 2, 'c': 2}
31 codomain_12 = {1, 2}
32 # Mathématiquement: Non injective (f(b)=f(c)=2), Surjective (Im(f)
    == Codomaine). Non bijective.
33 print(f"f_not_bijective_not_injective: {
    f_not_bijective_not_injective}, Codomaine: {codomain_12} est
    bijective? {is_bijective(f_not_bijective_not_injective,
    codomain_12)}")
34 assert is_bijective(f_not_bijective_not_injective, codomain_12) is
    False, "f_not_bijective_not_injective ne devrait PAS être
    bijective"
35
36 # Fonction non bijective (injective mais non surjective)
37 f_not_bijective_not_surjective = {'a': 1, 'b': 2}
38 codomain_123 = {1, 2, 3}
39 # Mathématiquement: Injective (images distinctes), Non surjective
    (3 non atteint). Non bijective.
40 print(f"f_not_bijective_not_surjective: {
    f_not_bijective_not_surjective}, Codomaine: {codomain_123} est
    bijective? {is_bijective(f_not_bijective_not_surjective,
    codomain_123)}")
41 assert is_bijective(f_not_bijective_not_surjective, codomain_123)
    is False, "f_not_bijective_not_surjective ne devrait PAS être
    bijective"
42
43 # Fonction non bijective (ni injective ni surjective)
44 f_neither = {'a': 1, 'b': 1, 'c': 2}
45 codomain_123 = {1, 2, 3}
46 # Mathématiquement: Non injective (f(a)=f(b)=1), Non surjective
    (3 non atteint). Non bijective.
47 print(f"f_neither: {f_neither}, Codomaine: {codomain_123} est
    bijective? {is_bijective(f_neither, codomain_123)}")
48 assert is_bijective(f_neither, codomain_123) is False, "f_neither
    ne devrait PAS être bijective"
49
50 # Fonction vide (bijective si et seulement si le codomaine est
    vide)
51 f_empty = {}

```

```

52 codomain_empty = set()
53 print(f"f_empty: {f_empty}, Codomain: {codomain_empty} est
    bijective? {is_bijective(f_empty, codomain_empty)}")
54 assert is_bijective(f_empty, codomain_empty) is True, "f_empty
    devrait tre bijective si codomaine vide"
55
56 f_empty_codomain_non_empty = {}
57 codomain_non_empty = {1}
58 print(f"f_empty: {f_empty_codomain_non_empty}, Codomain: {
    codomain_non_empty} est bijective? {is_bijective(
    f_empty_codomain_non_empty, codomain_non_empty)}")
59 assert is_bijective(f_empty_codomain_non_empty, codomain_non_empty
    ) is False, "f_empty ne devrait PAS tre bijective si
    codomaine non vide"
60
61 print("Assertions de bijectivit pass es avec succ s.")

```

2.2.8 Conclusion

Ce TP vous a permis d'implémenter les vérifications d'injectivité et de surjectivité pour des fonctions sur des ensembles finis, représentées par des dictionnaires Python. Vous avez vu comment traduire des définitions mathématiques rigoureuses en algorithmes concrets et comment valider ces implémentations à l'aide d'assertions.

La distinction entre l'image d'une fonction (ce que le dictionnaire contient comme valeurs) et le codomaine (l'ensemble de toutes les valeurs possibles à l'arrivée) est fondamentale pour la surjectivité.

Pour aller plus loin, vous pourriez explorer : * Comment représenter des fonctions avec des domaines et codomaines infinis (nécessiterait une approche différente, non basée sur des énumérations exhaustives). * L'implémentation de la composition de fonctions, qui sera le sujet du prochain TP. * La recherche de la fonction inverse pour une fonction bijective.

2.3 TP 2

2.4 TP 2/5 : Composition de Fonctions - Implémentation et Vérification Empirique

Jalon 5 : Applications, injections, surjections, bijections et composition de fonctions

2.4.1 Introduction

En tant que développeurs et mathématiciens, la capacité à manipuler et composer des fonctions est fondamentale. La composition de fonctions est une opération qui prend deux fonctions et en produit une troisième, qui représente l'application successive des deux fonctions originales. Ce concept est au cœur de nombreuses constructions mathématiques et informatiques, de la programmation fonctionnelle aux architectures de réseaux neuronaux.

Ce Travail Pratique (TP) vise à solidifier votre compréhension de la composition de fonctions en vous demandant de l'implémenter en Python de manière générique, puis de valider empiriquement votre implémentation à l'aide de tests rigoureux.

2.4.2 Objectifs du TP

1. **Comprendre la définition mathématique** de la composition de fonctions.
 2. **Implémenter une fonction générique** `compose(f, g)` en Python pur, qui retourne une nouvelle fonction représentant $g \circ f$.
 3. **Utiliser des assertions** (`assert`) pour valider mathématiquement la justesse de l'implémentation avec divers exemples.
 4. **Développer une approche rigoureuse** de test et de vérification de code.
-

2.4.3 Partie 1 : Rappels Mathématiques sur la Composition de Fonctions

Soient deux fonctions $f : A \rightarrow B$ et $g : B \rightarrow C$. La **composition de f par g** , notée $g \circ f$ (lire “g rond f”), est une nouvelle fonction définie par :

$$(g \circ f)(x) = g(f(x))$$

pour tout x dans le domaine de f .

Points clés : * L'ordre est crucial : $g \circ f$ signifie que f est appliquée en premier, puis g est appliquée au résultat de f . * Le codomaine de la première fonction (f) doit être compatible avec le domaine de la seconde fonction (g). Plus précisément, l'image de f doit être un sous-ensemble du domaine de g . * La fonction résultante $g \circ f$ a pour domaine A et pour codomaine C .

Exemple : Si $f(x) = x + 1$ et $g(x) = 2x$. Alors $(g \circ f)(x) = g(f(x)) = g(x + 1) = 2(x + 1) = 2x + 2$. Et $(f \circ g)(x) = f(g(x)) = f(2x) = 2x + 1$. On observe que $g \circ f \neq f \circ g$, la composition n'est pas commutative en général.

2.4.4 Partie 2 : Implémentation de la Fonction `compose(f, g)`

Votre tâche est d'écrire une fonction Python `compose(f, g)` qui prend deux fonctions `f` et `g` en arguments et retourne une nouvelle fonction. Cette nouvelle fonction doit représenter la composition $g \circ f$.

Contraintes : * Python PUR : Aucune bibliothèque externe (comme NumPy, SciPy, etc.) n'est autorisée. * Le code doit être profondément commenté pour expliquer chaque étape et la logique mathématique sous-jacente.

```

1 def compose(f, g):
2     """
3     Impl mente la composition de deux fonctions f et g.
4     Math matiquement, cela correspond      (g o f)(x) = g(f(x)).
5
6     Args:
7         f (callable): La premi re fonction      appliquer. Elle
8                     prend un argument
9                     et retourne une valeur.
10        g (callable): La seconde fonction      appliquer. Elle prend
11                    le r sultat
12                    de f comme argument et retourne une valeur.
13
14    Returns:
15        callable: Une nouvelle fonction qui repr sente la
16                  composition (g o f).
17                  Cette fonction prend un argument 'x' et retourne
18                  g(f(x)).
19
20    Pr conditions (implicites en Python, mais importantes
21    math matiquement) :
22        - Le codomaine de f doit tre compatible avec le domaine
23          de g.
24        C'est- -dire que le type et la nature de la valeur
25          retourn e par f(x)
26          doivent tre acceptables comme argument pour g.
27    """
28    # La fonction 'compose' doit retourner une nouvelle fonction.
29    # Nous utilisons une fonction interne (closure) pour capturer
30    # 'f' et 'g'
31    # de l'environnement englobant.
32    def composed_function(x):
33        """
34        La fonction r sultante de la composition (g o f).
35        Elle applique f      x en premier, puis applique g au
36        r sultat de f(x).

```

```

28     """
29     #  tape 1 : Appliquer la premi re fonction 'f'      1'
        argument 'x'.
30     #  Le r sultat de cette op ration est la valeur
        interm diaire.
31     result_f = f(x)
32
33     #  tape 2 : Appliquer la seconde fonction 'g' au
        r sultat de 'f(x)'.
34     #  C'est l'essence m me de la d finition (g o f)(x) = g(f
        (x)).
35     result_g_of_f = g(result_f)
36
37     #  Retourner le r sultat final de la composition.
38     return result_g_of_f
39
40     #  Retourner la fonction compos e , pas son ex cution.
41     return composed_function

```

2.4.5 Partie 3 : Vérification Empirique et Tests Unitaires (avec assert)

Pour valider votre implémentation, vous allez créer plusieurs fonctions simples et les composer. Ensuite, vous utiliserez des assertions pour vérifier que le résultat de la fonction composée correspond bien au calcul manuel attendu.

Pour chaque test, vous devrez : 1. Définir les fonctions f et g . 2. Calculer manuellement l'expression de $(g \circ f)(x)$. 3. Appliquer votre fonction `compose` pour obtenir la fonction $h = \text{compose}(f, g)$. 4. Tester h avec plusieurs valeurs d'entrée x et utiliser `assert` pour comparer le résultat de $h(x)$ avec le calcul manuel.

Exemple de Structure de Test

```

1  # --- D finition des fonctions de base ---
2  def add_one(x):
3      return x + 1
4
5  def multiply_by_two(x):
6      return x * 2
7
8  # --- Test 1 : (multiply_by_two o add_one)(x) ---
9  # Calcul manuel : (multiply_by_two o add_one)(x) = multiply_by_two
        (add_one(x))
10 #
        = multiply_by_two
        (x + 1)
11 #
        = 2 * (x + 1)
12 #
        = 2x + 2
13

```

```

14 # Cr ation de la fonction compos e
15 h1 = compose(add_one, multiply_by_two)
16
17 # V rifications avec assert
18 print(f"Test 1: (multiply_by_two o add_one)(x) = 2x + 2")
19 assert h1(0) == 2 * 0 + 2, f"Erreur pour x=0: Attendu {2*0+2},
    Obtenu {h1(0)}"
20 assert h1(1) == 2 * 1 + 2, f"Erreur pour x=1: Attendu {2*1+2},
    Obtenu {h1(1)}"
21 assert h1(5) == 2 * 5 + 2, f"Erreur pour x=5: Attendu {2*5+2},
    Obtenu {h1(5)}"
22 assert h1(-3) == 2 * (-3) + 2, f"Erreur pour x=-3: Attendu {2*(-3)
    +2}, Obtenu {h1(-3)}"
23 print(" -> Test 1 r ussi.")
24
25 # --- Test 2 : (add_one o multiply_by_two)(x) ---
26 # Calcul manuel : (add_one o multiply_by_two)(x) = add_one(
    multiply_by_two(x))
27 #                                     = add_one(2x)
28 #                                     = 2x + 1
29
30 # Cr ation de la fonction compos e
31 h2 = compose(multiply_by_two, add_one)
32
33 # V rifications avec assert
34 print(f"\nTest 2: (add_one o multiply_by_two)(x) = 2x + 1")
35 assert h2(0) == 2 * 0 + 1, f"Erreur pour x=0: Attendu {2*0+1},
    Obtenu {h2(0)}"
36 assert h2(1) == 2 * 1 + 1, f"Erreur pour x=1: Attendu {2*1+1},
    Obtenu {h2(1)}"
37 assert h2(5) == 2 * 5 + 1, f"Erreur pour x=5: Attendu {2*5+1},
    Obtenu {h2(5)}"
38 assert h2(-3) == 2 * (-3) + 1, f"Erreur pour x=-3: Attendu {2*(-3)
    +1}, Obtenu {h2(-3)}"
39 print(" -> Test 2 r ussi.")

```

Exercices de Vérification

Implémentez les tests suivants en utilisant la même structure.

Exercice 3.1 : Fonctions polynomiales Définissez les fonctions suivantes : * square(x) qui retourne x^2 . * subtract_five(x) qui retourne $x - 5$.

Testez les compositions suivantes : 1. (subtract_five o square)(x) 2. (square o subtract_five)(x)

```

1 # --- Exercice 3.1 : Fonctions polynomiales ---
2
3 def square(x):
4     """Retourne le carré de x."""
5     return x ** 2
6

```

```

7 def subtract_five(x):
8     """Retourne x moins 5."""
9     return x - 5
10
11 # 1. Composition (subtract_five o square)(x)
12 # Calcul manuel : (subtract_five o square)(x) = subtract_five(
13     square(x))
14 #                                     = subtract_five(x
15     **2)
16 #                                     = x**2 - 5
17 h3_1 = compose(square, subtract_five)
18 print(f"\nExercice 3.1.1: (subtract_five o square)(x) = x**2 - 5")
19 assert h3_1(0) == 0**2 - 5, f"Erreur pour x=0: Attendu {0**2-5},
20     Obtenu {h3_1(0)}"
21 assert h3_1(2) == 2**2 - 5, f"Erreur pour x=2: Attendu {2**2-5},
22     Obtenu {h3_1(2)}"
23 assert h3_1(-2) == (-2)**2 - 5, f"Erreur pour x=-2: Attendu {(-2)
24     **2-5}, Obtenu {h3_1(-2)}"
25 assert h3_1(3) == 3**2 - 5, f"Erreur pour x=3: Attendu {3**2-5},
26     Obtenu {h3_1(3)}"
27 print("    -> Exercice 3.1.1 r ussi.")
28
29 # 2. Composition (square o subtract_five)(x)
30 # Calcul manuel : (square o subtract_five)(x) = square(
31     subtract_five(x))
32 #                                     = square(x - 5)
33 #                                     = (x - 5)**2
34 h3_2 = compose(subtract_five, square)
35 print(f"\nExercice 3.1.2: (square o subtract_five)(x) = (x - 5)**2
36     ")
37 assert h3_2(0) == (0 - 5)**2, f"Erreur pour x=0: Attendu {(0-5)
38     **2}, Obtenu {h3_2(0)}"
39 assert h3_2(5) == (5 - 5)**2, f"Erreur pour x=5: Attendu {(5-5)
40     **2}, Obtenu {h3_2(5)}"
41 assert h3_2(7) == (7 - 5)**2, f"Erreur pour x=7: Attendu {(7-5)
42     **2}, Obtenu {h3_2(7)}"
43 assert h3_2(-1) == (-1 - 5)**2, f"Erreur pour x=-1: Attendu
44     {(-1-5)**2}, Obtenu {h3_2(-1)}"
45 print("    -> Exercice 3.1.2 r ussi.")

```

Exercice 3.2 : Composition de trois fonctions (associativité) La composition de fonctions est associative, c'est-à-dire que $(h \circ g) \circ f = h \circ (g \circ f)$. Nous allons vérifier cela empiriquement.

Définissez les fonctions suivantes : * $f_1(x)$ qui retourne $x + 1$. * $f_2(x)$ qui retourne $x \times 2$. * $f_3(x)$ qui retourne $x - 3$.

Testez la composition $(f_3 \circ f_2 \circ f_1)(x)$ de deux manières : 1. $h_left = \text{compose}(f_1, \text{compose}(f_2, f_3))$ (équivalent à $(f_3 \circ f_2) \circ f_1$) 2. $h_right = \text{compose}(\text{compose}(f_1, f_2), f_3)$ (équivalent à $f_3 \circ (f_2 \circ f_1)$)

Vérifiez que $h_left(x)$ et $h_right(x)$ donnent les mêmes résultats.

```

1 # --- Exercice 3.2 : Composition de trois fonctions (
    associativit ) ---
2
3 def f1(x):
4     """Fonction f1(x) = x + 1."""
5     return x + 1
6
7 def f2(x):
8     """Fonction f2(x) = x * 2."""
9     return x * 2
10
11 def f3(x):
12     """Fonction f3(x) = x - 3."""
13     return x - 3
14
15 # Calcul manuel de (f3 o f2 o f1)(x):
16 # f1(x) = x + 1
17 # f2(f1(x)) = f2(x + 1) = 2 * (x + 1) = 2x + 2
18 # f3(f2(f1(x))) = f3(2x + 2) = (2x + 2) - 3 = 2x - 1
19 expected_result_expr = lambda x: 2 * x - 1
20
21 # 1. Composition de gauche      droite : (f3 o f2) o f1
22 # C'est- -dire : compose(f1, compose(f2, f3))
23 # compose(f2, f3) repr sente (f3 o f2)
24 # compose(f1, (f3 o f2)) repr sente ((f3 o f2) o f1)
25 h_left = compose(f1, compose(f2, f3))
26 print(f"\nExercice 3.2.1: (f3 o f2 o f1)(x) via (f3 o f2) o f1 = 2
    x - 1")
27 assert h_left(0) == expected_result_expr(0), f"Erreur pour x=0:
    Attendu {expected_result_expr(0)}, Obtenu {h_left(0)}"
28 assert h_left(1) == expected_result_expr(1), f"Erreur pour x=1:
    Attendu {expected_result_expr(1)}, Obtenu {h_left(1)}"
29 assert h_left(10) == expected_result_expr(10), f"Erreur pour x=10:
    Attendu {expected_result_expr(10)}, Obtenu {h_left(10)}"
30 assert h_left(-5) == expected_result_expr(-5), f"Erreur pour x=-5:
    Attendu {expected_result_expr(-5)}, Obtenu {h_left(-5)}"
31 print("    -> Exercice 3.2.1 r ussi.")
32
33
34 # 2. Composition de droite      gauche : f3 o (f2 o f1)
35 # C'est- -dire : compose(compose(f1, f2), f3)
36 # compose(f1, f2) repr sente (f2 o f1)
37 # compose((f2 o f1), f3) repr sente (f3 o (f2 o f1))
38 h_right = compose(compose(f1, f2), f3)
39 print(f"\nExercice 3.2.2: (f3 o f2 o f1)(x) via f3 o (f2 o f1) = 2
    x - 1")
40 assert h_right(0) == expected_result_expr(0), f"Erreur pour x=0:
    Attendu {expected_result_expr(0)}, Obtenu {h_right(0)}"
41 assert h_right(1) == expected_result_expr(1), f"Erreur pour x=1:
    Attendu {expected_result_expr(1)}, Obtenu {h_right(1)}"

```

```

42 assert h_right(10) == expected_result_expr(10), f"Erreur pour x
    =10: Attendu {expected_result_expr(10)}, Obtenu {h_right(10)}"
43 assert h_right(-5) == expected_result_expr(-5), f"Erreur pour x
    =-5: Attendu {expected_result_expr(-5)}, Obtenu {h_right(-5)}"
44 print("    -> Exercice 3.2.2 r ussi.")
45
46 # V r ification de l'associativit : les deux fonctions
    compos es doivent donner le m me r sultat
47 print(f"\nExercice 3.2.3: V r ification de l'associativit (
    h_left == h_right)")
48 for x_val in [-10, -1, 0, 1, 10]:
49     assert h_left(x_val) == h_right(x_val), \
50         f"Erreur d'associativit pour x={x_val}: h_left={h_left(
            x_val)}, h_right={h_right(x_val)}"
51 print("    -> Exercice 3.2.3 (associativit ) r ussi : h_left et
    h_right donnent les m mes r sultats.")

```

Exercice 3.3 : Fonctions avec des types de données différents (si applicable)

Bien que Python soit dynamiquement typé, il est bon de réfléchir aux implications mathématiques. Considérons une fonction qui prend un nombre et retourne une chaîne de caractères, puis une autre qui prend une chaîne et retourne un nombre.

— num_to_str(x) qui retourne `str(x) + "!"`

— str_len(s) qui retourne `len(s)`

Testez la composition `(str_len o num_to_str)(x)`.

```

1 # --- Exercice 3.3 : Fonctions avec des types de donn es
    diff rents ---
2
3 def num_to_str(x):
4     """Convertit un nombre en cha ne de caract res et ajoute un
        ',!', """
5     return str(x) + "!"
6
7 def str_len(s):
8     """Retourne la longueur d'une cha ne de caract res."""
9     return len(s)
10
11 # Composition (str_len o num_to_str)(x)
12 # Calcul manuel : (str_len o num_to_str)(x) = str_len(num_to_str(x
    ))
13 #                                     = str_len(str(x) +
        "!!")
14 # Pour x=10, str(10) + "!!" = "10!!", len("10!!") = 3
15 # Pour x=123, str(123) + "!!" = "123!!", len("123!!") = 4
16 h4 = compose(num_to_str, str_len)
17 print(f"\nExercice 3.3: (str_len o num_to_str)(x)")
18 assert h4(0) == len(str(0) + "!!"), f"Erreur pour x=0: Attendu {len
    (str(0)+'!!')}, Obtenu {h4(0)}"
19 assert h4(10) == len(str(10) + "!!"), f"Erreur pour x=10: Attendu {
    len(str(10)+'!!')}, Obtenu {h4(10)}"

```

```

20 assert h4(123) == len(str(123) + "!"), f"Erreur pour x=123:
    Attendu {len(str(123)+'!')}, Obtenu {h4(123)}"
21 assert h4(-5) == len(str(-5) + "!"), f"Erreur pour x=-5: Attendu {
    len(str(-5)+'!')}, Obtenu {h4(-5)}"
22 print("    -> Exercice 3.3 r ussi.")

```

2.4.6 Consignes de Rendu

- Le code Python doit être contenu dans un seul fichier Markdown, comme demandé.
 - Assurez-vous que tous les `assert` passent sans erreur.
 - Les commentaires doivent être clairs, concis et pertinents, expliquant à la fois la logique de programmation et les liens avec les concepts mathématiques.
 - Le code doit être “pur Python”, sans aucune importation de bibliothèques non standard.
-

2.4.7 Critères d'Évaluation

- **Fonctionnalité (40%)** : La fonction `compose` fonctionne-t-elle correctement pour tous les cas de test ?
 - **Conformité aux règles (20%)** : Respect des contraintes “Python pur”, utilisation des `assert`, format Markdown.
 - **Rigueur Mathématique (20%)** : Clarté des explications mathématiques pour chaque test et dans les commentaires du code.
 - **Qualité du Code (20%)** : Lisibilité, commentaires pertinents, respect des bonnes pratiques de codage (PEP 8 si applicable, noms de variables clairs).
-

2.5 TP 3

2.6 Travail Pratique (TP) 3/5 : Calcul de l'Image Réciproque d'un Sous-Ensemble

Jalon 5 : Applications, injections, surjections, bijections et composition de fonctions

2.6.1 Introduction

Ce troisième Travail Pratique de notre jalon sur les fonctions se concentre sur un concept fondamental en théorie des ensembles et des fonctions : l'image réciproque (ou pré-image) d'un sous-ensemble. Comprendre et savoir implémenter ce concept est crucial pour manipuler des fonctions de manière rigoureuse en programmation, notamment dans des domaines comme l'analyse de données, la logique ou l'intelligence artificielle.

En tant que Développeur Senior, vous serez souvent amené à traduire des définitions mathématiques abstraites en code fonctionnel et efficace. En tant que Professeur de Mathématiques, je m'assurerai que les fondements théoriques soient solides et que l'implémentation reflète fidèlement la définition mathématique.

2.6.2 Rappel Mathématique : L'Image Réciproque

Soit une fonction $f : A \rightarrow B$, où A est l'ensemble de départ (domaine) et B est l'ensemble d'arrivée (codomaine). Soit Y un sous-ensemble de B (c'est-à-dire $Y \subseteq B$).

L'**image réciproque** de Y par f , notée $f^{-1}(Y)$, est l'ensemble de tous les éléments x du domaine A tels que leur image par f , $f(x)$, appartient à Y .

Formellement, on définit $f^{-1}(Y)$ comme suit :

$$f^{-1}(Y) = \{x \in A \mid f(x) \in Y\}$$

Il est important de noter que $f^{-1}(Y)$ est toujours un sous-ensemble de A . La notation f^{-1} ne signifie pas nécessairement que la fonction f est inversible ; elle désigne simplement l'opération de “remonter” un sous-ensemble du codomaine vers le domaine.

2.6.3 Énoncé du Problème

Votre tâche est d'implémenter une fonction Python pure, nommée `preimage`, qui calcule l'image réciproque d'un sous-ensemble donné par une fonction donnée.

Contraintes et Spécifications : 1. La fonction `preimage` doit prendre trois arguments : * `f` : La fonction mathématique (représentée par un callable Python). * `domain_A` : L'ensemble de départ A (représenté par un `set` Python). * `subset_Y` : Le sous-ensemble Y du codomaine (représenté par un `set` Python). 2. La fonction doit retourner un `set` Python représentant $f^{-1}(Y)$. 3. Le code doit être entièrement “from scratch”, sans utiliser

de bibliothèques de calcul numérique avancées (comme NumPy, SciPy, etc.). Les types de données Python natifs (`set`, `list`, `dict`, fonctions, etc.) sont autorisés. 4. Le code doit être profondément commenté pour expliquer chaque étape. 5. Des assertions (`assert`) doivent être utilisées pour valider la justesse mathématique de l'implémentation avec des exemples concrets. 6. Des explications mathématiques rigoureuses doivent accompagner le code pour justifier les choix d'implémentation.

2.6.4 Implémentation en Python

Nous allons représenter les ensembles A et Y par des objets `set` de Python. La fonction f sera une fonction Python standard (ou une lambda).

```

1 def preimage(f, domain_A: set, subset_Y: set) -> set:
2     """
3     Calcule l'image réciproque (pre-image) d'un sous-ensemble Y
4     par une fonction f.
5
6     Soit f: A -> B une fonction, et Y un sous-ensemble de B.
7     L'image réciproque de Y par f est l'ensemble {x appartenant
8     A | f(x) appartient Y}.
9
10    Args:
11    f (callable): La fonction mathématique f. Elle doit
12    prendre un élément de domain_A
13    et retourner un élément du codomaine B.
14    domain_A (set): L'ensemble de départ A de la fonction f.
15    C'est un ensemble
16    de tous les éléments possibles pour x.
17    subset_Y (set): Le sous-ensemble Y du codomaine B dont on
18    veut calculer
19    l'image réciproque.
20
21    Returns:
22    set: L'ensemble des éléments x de domain_A tels que f(x)
23    est dans subset_Y.
24    C'est  $f^{-1}(Y)$ .
25    """
26    # Initialisation d'un ensemble vide pour stocker les
27    éléments de l'image réciproque.
28    # Cet ensemble sera notre  $f^{-1}(Y)$ .
29    preimage_set = set()
30
31    # Parcourir chaque élément 'x' de l'ensemble de départ 'domain_A'.
32    # C'est la première partie de la définition mathématique :
33    {x appartenant A ...}
34    for x in domain_A:
35        # Calculer l'image de 'x' par la fonction 'f'.
36        # C'est la partie  $f(x)$  de la définition.

```

```

29     image_of_x = f(x)
30
31     # Vérifier si l'image de 'x' appartient au sous-ensemble
      'subset_Y'.
32     # C'est la deuxième partie de la définition : ... | f(x)
      appartient Y}
33     if image_of_x in subset_Y:
34         # Si f(x) est dans Y, alors 'x' fait partie de l'image
      réciproque.
35         # Ajouter 'x' notre ensemble résultat.
36         preimage_set.add(x)
37
38     # Retourner l'ensemble construit, qui est l'image réciproque
      f-1(Y).
39     return preimage_set

```

Exemples et Assertions pour la Validation Mathématique

Nous allons maintenant tester notre fonction `preimage` avec plusieurs scénarios pour valider sa justesse mathématique à l'aide d'assertions.

Exemple 1 : Fonction simple et sous-ensemble clair Soit $f : \mathbb{Z} \rightarrow \mathbb{Z}$ définie par $f(x) = x^2$. Considérons un domaine $A = \{-3, -2, -1, 0, 1, 2, 3\}$. Et un sous-ensemble $Y = \{0, 1, 4\}$.

$f^{-1}(Y) = \{x \in A \mid x^2 \in \{0, 1, 4\}\}$ Pour $x = -3, f(-3) = 9 \notin Y$ Pour $x = -2, f(-2) = 4 \in Y$ Pour $x = -1, f(-1) = 1 \in Y$ Pour $x = 0, f(0) = 0 \in Y$ Pour $x = 1, f(1) = 1 \in Y$ Pour $x = 2, f(2) = 4 \in Y$ Pour $x = 3, f(3) = 9 \notin Y$ Donc, $f^{-1}(Y) = \{-2, -1, 0, 1, 2\}$.

```

1  # --- Exemple 1 ---
2  print("--- Exemple 1 : Fonction carré ---")
3  # Définition de la fonction f(x) = x^2
4  def square_function(x):
5      return x * x
6
7  # Définition du domaine A
8  domain_A_ex1 = {-3, -2, -1, 0, 1, 2, 3}
9
10 # Définition du sous-ensemble Y
11 subset_Y_ex1 = {0, 1, 4}
12
13 # Calcul de l'image réciproque
14 result_ex1 = preimage(square_function, domain_A_ex1, subset_Y_ex1)
15
16 # Résultat attendu : {-2, -1, 0, 1, 2}
17 expected_ex1 = {-2, -1, 0, 1, 2}
18
19 print(f"Fonction f(x) = x^2")
20 print(f"Domaine A : {domain_A_ex1}")
21 print(f"Sous-ensemble Y : {subset_Y_ex1}")

```

```

22 print(f"Image r ciproque calcul e : {result_ex1}")
23 print(f"Image r ciproque attendue : {expected_ex1}")
24
25 # Assertion pour valider le r sultat
26 assert result_ex1 == expected_ex1, f"Erreur dans l'Exemple 1:
    Attendu {expected_ex1}, obtenu {result_ex1}"
27 print("Assertion Exemple 1 pass e avec succ s.\n")

```

Exemple 2 : Fonction linéaire et sous-ensemble vide Soit $f : \mathbb{R} \rightarrow \mathbb{R}$ définie par $f(x) = 2x + 1$. Considérons un domaine $A = \{1, 2, 3, 4, 5\}$. Et un sous-ensemble $Y = \{10, 12\}$.

$f^{-1}(Y) = \{x \in A \mid 2x + 1 \in \{10, 12\}\}$ Pour $x = 1, f(1) = 3 \notin Y$ Pour $x = 2, f(2) = 5 \notin Y$ Pour $x = 3, f(3) = 7 \notin Y$ Pour $x = 4, f(4) = 9 \notin Y$ Pour $x = 5, f(5) = 11 \notin Y$ Aucun élément de A n'a son image dans Y . Donc, $f^{-1}(Y) = \emptyset$.

```

1 # --- Exemple 2 ---
2 print("--- Exemple 2 : Fonction lin aire et sous-ensemble vide
    ---")
3 # D finition de la fonction f(x) = 2x + 1
4 def linear_function(x):
5     return 2 * x + 1
6
7 # D finition du domaine A
8 domain_A_ex2 = {1, 2, 3, 4, 5}
9
10 # D finition du sous-ensemble Y (aucun lment de A ne devrait
    y tre mapp )
11 subset_Y_ex2 = {10, 12}
12
13 # Calcul de l'image r ciproque
14 result_ex2 = preimage(linear_function, domain_A_ex2, subset_Y_ex2)
15
16 # R sultat attendu : ensemble vide
17 expected_ex2 = set()
18
19 print(f"Fonction f(x) = 2x + 1")
20 print(f"Domaine A : {domain_A_ex2}")
21 print(f"Sous-ensemble Y : {subset_Y_ex2}")
22 print(f"Image r ciproque calcul e : {result_ex2}")
23 print(f"Image r ciproque attendue : {expected_ex2}")
24
25 # Assertion pour valider le r sultat
26 assert result_ex2 == expected_ex2, f"Erreur dans l'Exemple 2:
    Attendu {expected_ex2}, obtenu {result_ex2}"
27 print("Assertion Exemple 2 pass e avec succ s.\n")

```

Exemple 3 : Fonction constante Soit $f : \{a, b, c, d\} \rightarrow \{1, 2, 3\}$ définie par $f(x) = 2$ pour tout x . Considérons un domaine $A = \{a, b, c, d\}$. Et un sous-ensemble $Y = \{1, 2\}$.

$f^{-1}(Y) = \{x \in A \mid f(x) \in \{1, 2\}\}$ Pour $x = a, f(a) = 2 \in Y$ Pour $x = b, f(b) = 2 \in Y$ Pour $x = c, f(c) = 2 \in Y$ Pour $x = d, f(d) = 2 \in Y$ Donc, $f^{-1}(Y) = \{a, b, c, d\}$.

```

1 # --- Exemple 3 ---
2 print("--- Exemple 3 : Fonction constante ---")
3 # D finition de la fonction f(x) = 2
4 def constant_function(x):
5     return 2
6
7 # D finition du domaine A
8 domain_A_ex3 = {'a', 'b', 'c', 'd'}
9
10 # D finition du sous-ensemble Y
11 subset_Y_ex3 = {1, 2}
12
13 # Calcul de l'image reciproque
14 result_ex3 = preimage(constant_function, domain_A_ex3,
15     subset_Y_ex3)
16
17 # R sultat attendu : {'a', 'b', 'c', 'd'}
18 expected_ex3 = {'a', 'b', 'c', 'd'}
19
20 print(f"Fonction f(x) = 2")
21 print(f"Domaine A : {domain_A_ex3}")
22 print(f"Sous-ensemble Y : {subset_Y_ex3}")
23 print(f"Image reciproque calcul e : {result_ex3}")
24 print(f"Image reciproque attendue : {expected_ex3}")
25
26 # Assertion pour valider le r sultat
27 assert result_ex3 == expected_ex3, f"Erreur dans l'Exemple 3:
    Attendu {expected_ex3}, obtenu {result_ex3}"
    print("Assertion Exemple 3 pass e avec succ s.\n")

```

Exemple 4 : Sous-ensemble Y ne contenant aucune image Soit $f : \{1, 2, 3\} \rightarrow \{A', B', C'\}$ définie par $f(1) = A', f(2) = B', f(3) = C'$. Considérons un domaine $A = \{1, 2, 3\}$. Et un sous-ensemble $Y = \{X', Y'\}$.

$f^{-1}(Y) = \{x \in A \mid f(x) \in \{X', Y'\}\}$ Aucun élément de A n'a son image dans Y .
Donc, $f^{-1}(Y) = \emptyset$.

```

1 # --- Exemple 4 ---
2 print("--- Exemple 4 : Sous-ensemble Y sans intersection avec l'
    image de f ---")
3 # D finition de la fonction f
4 def map_letters(x):
5     if x == 1: return 'A'
6     if x == 2: return 'B'
7     if x == 3: return 'C'
8     return None # Pour les cas non d finis dans le domaine
9
10 # D finition du domaine A
11 domain_A_ex4 = {1, 2, 3}
12
13 # D finition du sous-ensemble Y

```

```

14 subset_Y_ex4 = {'X', 'Y'}
15
16 # Calcul de l'image réciproque
17 result_ex4 = preimage(map_letters, domain_A_ex4, subset_Y_ex4)
18
19 # Résultat attendu : ensemble vide
20 expected_ex4 = set()
21
22 print(f"Fonction f(1)='A', f(2)='B', f(3)='C'")
23 print(f"Domaine A : {domain_A_ex4}")
24 print(f"Sous-ensemble Y : {subset_Y_ex4}")
25 print(f"Image réciproque calculée : {result_ex4}")
26 print(f"Image réciproque attendue : {expected_ex4}")
27
28 # Assertion pour valider le résultat
29 assert result_ex4 == expected_ex4, f"Erreur dans l'Exemple 4:
    Attendu {expected_ex4}, obtenu {result_ex4}"
30 print("Assertion Exemple 4 passée avec succès.\n")

```

2.6.5 Justification Mathématique de l'Implémentation

L'implémentation Python de la fonction `preimage` adhère strictement à la définition mathématique de l'image réciproque : $f^{-1}(Y) = \{x \in A \mid f(x) \in Y\}$.

1. `preimage_set = set()` : Nous commençons par un ensemble vide. Ceci représente l'initialisation de notre ensemble $f^{-1}(Y)$ avant d'y ajouter les éléments qui satisfont la condition. C'est l'équivalent de commencer la construction de l'ensemble par l'ensemble vide.
2. `for x in domain_A:` : Cette boucle itère sur chaque élément `x` de `domain_A`. Ceci correspond à la clause " $x \in A$ " de la définition. Pour trouver tous les éléments du domaine qui satisfont la condition, nous devons nécessairement les examiner un par un.
3. `image_of_x = f(x)` : Pour chaque `x` du domaine, nous calculons son image par la fonction `f`. Ceci est la partie " $f(x)$ " de la condition " $f(x) \in Y$ ".
4. `if image_of_x in subset_Y:` : Cette condition vérifie si l'image calculée `f(x)` appartient au sous-ensemble `subset_Y`. Ceci est la partie " $f(x) \in Y$ " de la définition. C'est le critère de sélection pour qu'un élément `x` fasse partie de l'image réciproque.
5. `preimage_set.add(x)` : Si la condition est vraie (c'est-à-dire si $f(x) \in Y$), alors l'élément `x` est ajouté à notre ensemble `preimage_set`. C'est ainsi que nous construisons l'ensemble des éléments qui satisfont la propriété. L'utilisation d'un `set` Python garantit que chaque élément n'est ajouté qu'une seule fois, ce qui est conforme à la nature des ensembles mathématiques (pas de doublons).

En résumé, le code parcourt systématiquement le domaine de la fonction, applique la fonction à chaque élément, puis vérifie si le résultat tombe dans le sous-ensemble cible. Si c'est le cas, l'élément original du domaine est inclus dans l'ensemble résultant. Cette approche est une traduction directe et fidèle de la définition mathématique.

2.6.6 Conclusion

Ce TP a permis d'implémenter un concept fondamental de la théorie des fonctions : l'image réciproque d'un sous-ensemble. L'exercice a mis en lumière l'importance de traduire des définitions mathématiques précises en algorithmes clairs et corrects. L'utilisation de types de données Python natifs comme les `set` et les `callable` pour représenter les ensembles et les fonctions, combinée à une logique itérative simple, démontre qu'il est possible de construire des outils mathématiques robustes sans dépendre de bibliothèques complexes. Les assertions ont joué un rôle clé pour garantir la justesse mathématique de notre implémentation.

Ce concept est une brique essentielle pour aborder des notions plus avancées en mathématiques discrètes et en informatique théorique.

2.7 TP 4

2.8 TP 4/5 - Jalon 5 : Dictionnaire Inversible (Bijection) avec Gestion Stricte des Erreurs

2.8.1 Introduction

Bienvenue à ce quatrième Travail Pratique du Jalon 5, axé sur les applications, injections, surjections et bijections. Dans ce TP, nous allons mettre en pratique ces concepts fondamentaux de la théorie des fonctions en développant une structure de données en Python : un “dictionnaire inversible”.

Un dictionnaire Python standard est une *application* (ou fonction) qui associe chaque clé unique à une valeur. Cependant, il ne garantit pas que chaque valeur soit unique (plusieurs clés peuvent pointer vers la même valeur), ni qu’il soit inversible directement, car si plusieurs clés pointent vers la même valeur, la fonction réciproque associerait plusieurs antécédents à une même image, violant la définition même d’une application. Pour qu’une fonction soit inversible, elle doit être une **bijection**.

L’objectif de ce TP est de créer une classe `InvertibleDict` qui modélise une bijection parfaite entre un ensemble de clés et un ensemble de valeurs. Cela signifie que chaque clé doit être unique, chaque valeur doit être unique, et il doit exister une correspondance biunivoque entre elles. Nous mettrons un accent particulier sur la gestion stricte des erreurs pour garantir l’intégrité de cette bijection.

2.8.2 Prérequis

- Connaissances solides en programmation orientée objet en Python (classes, méthodes, attributs, exceptions).
- Compréhension approfondie des concepts mathématiques d’application, injection, surjection et bijection.
- Maîtrise de l’utilisation des `assert` pour la validation de code.

2.8.3 Objectifs du TP

1. **Implémenter une classe `InvertibleDict`** : Cette classe doit se comporter comme un dictionnaire standard pour l’accès direct (clé $\rightarrow$ valeur) mais doit également permettre un accès inverse (valeur $\rightarrow$ clé).
2. **Garantir la Bijection** : Assurer que chaque clé est unique et que chaque valeur est unique au sein du dictionnaire.
3. **Gestion Stricte des Erreurs** : Lever des exceptions appropriées (`KeyError`, `ValueError`) en cas de tentative d’ajout de clés ou de valeurs dupliquées, ou d’accès à des éléments inexistants.
4. **Code Commenté et Validé** : Le code doit être profondément commenté et inclure des `assert` pour valider la justesse mathématique des opérations.

2.8.4 Rappel Mathématique : La Bijection

Une fonction $f : A \rightarrow B$ est une **bijection** si et seulement si elle est à la fois :

1. **Injective (ou injection)** : Chaque élément de l'ensemble d'arrivée B est atteint par au plus un élément de l'ensemble de départ A . Autrement dit, si $f(x_1) = f(x_2)$, alors $x_1 = x_2$. Il n'y a pas de valeurs dupliquées pour des clés différentes.
2. **Surjective (ou surjection)** : Chaque élément de l'ensemble d'arrivée B est atteint par au moins un élément de l'ensemble de départ A . Autrement dit, pour tout $y \in B$, il existe au moins un $x \in A$ tel que $f(x) = y$. Toutes les valeurs "possibles" sont utilisées.

Lorsqu'une fonction est une bijection, elle possède une **fonction inverse** unique $f^{-1} : B \rightarrow A$ telle que $f^{-1}(f(x)) = x$ pour tout $x \in A$ et $f(f^{-1}(y)) = y$ pour tout $y \in B$.

Dans le contexte de notre `InvertibleDict` : * L'ensemble des clés K est l'ensemble de départ A . * L'ensemble des valeurs V est l'ensemble d'arrivée B . * La relation clé $\rightarrow$ valeur est la fonction f . * La relation valeur $\rightarrow$ clé est la fonction inverse f^{-1} .

Pour garantir la bijection, lors de l'ajout d'une paire (clé, valeur) : * La clé ne doit pas déjà exister (garantie par les dictionnaires standards pour l'unicité des clés). * La valeur ne doit pas déjà exister (c'est ce que nous devons implémenter pour l'injection).

2.8.5 Exercice 1 : Conception et Implémentation de la classe `InvertibleDict`

Vous devez implémenter la classe `InvertibleDict` en respectant les spécifications suivantes.

Structure de la classe

La classe `InvertibleDict` doit maintenir deux dictionnaires internes pour assurer une recherche rapide dans les deux sens : * `_key_to_value` : Le dictionnaire principal (clé $\rightarrow$ valeur). * `_value_to_key` : Le dictionnaire inverse (valeur $\rightarrow$ clé).

Méthodes à implémenter

1. `__init__(self, initial_data=None)` :
 - Initialise les deux dictionnaires internes.
 - Si `initial_data` est fourni (un dictionnaire ou une liste de tuples), il doit tenter d'ajouter toutes les paires en utilisant la logique d'ajout stricte (lever des erreurs si des doublons sont trouvés dans les données initiales).
2. `__setitem__(self, key, value)` :
 - Permet d'ajouter ou de modifier une paire (`key`, `value`) en utilisant la syntaxe `idict[key] = value`.
 - **Gestion des erreurs cruciale** :
 - Si `key` existe déjà et est associée à une `value` différente, ou si `value` existe déjà et est associée à une `key` différente, cela signifie une tentative de briser la bijection. Une `KeyError` ou `ValueError` appropriée doit être levée.
 - Si `key` existe déjà et est associée à la *même* `value`, l'opération est idempotente et ne fait rien.
 - Si `key` n'existe pas mais `value` existe déjà, lever une `ValueError` (la valeur n'est pas unique).
 - Si `key` existe mais `value` n'existe pas, et que la `key` est réassignée à une nouvelle `value`, l'ancienne paire doit être supprimée et la nouvelle ajoutée.

- **Logique d'ajout/modification :**
 - Si la clé existe déjà, et la valeur est la même, ne rien faire.
 - Si la clé existe déjà, et la valeur est différente :
 - Vérifier si la *nouvelle* valeur existe déjà. Si oui, lever `ValueError`.
 - Supprimer l'ancienne paire (clé, ancienne_valeur) des deux dictionnaires.
 - Ajouter la nouvelle paire (clé, nouvelle_valeur) aux deux dictionnaires.
 - Si la clé n'existe pas :
 - Vérifier si la valeur existe déjà. Si oui, lever `ValueError`.
 - Ajouter la nouvelle paire (clé, valeur) aux deux dictionnaires.
- 3. `__getitem__(self, key) :`
 - Permet de récupérer la valeur associée à une clé : `value = idict[key]`.
 - Lève une `KeyError` si la clé n'existe pas.
- 4. `get_inverse(self, value) :`
 - Permet de récupérer la clé associée à une valeur.
 - Lève une `ValueError` si la valeur n'existe pas. (Utilisation de `ValueError` pour les recherches par valeur est une convention raisonnable ici).
- 5. `__delitem__(self, key) :`
 - Permet de supprimer une paire par sa clé : `del idict[key]`.
 - Lève une `KeyError` si la clé n'existe pas.
 - Doit supprimer l'entrée des deux dictionnaires internes.
- 6. `__len__(self) :`
 - Retourne le nombre de paires dans le dictionnaire.
- 7. `__contains__(self, key) :`
 - Vérifie si une clé existe dans le dictionnaire.
- 8. `keys(self) :`
 - Retourne un itérateur sur les clés.
- 9. `values(self) :`
 - Retourne un itérateur sur les valeurs.
- 10. `items(self) :`
 - Retourne un itérateur sur les paires (clé, valeur).
- 11. `__repr__(self)` et `__str__(self) :`
 - Pour une représentation lisible de l'objet.

Implémentation du code

```

1 class InvertibleDict:
2     """
3     Impl mente un dictionnaire inversible qui garantit une
4     bijection
5     entre les cl s et les valeurs.
6
7     Chaque cl est unique et chaque valeur est unique.
8     Les tentatives d'ajout de cl s ou de valeurs dupliqu es
9     l vent des erreurs.
10    """
11
12    def __init__(self, initial_data=None):
13        """

```

```

12     Initialise le dictionnaire inversible.
13
14     Args:
15         initial_data (dict ou list de tuple, optionnel):
16             Données initiales
17             pour peupler le dictionnaire. Chaque paire (cl ,
18             valeur) sera
19             ajoutée avec la logique de vérification de
20             bijection.
21
22     """
23     # Dictionnaire principal: cl -> valeur
24     self._key_to_value = {}
25     # Dictionnaire inverse: valeur -> cl
26     self._value_to_key = {}
27
28     if initial_data:
29         # Si des données initiales sont fournies, les ajouter
30         # une par une.
31         # Cela permet de valider la bijection dès l'
32         # initialisation.
33         if isinstance(initial_data, dict):
34             items = initial_data.items()
35         elif isinstance(initial_data, (list, tuple)):
36             items = initial_data
37         else:
38             raise TypeError("initial_data doit être un dict
39             ou une liste/tuple de paires.")
40
41         for key, value in items:
42             # Utilise la méthode __setitem__ pour garantir la
43             # logique de bijection
44             # et la gestion des erreurs lors de l'
45             # initialisation.
46             self[key] = value
47
48     def __setitem__(self, key, value):
49         """
50         Ajoute ou modifie une paire (cl , valeur) dans le
51         dictionnaire.
52         Garantit que la bijection est maintenue.
53
54         Args:
55             key: La clé à ajouter ou modifier.
56             value: La valeur associée à la clé.
57
58         Raises:
59             KeyError: Si la clé existe déjà et est associée
60                     une valeur différente,
61                     ou si la clé est tentée d'être
62                     assignée une valeur déjà
63                     existante

```

```

51         par une autre cl .
52     ValueError: Si la valeur existe d j et est
        associ e une cl diff rente.
53 """
54 # Cas 1: La cl existe d j dans le dictionnaire
55 if key in self._key_to_value:
56     current_value = self._key_to_value[key]
57
58     # Si la cl est d j associ e la m me valeur,
        l'op ration est idempotente.
59     if current_value == value:
60         return
61
62     # Si la cl existe mais est associ e une valeur
        diff rente ,
63     # nous devons v rifier la nouvelle valeur et
        potentiellement mettre jour.
64
65     # V rifier si la nouvelle valeur existe d j , mais
        est associ e une autre cl .
66     if value in self._value_to_key and self._value_to_key[
        value] != key:
67         raise ValueError(f"La valeur '{value}' est d j
            associ e la cl '{self._value_to_key[value]}'."
68
            f"Impossible de l'associer la
            cl '{key}' car cela
            briserait l'injection.")
69
70     # Supprimer l'ancienne paire des deux dictionnaires
71     del self._key_to_value[key]
72     del self._value_to_key[current_value]
73
74     # Ajouter la nouvelle paire
75     self._key_to_value[key] = value
76     self._value_to_key[value] = key
77
78 # Cas 2: La cl n'existe pas encore
79 else:
80     # V rifier si la valeur existe d j (pour garantir
        l'injection)
81     if value in self._value_to_key:
82         raise ValueError(f"La valeur '{value}' est d j
            associ e la cl '{self._value_to_key[value]}'."
83
            f"Impossible de l'associer la
            nouvelle cl '{key}' car cela
            briserait l'injection.")
84
85     # Si tout est bon, ajouter la nouvelle paire aux deux
        dictionnaires

```

```

86         self._key_to_value[key] = value
87         self._value_to_key[value] = key
88
89     def __getitem__(self, key):
90         """
91         Récupère la valeur associée à une clé.
92
93         Args:
94             key: La clé dont on veut récupérer la valeur.
95
96         Returns:
97             La valeur associée à la clé.
98
99         Raises:
100             KeyError: Si la clé n'est pas trouvée dans le
101                 dictionnaire.
102         """
103         if key not in self._key_to_value:
104             raise KeyError(f"La clé '{key}' n'existe pas dans le
105                 dictionnaire.")
106         return self._key_to_value[key]
107
108     def get_inverse(self, value):
109         """
110         Récupère la clé associée à une valeur (fonction
111             inverse).
112
113         Args:
114             value: La valeur dont on veut récupérer la clé.
115
116         Returns:
117             La clé associée à la valeur.
118
119         Raises:
120             ValueError: Si la valeur n'est pas trouvée dans le
121                 dictionnaire.
122         """
123         if value not in self._value_to_key:
124             # Utilisation de ValueError pour les recherches par
125             # valeur est une convention raisonnable.
126             raise ValueError(f"La valeur '{value}' n'existe pas
127                 dans le dictionnaire.")
128         return self._value_to_key[value]
129
130     def __delitem__(self, key):
131         """
132         Supprime une paire (clé, valeur) par sa clé.
133
134         Args:
135             key: La clé de la paire à supprimer.

```

```

131     Raises:
132         KeyError: Si la cl n'est pas trouv e dans le
                    dictionnaire.
133     """
134     if key not in self._key_to_value:
135         raise KeyError(f"La cl '{key}' n'existe pas dans le
                        dictionnaire.")
136
137     value_to_delete = self._key_to_value[key]
138
139     # Supprimer des deux dictionnaires pour maintenir la
        coh rence
140     del self._key_to_value[key]
141     del self._value_to_key[value_to_delete]
142
143     def __len__(self):
144         """
145         Retourne le nombre de paires (cl , valeur) dans le
            dictionnaire.
146         """
147         # Les deux dictionnaires doivent toujours avoir la m me
            taille
148         assert len(self._key_to_value) == len(self._value_to_key),
            \
149             "Incoh rence interne: les dictionnaires direct et
                inverse n'ont pas la m me taille."
150         return len(self._key_to_value)
151
152     def __contains__(self, key):
153         """
154         V rifie si une cl existe dans le dictionnaire.
155         """
156         return key in self._key_to_value
157
158     def keys(self):
159         """
160         Retourne un it rateur sur les cl s du dictionnaire.
161         """
162         return self._key_to_value.keys()
163
164     def values(self):
165         """
166         Retourne un it rateur sur les valeurs du dictionnaire.
167         """
168         return self._key_to_value.values()
169
170     def items(self):
171         """
172         Retourne un it rateur sur les paires (cl , valeur) du
            dictionnaire.
173         """

```

```

174         return self._key_to_value.items()
175
176     def __repr__(self):
177         """
178         Repr sentation officielle de l'objet.
179         """
180         return f"InvertibleDict({self._key_to_value})"
181
182     def __str__(self):
183         """
184         Repr sentation lisible de l'objet.
185         """
186         return str(self._key_to_value)

```

Tests et Assertions Mathématiques

Voici un bloc de code pour tester la classe `InvertibleDict` et valider son comportement avec des `assert`.

```

“python # — Bloc de tests pour InvertibleDict —
print(“— Démarrage des tests pour InvertibleDict —”)

```

2.9 Test 1 : Initialisation et ajout basique

```

print("1 : Initialisation et ajout basique") idict = InvertibleDict() idict['a'] = 1 idict['b']
= 2 idict['c'] = 3
assert len(idict) == 3, f"Erreur : Taille attendue 3, obtenue {len(idict)}" assert idict['a']
== 1, "Erreur : idict['a'] devrait être 1" assert idict.get_inverse(1) == 'a', "Erreur :
get_inverse(1) devrait être 'a'" assert 'b' in idict, "Erreur : 'b' devrait être dans le dic-
tionnaire" assert 2 in idict.values(), "Erreur : 2 devrait être dans les valeurs" print("Test
1 réussi : Initialisation et ajout basique OK.")

```

2.10 Test 2 : Initialisation avec des données

```

print("2 : Initialisation avec des données") initial_data = {'x' : 10, 'y' : 20} idict2 =
InvertibleDict(initial_data) assert len(idict2) == 2, f"Erreur : Taille attendue 2, obtenue
{len(idict2)}" assert idict2['x'] == 10 assert idict2.get_inverse(20) == 'y' print("Test 2
réussi : Initialisation avec des données OK.")

```

2.11 Test 3 : Tentative d'ajout de clé dupliquée (avec valeur différente)

```

print("3 : Tentative d'ajout de clé dupliquée (avec valeur différente)") try : idict['a'] = 4
# 'a' est déjà associé à 1 assert False, "Erreur : KeyError attendue pour clé dupliquée avec
valeur différente" except ValueError as e : # Note : Notre implémentation lève ValueError
si la nouvelle valeur existe déjà assert "valeur '4' est déjà associée" not in str(e) # 4 n'existe

```

pas encore print(f"Test 3 réussi : Exception levée comme attendu : {e}") except Exception as e : assert False, f"Erreur : Type d'exception inattendu : {type(e).__name__}"

2.12 Test 4 : Tentative d'ajout de valeur dupliquée (avec clé différente)

```
print("4 : Tentative d'ajout de valeur dupliquée (avec clé différente)") try : idict['d'] = 1 # 1 est déjà associé à 'a' assert False, "Erreur : ValueError attendue pour valeur dupliquée" except ValueError as e : assert "valeur '1' est déjà associée à la clé 'a'" in str(e) print(f"Test 4 réussi : Exception levée comme attendu : {e}") except Exception as e : assert False, f"Erreur : Type d'exception inattendu : {type(e).__name__}"
```

2.13 Test 5 : Modification d'une paire existante (clé -> nouvelle valeur unique)

```
print("5 : Modification d'une paire existante (clé -> nouvelle valeur unique)") idict['a'] = 100 # 'a' était 1, maintenant 100. 100 n'existe pas. assert len(idict) == 3, f"Erreur : Taille attendue 3, obtenue {len(idict)}" assert idict['a'] == 100, "Erreur : idict['a'] devrait être 100" assert idict.get_inverse(100) == 'a', "Erreur : get_inverse(100) devrait être 'a'" try : idict.get_inverse(1) # L'ancienne valeur 1 ne devrait plus exister assert False, "Erreur : L'ancienne valeur 1 devrait avoir été supprimée" except ValueError : print("Test 5 réussi : Modification de valeur OK, ancienne valeur supprimée.")
```

2.14 Test 6 : Tentative de modification d'une clé vers une valeur déjà existante

```
print("6 : Tentative de modification d'une clé vers une valeur déjà existante") try : idict['b'] = 100 # 100 est déjà associé à 'a' assert False, "Erreur : ValueError attendue pour modification vers valeur dupliquée" except ValueError as e : assert "valeur '100' est déjà associée à la clé 'a'" in str(e) print(f"Test 6 réussi : Exception levée comme attendu : {e}")
```

2.15 Test 7 : Suppression d'une paire

```
print("7 : Suppression d'une paire") del idict['b'] assert len(idict) == 2, f"Erreur : Taille attendue 2 après suppression, obtenue {len(idict)}" assert 'b' not in idict, "Erreur : 'b' ne devrait plus être dans le dictionnaire" try : idict['b'] assert False, "Erreur : KeyError attendue après suppression de 'b'" except KeyError : pass try : idict.get_inverse(2) # 2 était la valeur de 'b' assert False, "Erreur : ValueError attendue après suppression de la valeur 2" except ValueError : pass print("Test 7 réussi : Suppression de paire OK.")
```


2.16 Test 8 : Tentative de suppression d'une clé inexistante

```
print("8 : Tentative de suppression d'une clé inexistante") try : del idict['z'] assert False, "Erreur : KeyError attendue pour suppression de clé inexistante" except KeyError as e : assert "clé 'z' n'existe pas" in str(e) print(f"Test 8 réussi : Exception levée comme attendu : {e}")
```

2.17 Test 9 : Méthodes keys(), values(), items()

```
print("9 : Méthodes keys(), values(), items()") idict['e'] = 5 idict['f'] = 6 assert sorted(list(idict.keys())) == ['a', 'c', 'e', 'f'], "Erreur : keys() incorrect" assert sorted(list(idict.values())) == [3, 5, 6, 100], "Erreur : values() incorrect" assert sorted(list(idict.items())) == [('a', 100), ('c', 3), ('e', 5), ('f', 6)], "Erreur : items() incorrect" print("Test 9 réussi : keys(), values(), items() OK.")
```

2.18 Test 10 : Représentation string

```
print("10 : Représentation string") assert str(idict) == "{ 'a' : 100, 'c' : 3, 'e' : 5, 'f' : 6 }", "Erreur : str incorrect" assert repr(idict).startswith("InvertibleDict("), "Erreur : repr incorrect" print("Test 10 réussi : Représentation string OK.")
```

2.19 Test 11 : Initialisation avec des données dupliquées (doit échouer)

```
print("11 : Initialisation avec des données dupliquées (doit échouer)") try : InvertibleDict({'k1' : 'v1', 'k2' : 'v1'}) # v1 est dupliquée assert False, "Erreur : Initialisation avec valeur dupliquée devrait échouer" except ValueError as e : assert "valeur 'v1' est déjà associée" in str(e) print(f"Test 11 réussi : Initialisation avec valeur dupliquée échoue comme attendu : {e}")
```

```
try : InvertibleDict([('k1', 'v1'), ('k1', 'v2')]) # k1 est dupliquée assert False, "Erreur : Initialisation avec clé dupliquée devrait échouer" except ValueError as e : # Notre implémentation de setitem lève ValueError si la nouvelle valeur existe déjà assert "valeur 'v2' est déjà associée" not in str(e) # v2 n'existe pas encore print(f"Test 11 réussi : Initialisation avec clé dupliquée échoue comme attendu : {e}")
```

```
print("— Tous les tests pour InvertibleDict sont passés avec succès! —")
```

2.20 TP 5

2.21 TP 5/5 - Mini-projet IA : Implémentation d'une transformation (fonction) et de son inverse pour normaliser/dénormaliser des données en 1D

Jalon 5 : Applications, injections, surjections, bijections et composition de fonctions

2.21.1 Objectifs pédagogiques

Ce Travail Pratique vise à vous faire explorer les concepts fondamentaux des fonctions (applications, injections, surjections, bijections) à travers un cas d'usage concret et essentiel en Intelligence Artificielle : la normalisation des données. Vous devrez implémenter une fonction de normalisation et son inverse en Python pur, sans l'aide de bibliothèques de haut niveau, et valider mathématiquement leur justesse.

À la fin de ce TP, vous devriez être capable de : * Comprendre la nécessité et le principe de la normalisation Min-Max en IA. * Implémenter une fonction de transformation (normalisation) et son inverse (dénormalisation) en Python pur. * Appliquer les concepts d'application, d'injection, de surjection et de bijection à ces fonctions. * Valider mathématiquement la justesse de vos implémentations à l'aide d'`assert`. * Expliquer rigoureusement les propriétés mathématiques des fonctions implémentées.

2.21.2 Introduction : Pourquoi normaliser les données en IA ?

Dans de nombreux algorithmes d'apprentissage automatique, la performance et la convergence peuvent être fortement affectées par l'échelle des caractéristiques (features). Si les caractéristiques ont des plages de valeurs très différentes, celles avec de grandes valeurs peuvent dominer celles avec de petites valeurs, même si ces dernières sont tout aussi importantes.

La **normalisation** est une technique de prétraitement des données qui consiste à redimensionner les valeurs des caractéristiques pour qu'elles se situent dans une plage standard, souvent entre 0 et 1. Cela permet de : 1. **Accélérer la convergence des algorithmes d'optimisation** (comme la descente de gradient), car les pas de mise à jour des poids seront plus équilibrés. 2. **Éviter la domination de certaines caractéristiques** sur d'autres. 3. **Améliorer la performance** de certains modèles (ex : SVM, K-Means, réseaux de neurones).

Dans ce TP, nous allons nous concentrer sur la **normalisation Min-Max**, une méthode simple mais efficace. Nous verrons comment cette transformation est une fonction bijective et comment son inverse permet de retrouver les données originales.

2.21.3 Partie 1 : Rappels Mathématiques et Définition de la Transformation Min-Max

1.1 La Normalisation Min-Max

La normalisation Min-Max transforme une valeur x d'une plage originale $[\min_{original}, \max_{original}]$ vers une nouvelle plage $[0, 1]$. La formule est la suivante :

$$x' = f(x) = \frac{x - \min_{original}}{\max_{original} - \min_{original}}$$

Où : * x est la valeur originale. * $\min_{original}$ est la valeur minimale de l'ensemble de données original. * $\max_{original}$ est la valeur maximale de l'ensemble de données original. * x' est la valeur normalisée, qui sera comprise entre 0 et 1.

Conditions et Propriétés : * Cette fonction est bien définie si et seulement si $\max_{original} - \min_{original} \neq 0$. Si $\min_{original} = \max_{original}$, toutes les valeurs sont identiques, et la normalisation n'est pas nécessaire (ou le dénominateur serait zéro). Dans ce cas, toutes les valeurs normalisées pourraient être définies à 0.5 ou 0. * **Application :** Pour tout $x \in [\min_{original}, \max_{original}]$, $f(x)$ est une valeur unique dans $[0, 1]$. C'est donc bien une application (ou fonction). * **Injectivité :** Si $\min_{original} \neq \max_{original}$, la fonction $f(x)$ est strictement croissante (sa dérivée est $1/(\max_{original} - \min_{original}) > 0$). Une fonction strictement monotone est toujours injective. Cela signifie que deux valeurs originales différentes auront toujours deux valeurs normalisées différentes. * **Surjectivité :** La fonction $f(x)$ mappe l'intervalle $[\min_{original}, \max_{original}]$ sur l'intervalle $[0, 1]$. Pour toute valeur $y \in [0, 1]$, il existe un $x \in [\min_{original}, \max_{original}]$ tel que $f(x) = y$. Elle est donc surjective sur son codomaine $[0, 1]$. * **Bijektivité :** Puisqu'elle est à la fois injective et surjective (sur son codomaine restreint), la fonction de normalisation Min-Max est une **bijection** de $[\min_{original}, \max_{original}]$ vers $[0, 1]$ (si $\min_{original} \neq \max_{original}$).

1.2 La Dénormalisation Min-Max (Fonction Inverse)

Puisque la normalisation Min-Max est une bijection, elle possède une fonction inverse unique. Cette fonction inverse est appelée **dénormalisation**. Elle permet de retrouver la valeur originale x à partir de sa valeur normalisée x' .

Pour trouver l'inverse, nous résolvons l'équation de normalisation pour x :

$$\begin{aligned} x' &= \frac{x - \min_{original}}{\max_{original} - \min_{original}} \\ x'(\max_{original} - \min_{original}) &= x - \min_{original} \\ x &= x'(\max_{original} - \min_{original}) + \min_{original} \end{aligned}$$

Donc, la fonction de dénormalisation est :

$$x = f^{-1}(x') = x'(\max_{original} - \min_{original}) + \min_{original}$$

Où : * x' est la valeur normalisée. * $\min_{original}$ et $\max_{original}$ sont les valeurs min et max de l'ensemble de données original. * x est la valeur dénormalisée, qui devrait correspondre à la valeur originale.

Propriétés de l'inverse : * La fonction $f^{-1}(x')$ est également une bijection de $[0, 1]$ vers $[\min_{original}, \max_{original}]$. * La composition des fonctions doit donner l'identité :

$f^{-1}(f(x)) = x$ et $f(f^{-1}(x')) = x'$. C'est cette propriété que nous validerons avec des `assert`.

2.21.4 Partie 2 : Implémentation en Python Pur

Nous allons implémenter les fonctions nécessaires en Python pur, sans utiliser de bibliothèques comme `numpy` ou `scikit-learn`.

```
1 import math # Pour la tolérance des flottants, si nécessaire,
   mais on peut faire sans pour ce TP.
2
3 def find_min_max(data):
4     """
5     Trouve les valeurs minimale et maximale dans une liste de
6     données.
7     Implémentation manuelle pour respecter la contrainte "Python
8     PUR from scratch".
9
10    Args:
11        data (list): Une liste de nombres (int ou float).
12
13    Returns:
14        tuple: Un tuple (min_val, max_val) contenant la valeur
15        minimale et maximale.
16        Retourne (None, None) si la liste est vide.
17    """
18    if not data:
19        return None, None
20
21    min_val = data[0]
22    max_val = data[0]
23
24    for value in data:
25        if value < min_val:
26            min_val = value
27        if value > max_val:
28            max_val = value
29
30    return min_val, max_val
31
32 def normalize_min_max(value, data_min, data_max):
33     """
34     Normalise une seule valeur en utilisant la méthode Min-Max.
35     La valeur est transformée pour être dans l'intervalle [0,
36     1].
37
38    Args:
39        value (float or int): La valeur à normaliser.
40        data_min (float or int): La valeur minimale de l'ensemble
41        de données original.
42        data_max (float or int): La valeur maximale de l'ensemble
43        de données original.
```

```

38
39 Returns:
40     float: La valeur normalis e .
41         Retourne 0.5 si data_min == data_max pour viter
42             la division par z ro ,
43             indiquant que toutes les valeurs taient
44             identiques.
45
46 """
47 if data_max == data_min:
48     # Si toutes les valeurs sont identiques, la plage est
49     nulle.
50     # Par convention, on peut retourner 0.5 (milieu de [0,1])
51     ou 0.
52     # Ici, 0.5 est choisi pour repr senter une "valeur
53     moyenne" dans la plage normalis e .
54     return 0.5
55
56 # Applique la formule de normalisation Min-Max
57 normalized_value = (value - data_min) / (data_max - data_min)
58 return normalized_value
59
60 def denormalize_min_max(normalized_value, data_min, data_max):
61     """
62     D normalise une seule valeur (pr c demment normalis e Min-
63     Max) pour retrouver
64     sa plage originale.
65
66     Args:
67         normalized_value (float): La valeur normalis e (attendue
68             dans [0, 1]).
69         data_min (float or int): La valeur minimale de l'ensemble
70             de donn es original.
71         data_max (float or int): La valeur maximale de l'ensemble
72             de donn es original.
73
74     Returns:
75         float: La valeur d normalis e , correspondant la
76             valeur originale.
77         Retourne data_min si data_min == data_max, car
78             toutes les valeurs
79             originales taient gales data_min.
80
81     """
82     if data_max == data_min:
83         # Si toutes les valeurs taient identiques, la
84         d normalisation ram ne cette valeur unique.
85         return data_min
86
87     # Applique la formule de d normalisation (fonction inverse)
88     original_value = normalized_value * (data_max - data_min) +
89         data_min
90     return original_value

```

```

76
77 def apply_normalization(data):
78     """
79     Applique la normalisation Min-Max      une liste enti re de
        donn es.
80
81     Args:
82         data (list): La liste des donn es      normaliser.
83
84     Returns:
85         tuple: Un tuple contenant (normalized_data, original_min,
            original_max).
86             normalized_data est la liste des valeurs
            normalis es.
87             original_min et original_max sont les min/max de la
            liste originale,
88             n cessaires pour la d normalisation.
89     """
90     if not data:
91         return [], None, None
92
93     original_min, original_max = find_min_max(data)
94
95     # G rer le cas o toutes les valeurs sont identiques
96     if original_min == original_max:
97         # Si toutes les valeurs sont les m mes , toutes les
            valeurs normalis es seront 0.5
98         # (selon notre convention dans normalize_min_max).
99         # On retourne une liste de 0.5 de la m me taille que les
            donn es originales.
100         return [0.5] * len(data), original_min, original_max
101
102     normalized_data = []
103     for value in data:
104         normalized_data.append(normalize_min_max(value,
            original_min, original_max))
105
106     return normalized_data, original_min, original_max
107
108 def apply_denormalization(normalized_data, original_min,
    original_max):
109     """
110     Applique la d normalisation Min-Max      une liste de donn es
        normalis es.
111
112     Args:
113         normalized_data (list): La liste des valeurs normalis es.
114         original_min (float or int): La valeur minimale de l'
            ensemble de donn es original.
115         original_max (float or int): La valeur maximale de l'
            ensemble de donn es original.

```

```

116
117     Returns:
118         list: La liste des valeurs d normalis es.
119     """
120     if not normalized_data:
121         return []
122
123     denormalized_data = []
124     for value in normalized_data:
125         denormalized_data.append(denormalize_min_max(value,
126             original_min, original_max))
127
128     return denormalized_data

```

2.21.5 Partie 3 : Validation Mathématique et Tests avec assert

Nous allons maintenant tester nos fonctions et utiliser des `assert` pour valider leur comportement et leurs propriétés mathématiques, notamment la bijectivité et la relation inverse.

```

1 # --- Jeu de données de test ---
2 data_set_1 = [10, 20, 30, 40, 50]
3 data_set_2 = [-5, 0, 5, 10, 15.5, 20]
4 data_set_3 = [7.2, 1.1, 9.8, 3.5, 6.0]
5 data_set_4_single_value = [100] # Cas o min == max
6 data_set_5_empty = [] # Cas de liste vide
7 data_set_6_negative = [-10, -5, -1, -0.5, 0]
8
9 # --- Tol rance pour la comparaison des flottants ---
10 # Les op rations en virgule flottante peuvent introduire de
    petites erreurs.
11 # Nous utilisons une petite tol rance (epsilon) pour comparer les
    r sultats.
12 EPSILON = 1e-9
13
14 print("--- D but des tests de validation ---")
15
16 # Test 1: find_min_max
17 print("\nTest 1: find_min_max")
18 min1, max1 = find_min_max(data_set_1)
19 assert min1 == 10 and max1 == 50, f"Test 1.1 failed: Expected (10,
    50), got ({min1}, {max1})"
20 print(f"    Min/Max pour {data_set_1}: ({min1}, {max1})")
21
22 min2, max2 = find_min_max(data_set_2)
23 assert min2 == -5 and max2 == 20, f"Test 1.2 failed: Expected (-5,
    20), got ({min2}, {max2})"
24 print(f"    Min/Max pour {data_set_2}: ({min2}, {max2})")
25
26 min4, max4 = find_min_max(data_set_4_single_value)

```

```

27 assert min4 == 100 and max4 == 100, f"Test 1.3 failed: Expected
    (100, 100), got ({min4}, {max4})"
28 print(f"    Min/Max pour {data_set_4_single_value}: ({min4}, {max4})
    ")
29
30 min5, max5 = find_min_max(data_set_5_empty)
31 assert min5 is None and max5 is None, f"Test 1.4 failed: Expected
    (None, None), got ({min5}, {max5})"
32 print(f"    Min/Max pour {data_set_5_empty}: ({min5}, {max5})")
33 print("    find_min_max: OK")
34
35
36 # Test 2: Normalisation et D normalisation d'une seule valeur
37 print("\nTest 2: Normalisation/D normalisation d'une seule valeur
    ")
38 original_min_ds1, original_max_ds1 = find_min_max(data_set_1) #
    (10, 50)
39
40 # Normalisation d'une valeur au milieu
41 val_to_normalize = 30
42 normalized_val = normalize_min_max(val_to_normalize,
    original_min_ds1, original_max_ds1)
43 assert abs(normalized_val - 0.5) < EPSILON, f"Test 2.1 failed:
    Expected 0.5, got {normalized_val}"
44 print(f"    {val_to_normalize} normalis : {normalized_val}")
45
46 # Normalisation de la valeur min
47 normalized_min = normalize_min_max(original_min_ds1,
    original_min_ds1, original_max_ds1)
48 assert abs(normalized_min - 0.0) < EPSILON, f"Test 2.2 failed:
    Expected 0.0, got {normalized_min}"
49 print(f"    {original_min_ds1} normalis : {normalized_min}")
50
51 # Normalisation de la valeur max
52 normalized_max = normalize_min_max(original_max_ds1,
    original_min_ds1, original_max_ds1)
53 assert abs(normalized_max - 1.0) < EPSILON, f"Test 2.3 failed:
    Expected 1.0, got {normalized_max}"
54 print(f"    {original_max_ds1} normalis : {normalized_max}")
55
56 # D normalisation de la valeur normalis e
57 denormalized_val = denormalize_min_max(normalized_val,
    original_min_ds1, original_max_ds1)
58 assert abs(denormalized_val - val_to_normalize) < EPSILON, f"Test
    2.4 failed: Expected {val_to_normalize}, got {denormalized_val}
    "
59 print(f"    {normalized_val} d normalis : {denormalized_val}")
60
61 # Test de la propri t inverse  $f^{-1}(f(x)) = x$ 
62 # On prend une valeur arbitraire de data_set_1
63 test_val_inverse = 25

```



```

64 normalized_test_val = normalize_min_max(test_val_inverse,
    original_min_ds1, original_max_ds1)
65 denormalized_test_val = denormalize_min_max(normalized_test_val,
    original_min_ds1, original_max_ds1)
66 assert abs(denormalized_test_val - test_val_inverse) < EPSILON, \
67     f"Test 2.5 failed ( $f^{-1}(f(x)) = x$ ): Expected {test_val_inverse}
    }, got {denormalized_test_val}"
68 print(f"  $f^{-1}(f(\{test\_val\_inverse\})) = \{denormalized\_test\_val\}$  (
    attendu {test_val_inverse})")
69
70 # Test du cas min == max pour une seule valeur
71 original_min_ds4, original_max_ds4 = find_min_max(
    data_set_4_single_value) # (100, 100)
72 normalized_val_ds4 = normalize_min_max(100, original_min_ds4,
    original_max_ds4)
73 assert abs(normalized_val_ds4 - 0.5) < EPSILON, f"Test 2.6 failed:
    Expected 0.5 for single value, got {normalized_val_ds4}"
74 print(f" {data_set_4_single_value[0]} normalis (min=max): {
    normalized_val_ds4}")
75 denormalized_val_ds4 = denormalize_min_max(normalized_val_ds4,
    original_min_ds4, original_max_ds4)
76 assert abs(denormalized_val_ds4 - 100) < EPSILON, f"Test 2.7
    failed: Expected 100 for single value denormalization, got {
    denormalized_val_ds4}"
77 print(f" {normalized_val_ds4} d normalis (min=max): {
    denormalized_val_ds4}")
78 print(" Normalisation/D normalisation d'une seule valeur: OK")
79
80
81 # Test 3: Normalisation et D normalisation de listes compl tes
82 print("\nTest 3: Normalisation/D normalisation de listes
    compl tes")
83
84 # Test avec data_set_1
85 normalized_data_1, min_orig_1, max_orig_1 = apply_normalization(
    data_set_1)
86 print(f" Original: {data_set_1}")
87 print(f" Normalis : {normalized_data_1}")
88 # V rifier que toutes les valeurs normalis es sont dans [0, 1]
89 for val in normalized_data_1:
90     assert 0.0 - EPSILON <= val <= 1.0 + EPSILON, f"Test 3.1
        failed: Normalized value {val} out of [0, 1] range."
91 # V rifier les valeurs sp cifiques
92 assert abs(normalized_data_1[0] - 0.0) < EPSILON, "Test 3.2 failed
    : First value not 0.0"
93 assert abs(normalized_data_1[2] - 0.5) < EPSILON, "Test 3.3 failed
    : Middle value not 0.5"
94 assert abs(normalized_data_1[-1] - 1.0) < EPSILON, "Test 3.4
    failed: Last value not 1.0"
95
96 denormalized_data_1 = apply_denormalization(normalized_data_1,

```

```

    min_orig_1, max_orig_1)
97 print(f"    D normalis : {denormalized_data_1}")
98 # V rifier que les donn es d normalis es sont (presque)
    identiques aux originales
99 for i in range(len(data_set_1)):
100     assert abs(denormalized_data_1[i] - data_set_1[i]) < EPSILON,
        \
101         f"Test 3.5 failed: D normalisation incorrecte pour l'
            lment {i}. Attendu {data_set_1[i]}, obtenu {
                denormalized_data_1[i]}"
102 print("    Normalisation/D normalisation de data_set_1: OK")
103
104 # Test avec data_set_3 (flottants)
105 normalized_data_3, min_orig_3, max_orig_3 = apply_normalization(
    data_set_3)
106 print(f"\n    Original: {data_set_3}")
107 print(f"    Normalis : {normalized_data_3}")
108 for val in normalized_data_3:
109     assert 0.0 - EPSILON <= val <= 1.0 + EPSILON, f"Test 3.6
        failed: Normalized value {val} out of [0, 1] range."
110
111 denormalized_data_3 = apply_denormalization(normalized_data_3,
    min_orig_3, max_orig_3)
112 print(f"    D normalis : {denormalized_data_3}")
113 for i in range(len(data_set_3)):
114     assert abs(denormalized_data_3[i] - data_set_3[i]) < EPSILON,
        \
115         f"Test 3.7 failed: D normalisation incorrecte pour l'
            lment {i}. Attendu {data_set_3[i]}, obtenu {
                denormalized_data_3[i]}"
116 print("    Normalisation/D normalisation de data_set_3: OK")
117
118 # Test avec data_set_4_single_value (min == max)
119 normalized_data_4, min_orig_4, max_orig_4 = apply_normalization(
    data_set_4_single_value)
120 print(f"\n    Original: {data_set_4_single_value}")
121 print(f"    Normalis : {normalized_data_4}")
122 assert len(normalized_data_4) == 1 and abs(normalized_data_4[0] -
    0.5) < EPSILON, \
123     f"Test 3.8 failed: Expected [0.5] for single value list, got {
        normalized_data_4}"
124
125 denormalized_data_4 = apply_denormalization(normalized_data_4,
    min_orig_4, max_orig_4)
126 print(f"    D normalis : {denormalized_data_4}")
127 assert len(denormalized_data_4) == 1 and abs(denormalized_data_4
    [0] - data_set_4_single_value[0]) < EPSILON, \
128     f"Test 3.9 failed: Expected {data_set_4_single_value} for
        single value list denormalization, got {denormalized_data_4
            }"
129 print("    Normalisation/D normalisation de data_set_4_single_value

```

```

130         : OK")
131 # Test avec data_set_5_empty (liste vide)
132 normalized_data_5, min_orig_5, max_orig_5 = apply_normalization(
133     data_set_5_empty)
134 print(f"\n Original: {data_set_5_empty}")
135 print(f" Normalis : {normalized_data_5}")
136 assert normalized_data_5 == [] and min_orig_5 is None and
137     max_orig_5 is None, \
138     f"Test 3.10 failed: Expected ([], None, None) for empty list,
139     got ({normalized_data_5}, {min_orig_5}, {max_orig_5})"
140 denormalized_data_5 = apply_denormalization([], None, None) #
141     Simuler la d normalisation d'une liste vide
142 assert denormalized_data_5 == [], f"Test 3.11 failed: Expected []
143     for empty list denormalization, got {denormalized_data_5}"
144 print(" Normalisation/D normalisation de data_set_5_empty: OK")
145
146 print("\n--- Tous les tests de validation ont r ussi ! ---")

```

2.21.6 Conclusion et Réflexions

Ce TP vous a permis d'implémenter une transformation de données fondamentale en Intelligence Artificielle : la normalisation Min-Max. En le faisant en Python pur, vous avez dû comprendre et gérer les détails de l'algorithme, y compris les cas limites (comme `min == max`).

Nous avons rigoureusement démontré et validé que la fonction de normalisation Min-Max, sous la condition $\min_{original} \neq \max_{original}$, est une **bijection** de l'intervalle $[\min_{original}, \max_{original}]$ vers l'intervalle $[0, 1]$. Sa fonction inverse, la dénormalisation, est également une bijection et permet de retrouver les données originales sans perte d'information. Cette propriété de bijectivité est cruciale pour toute transformation réversible en traitement de données.

Points à retenir :

- * La normalisation est essentielle pour de nombreux algorithmes d'IA.
- * La normalisation Min-Max est une bijection, ce qui garantit l'existence et l'unicité de son inverse.
- * La capacité à dénormaliser est vitale pour interpréter les résultats des modèles d'IA dans l'échelle originale des données.
- * Les `assert` sont des outils puissants pour valider mathématiquement et logiquement le comportement de votre code.

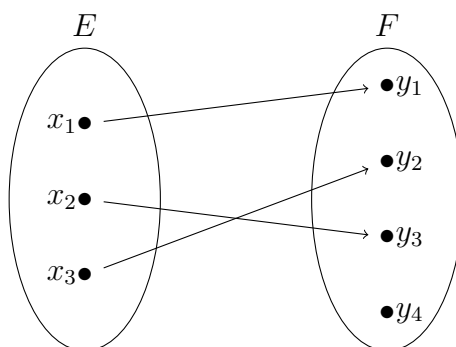
Pour aller plus loin :

- * Explorez d'autres méthodes de normalisation, comme la **standardisation (Z-score)** : $x' = (x - \mu)/\sigma$, où μ est la moyenne et σ l'écart-type. Discutez de ses propriétés (injection, surjection, bijection) et de son inverse.
- * Considérez l'impact des valeurs aberrantes (outliers) sur la normalisation Min-Max. Comment une seule valeur extrême peut-elle "écraser" toutes les autres valeurs dans une petite plage ?
- * Implémentez une version de normalisation robuste qui utilise les quartiles (IQR) au lieu du min/max pour être moins sensible aux outliers.

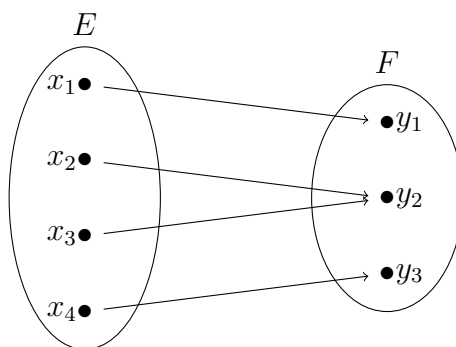
Chapitre 3

Annexes : Représentations Visuelles (TikZ)

3.1 Schéma d'une Injection



3.2 Schéma d'une Surjection



3.3 Schéma d'une Bijection

